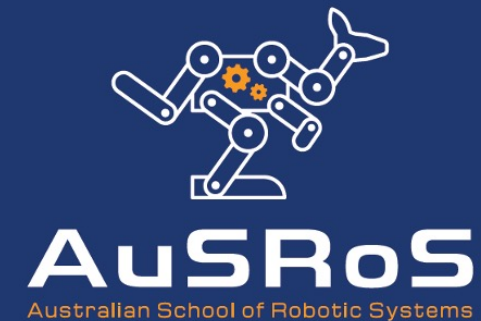


AUSROS 2025

Localisation and Mapping



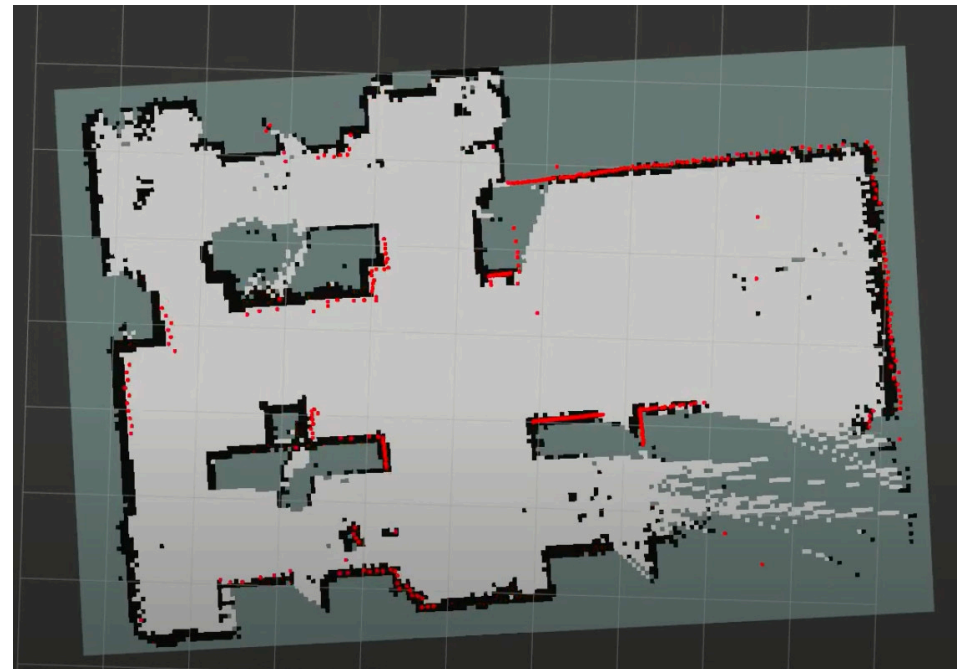
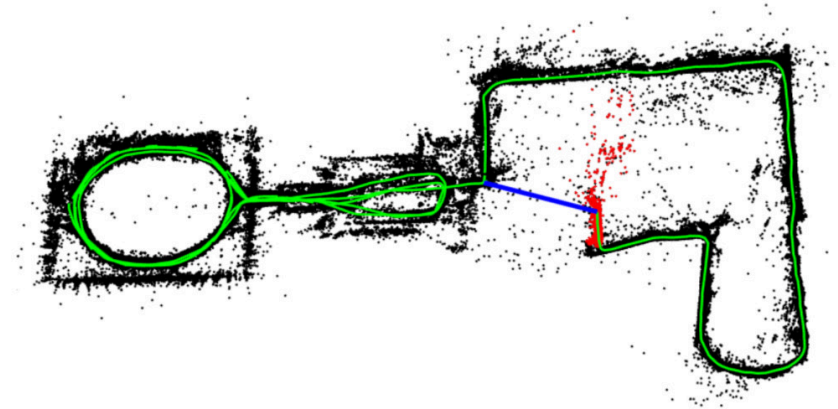
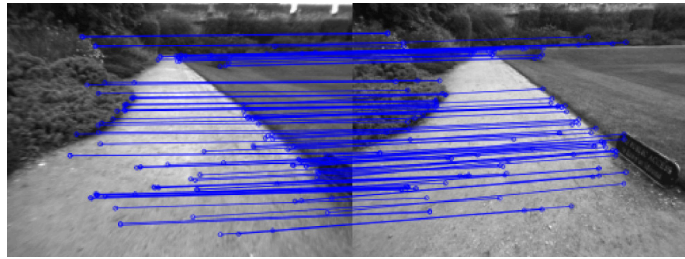
Prof. Will Browne

With thanks to Dr. Mitch Bryson
Australian Centre for Robotics University of Sydney
Plus, Peter, Feras, Tobi, Niko, Roland...



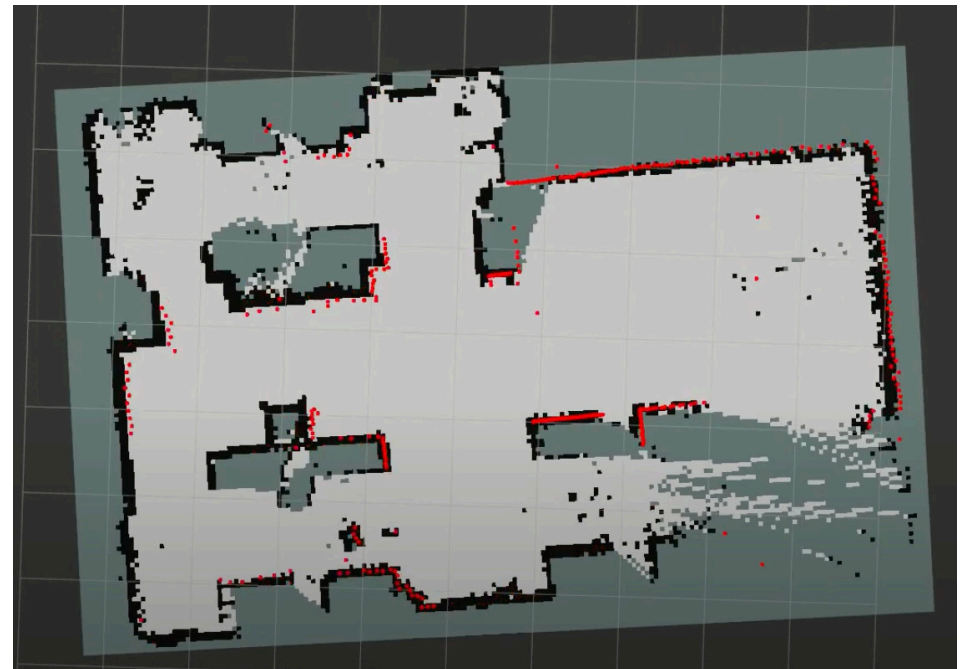
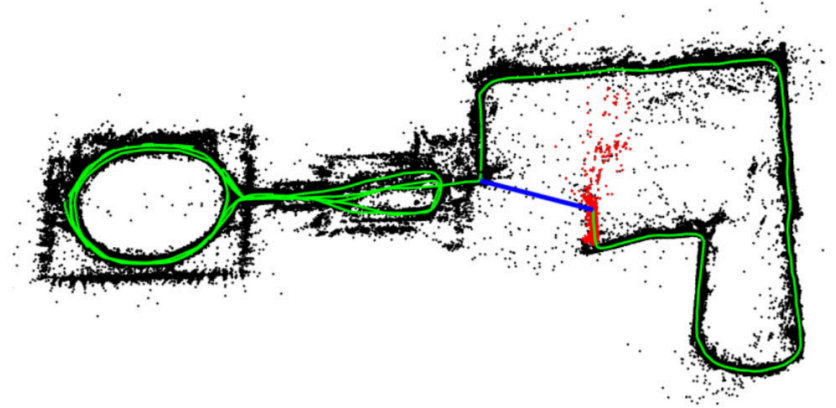
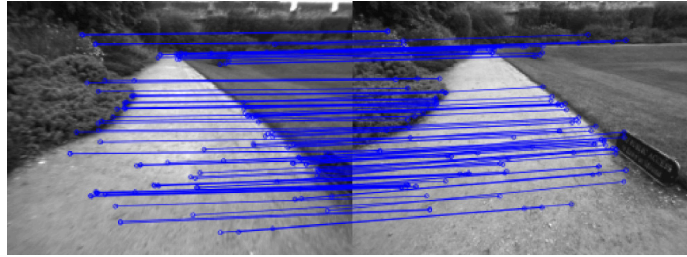
Introduction

- **Localisation:** the art/science of determining one's position and orientation (pose) with respect to a known frame of reference
- **Mapping:** the art/science of determining the spatial configuration of the surrounding environment with respect to a known frame of reference



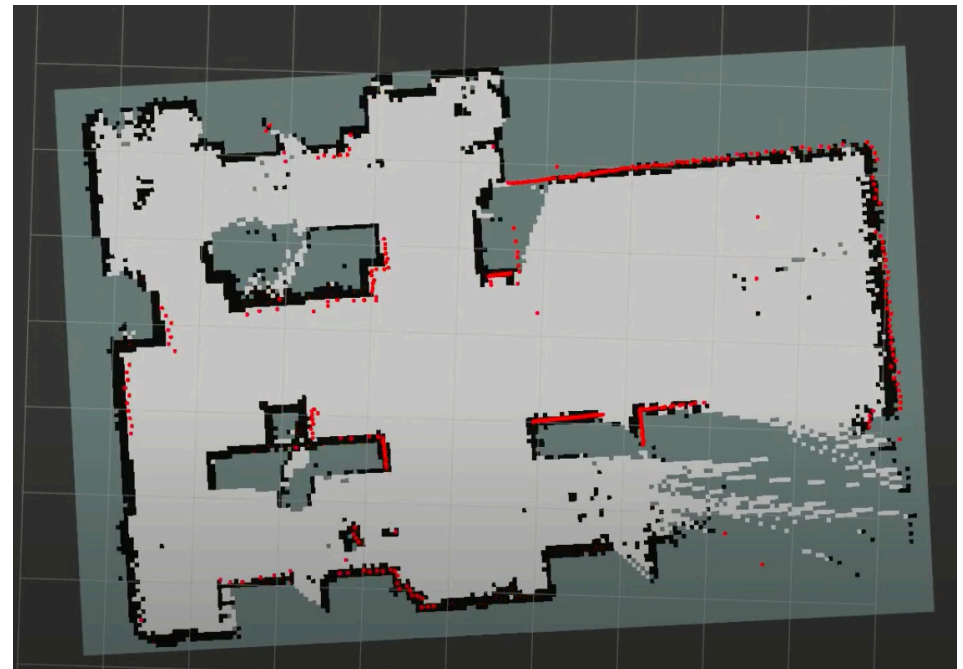
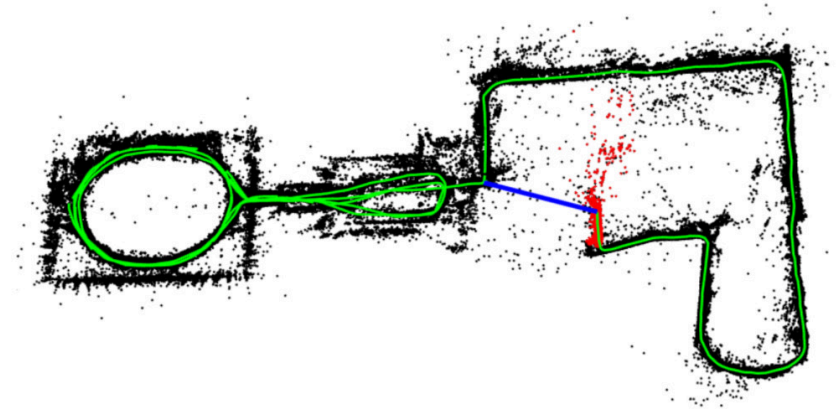
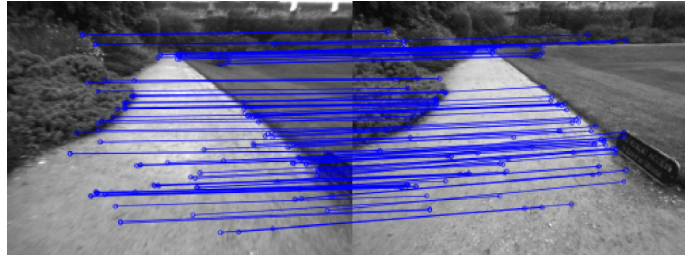
Introduction

- **Localisation:** the art/science of determining one's position and orientation (pose) with respect to a known frame of reference
- **Mapping:** the art/science of determining the spatial configuration of the surrounding environment with respect to a known frame of reference

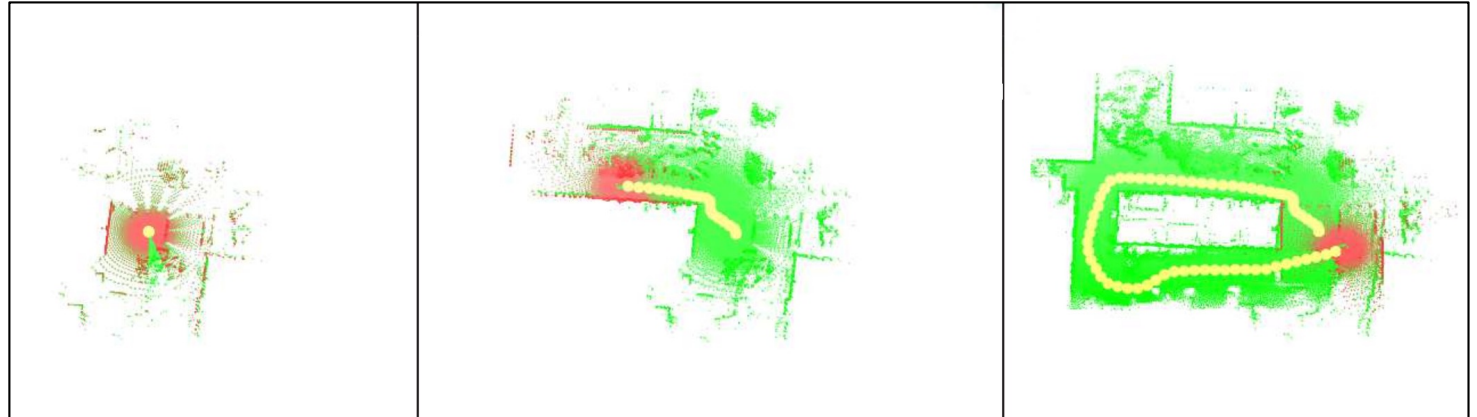


Introduction

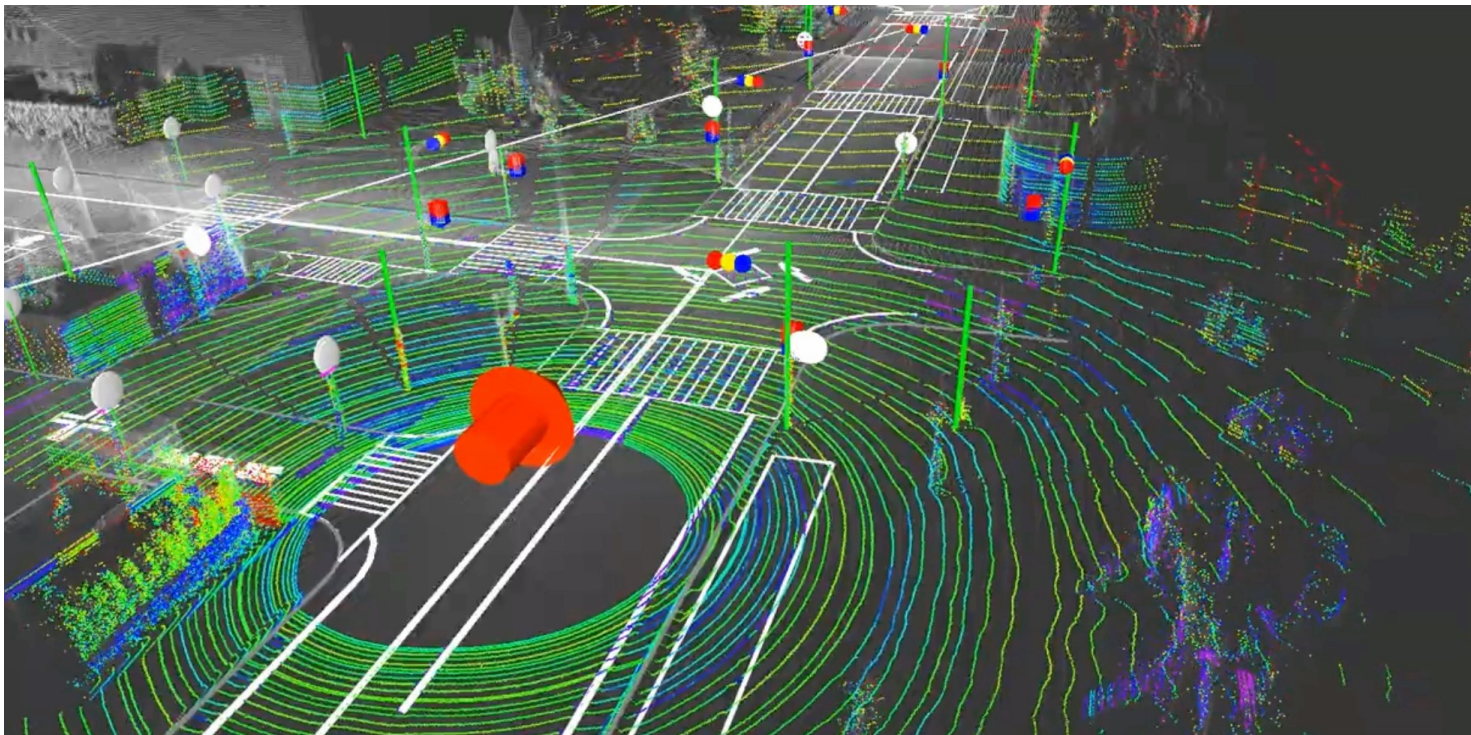
- **Localisation:** the art/science of determining one's position and orientation (pose) with respect to a known frame of reference
- **Mapping:** the art/science of determining the spatial configuration of the surrounding environment with respect to a known frame of reference
- Why localise and map?
 - Robot control, stability
 - Obstacle avoidance
 - Navigation from one location to another
 - Spatial information itself as an end goal for robotic exploration



Examples: LiDAR-based localization and mapping



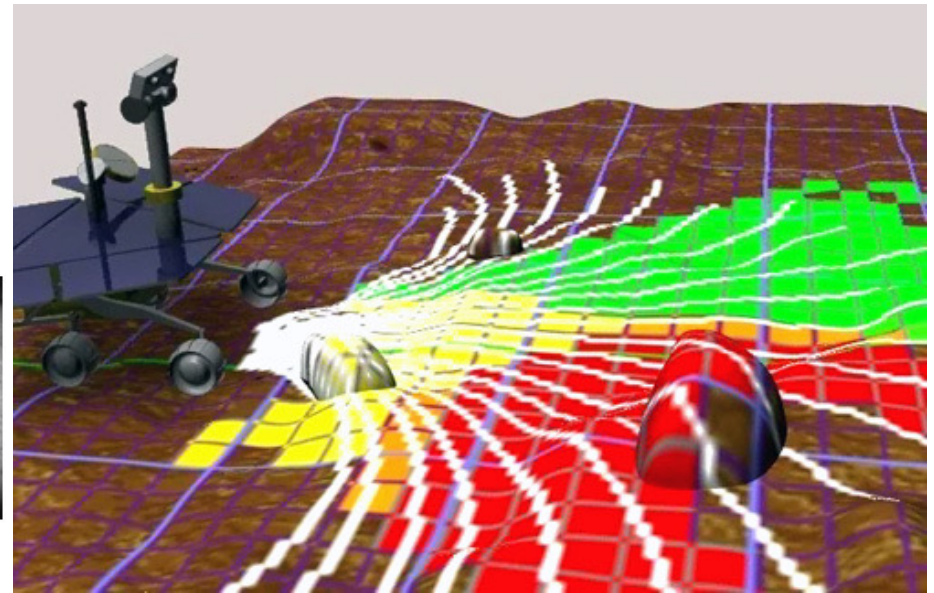
Bormann et. al., Globally Consistent 3D Mapping with Scan Matching, 2007



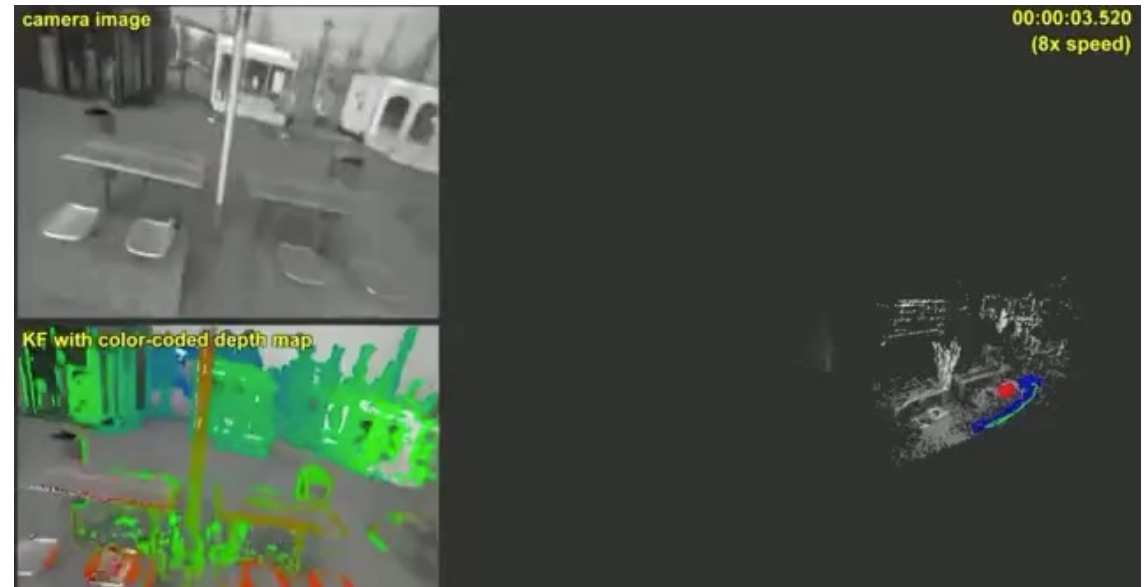
Autoware.AI LiDAR localization using NDT scan matching

Examples: Vision-based Localisation and Navigation

S. Goldberg, M. Maimone and L. Matthies, "Stereo Vision and Rover Navigation Software for Planetary Exploration", IEEE Aerospace Conference, 2012



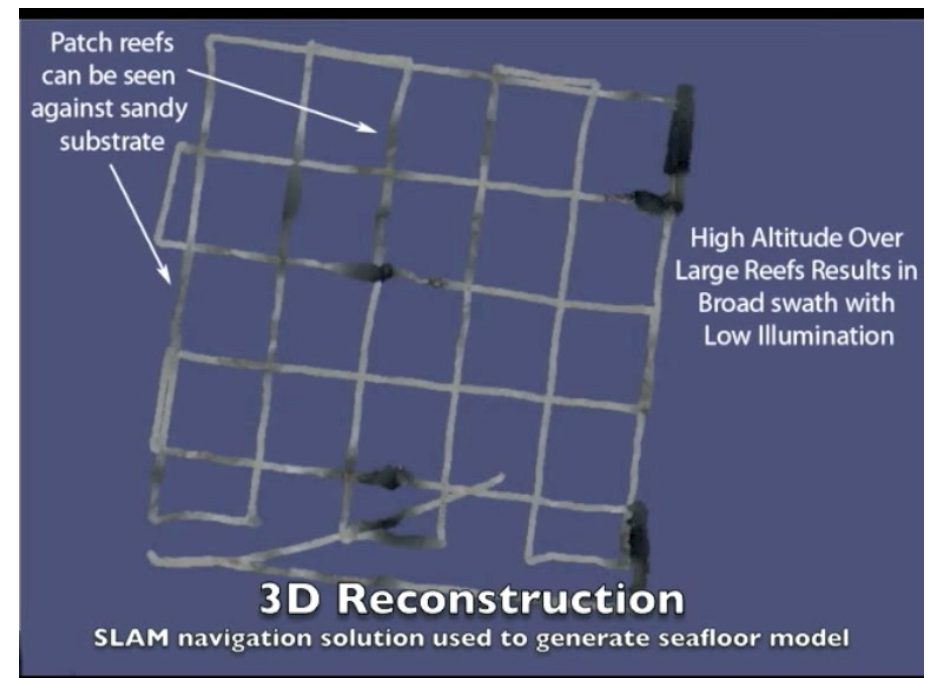
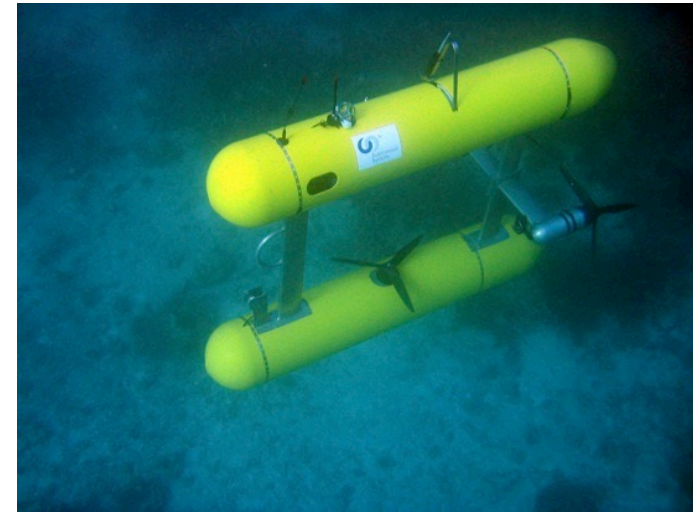
G. Conte and P. Doherty, "An Integrated UAV Navigation System Based on Aerial Image Matching", IEEE Aerospace Conference 2007.



Jakob Engel and Thomas Schops and Daniel Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM", ECCV 2014

Examples: 3D mapping using vision/structure-from-motion

Furukawa et. al., "Towards Internet-scale Multi-view Stereo", CVPR, 2010



Johnson-Roberson et. al., "Generation and visualization of large-scale three-dimensional reconstructions from underwater robotic surveys", JFR, 2010

Place Recognition

- Recognising a place the robot has previously visited using sensor data
 - under potentially large domain shifts (perspective shift, day vs. night, seasonal etc.)
 - also reject false positives containing repetitious scene elements
- Typical approaches segment the environment and assign descriptors (handcrafted or learnt), perform matching in feature and/or location space

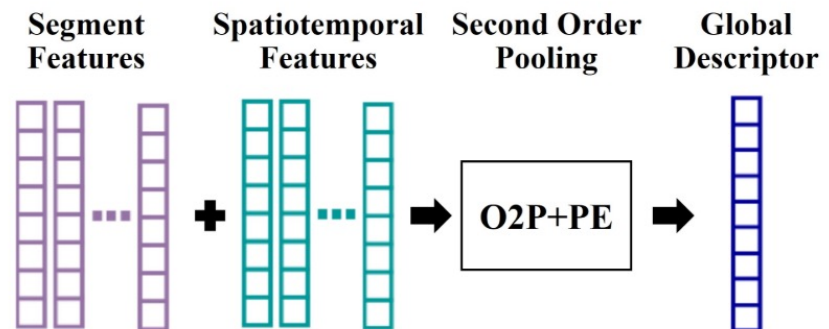
Lowry et. al., Visual Place Recognition: A Survey", IEEE TRO, 2016



(a)



(b)

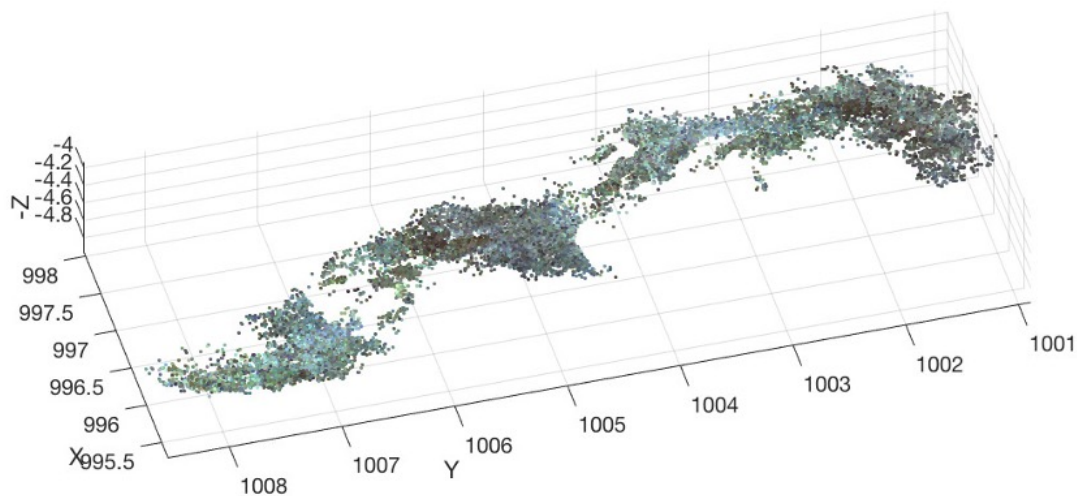
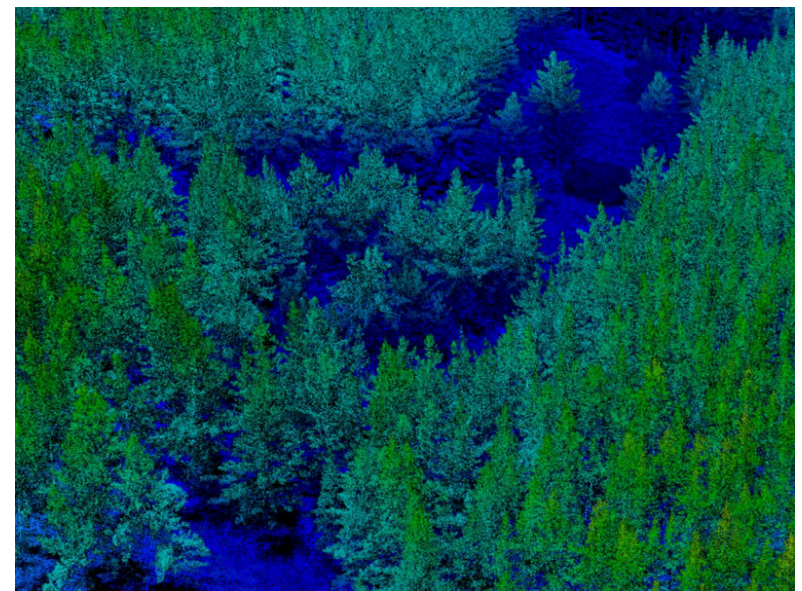
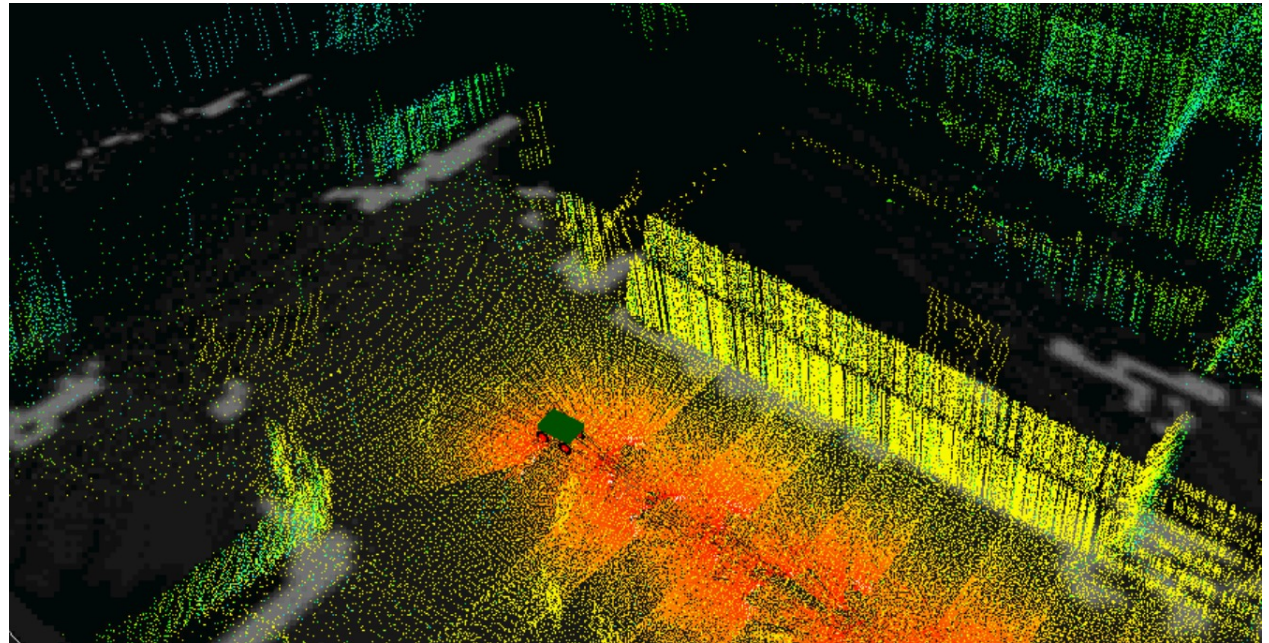


Vidanapathirana et.al., "Locus: LiDAR-based Place Recognition using Spatiotemporal Higher-Order Pooling." ICRA 2021

Map Representations

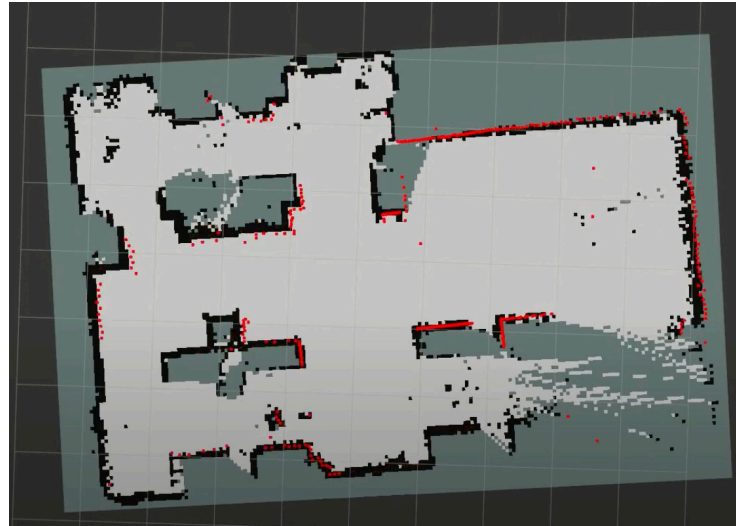
Whitty et. al., Autonomous Navigation using a Real-Time 3D Point Cloud, ACRA 2010

- Different types of map data to suit different purposes
 - Pointclouds
 - Occupancy maps
 - Surfaces: meshes, signed distance fields etc.

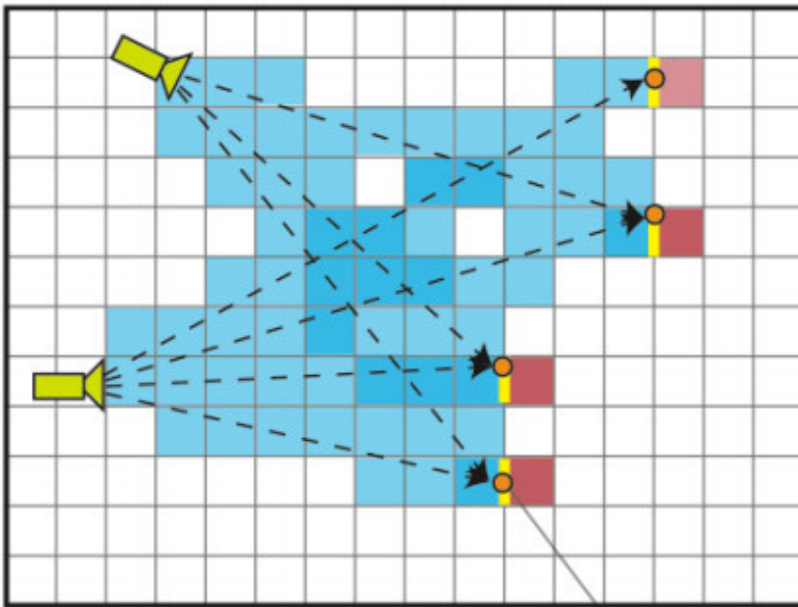


Map Representations

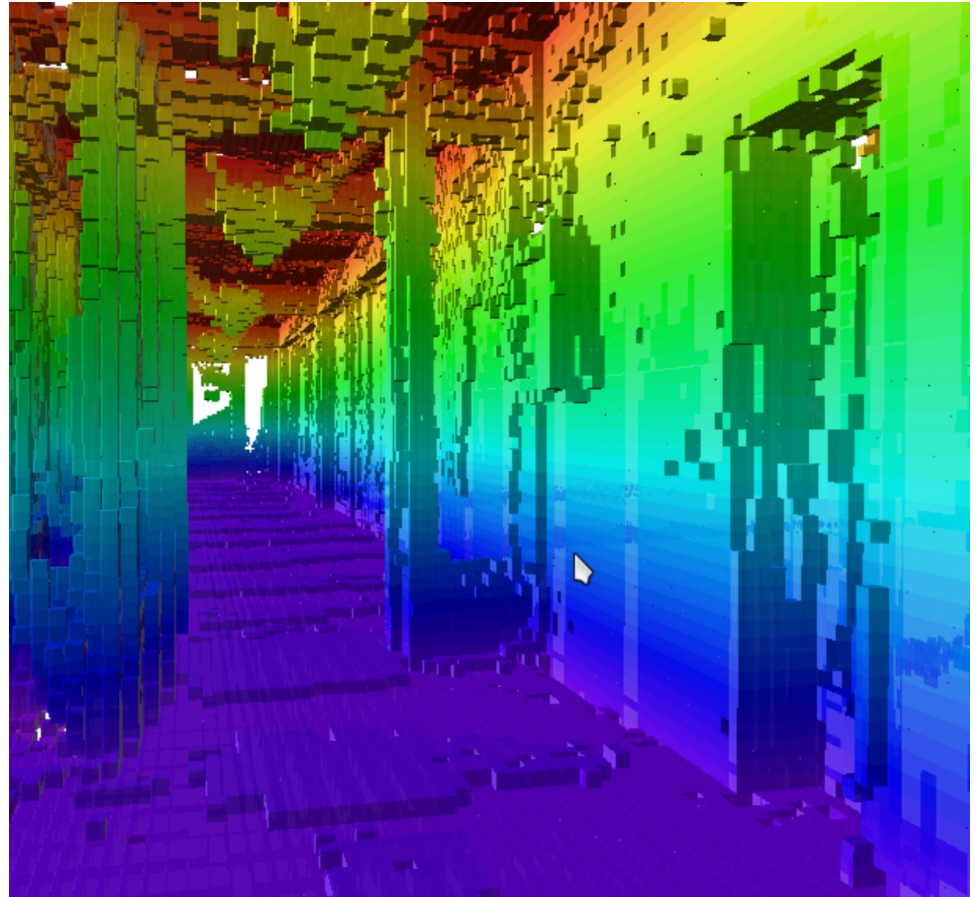
- Different types of map data to suit different purposes
 - Pointclouds
 - Occupancy maps
 - Surfaces: meshes, signed distance fields etc.



Hornug et. al.,
"OctoMap: An Efficient
Probabilistic 3D
Mapping Framework
Based on Octrees",
2013



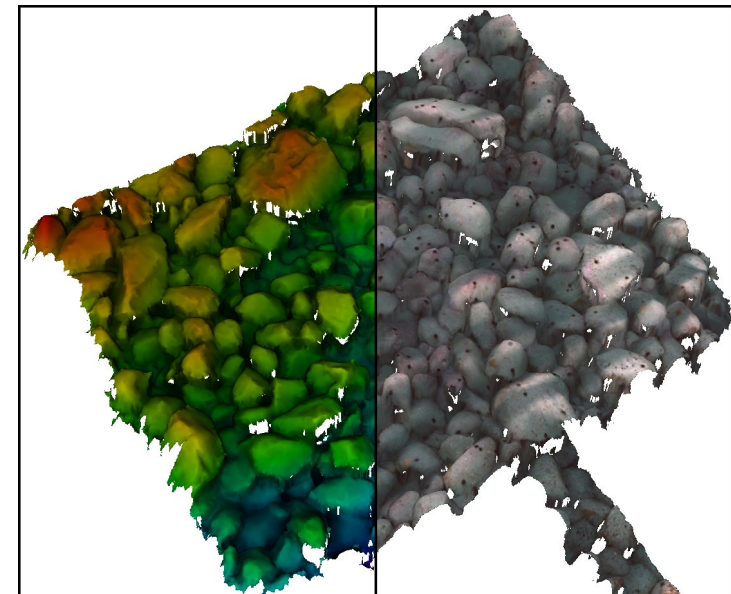
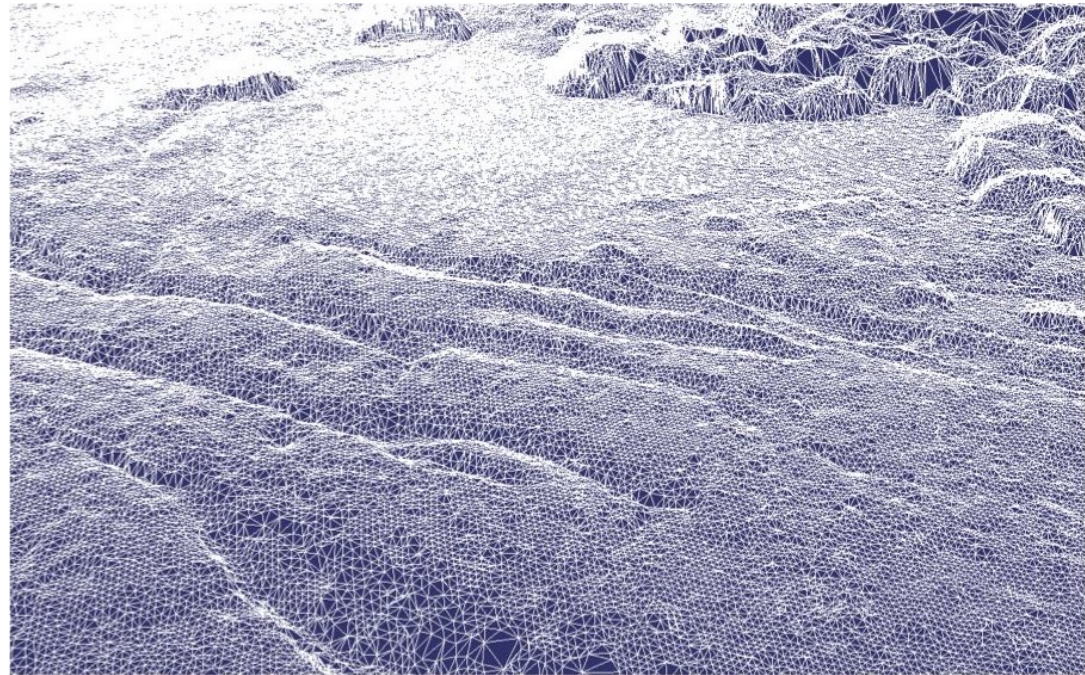
Vogiatzis et. al., "Multi-view stereo via volumetric graph-cuts", 2005.



Map Representations

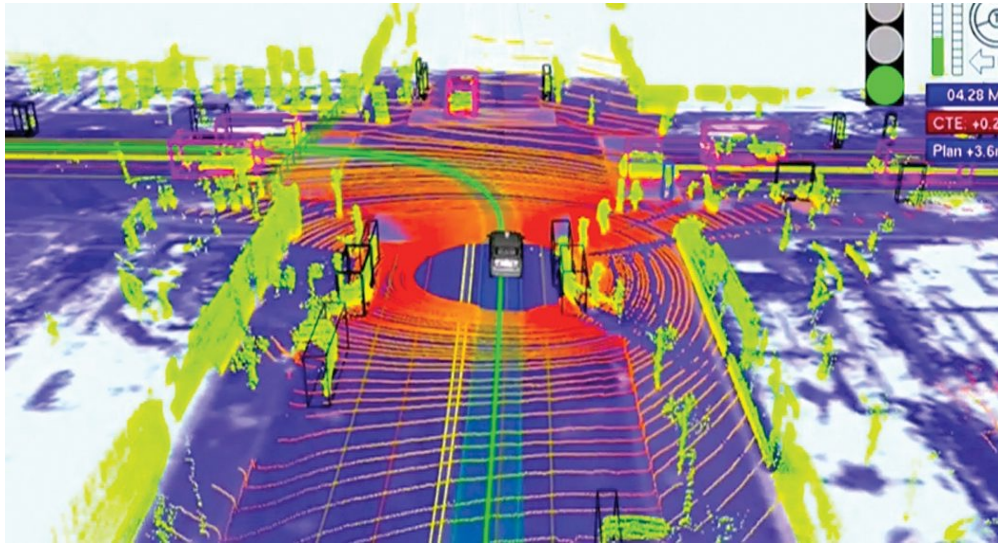
- Different types of map data to suit different purposes
 - Pointclouds
 - Occupancy maps
 - Surfaces: meshes, signed distance fields etc.

Triangulated surface mesh from point cloud

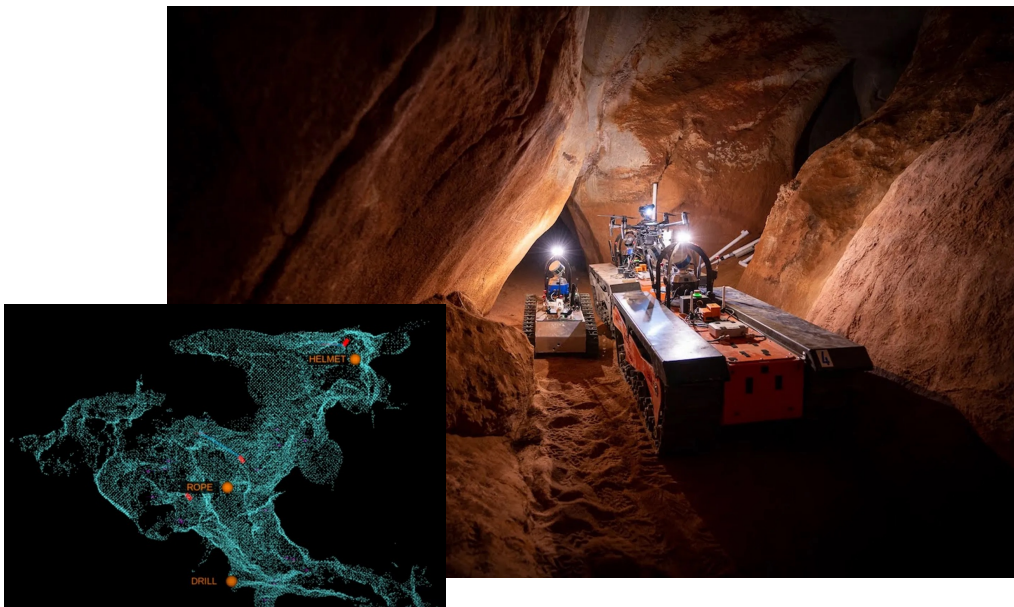


Examples of photo-textured triangulated surface mesh models

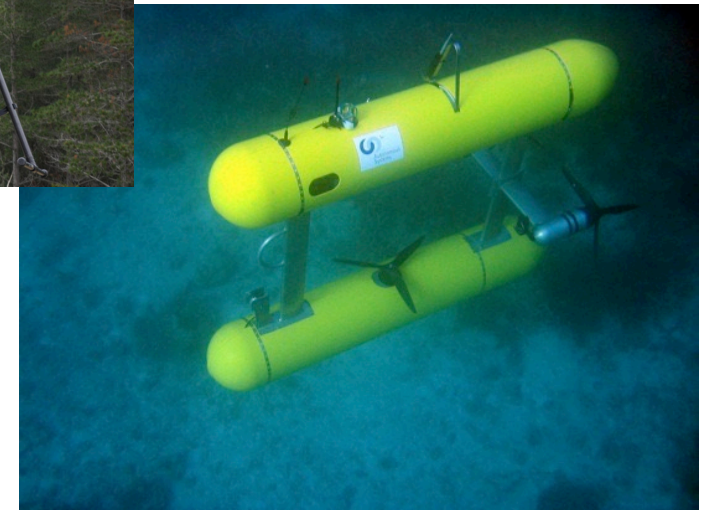
Considerations in different environments/domains



- Availability of external information/signals
- Sensing through medium (e.g. underwater and acoustics)
- Dynamic nature of the environment (urban etc.)
- Platform dynamics, maneuverability, 2D vs. 3D



DARPA SubT Challenge Cave Circuit event, CSIRO Data61 team



Terraluma LiDAR drone and AUV Sirius, ACFR

Overview of the lecture:

- Introduction to Robot Localisation and Mapping
- Robot Localisation and Mapping:
 - Basics: Coordinate Systems and Frames of Reference
 - Localisation and Mapping as an estimation problem
- SLAM:
 - Basics
 - How to implement
 - Simultaneous Localisation and Mapping (SLAM)
- Conclusions and Future outlook

Overview of the lecture:

- Introduction to Robot Localisation and Mapping
- Robot Localisation and Mapping:
 - Basics: Coordinate Systems and Frames of Reference
 - Localisation and Mapping as an estimation problem
- Examples:
 - Beacon-based localisation
 - Aided-inertial navigation
 - Simultaneous Localisation and Mapping (SLAM)
- Conclusions and Future outlook

Frame of reference

- Allocentric
- Egocentric
- Route-centric
- Hybrid

A EGOCENTRIC



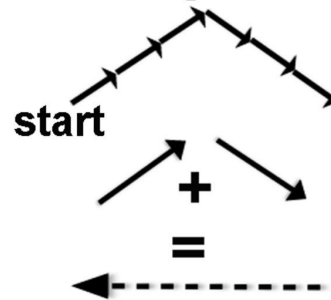
Ex. To get to the Bookstore I need to turn right then go straight one block and turn left.

B ALLOCENTRIC

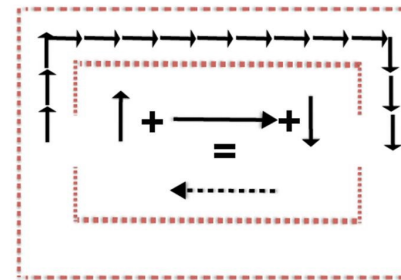


Ex. The Bookstore is 30 meters north of the Gym and 20 meters east of the Camera Store.

C Simple path integration

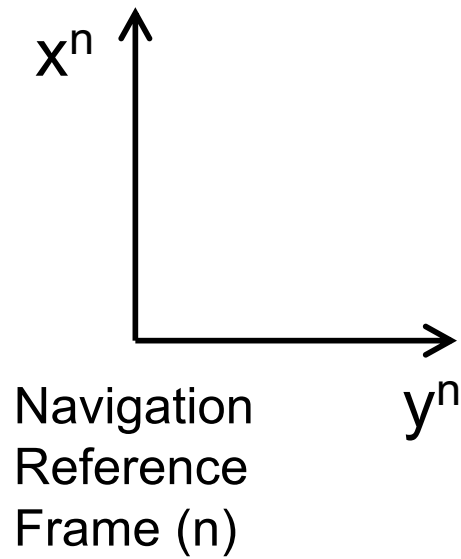


D Egocentric short-cut strategy

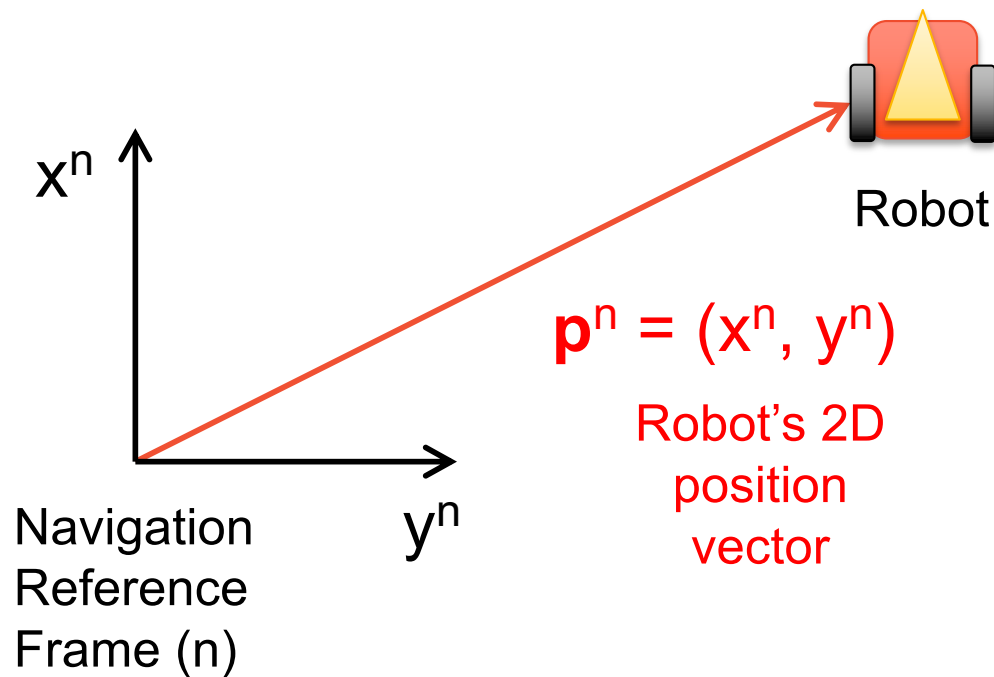


<https://www.frontiersin.org/articles/10.3389/fnhum.2014.00803/full>

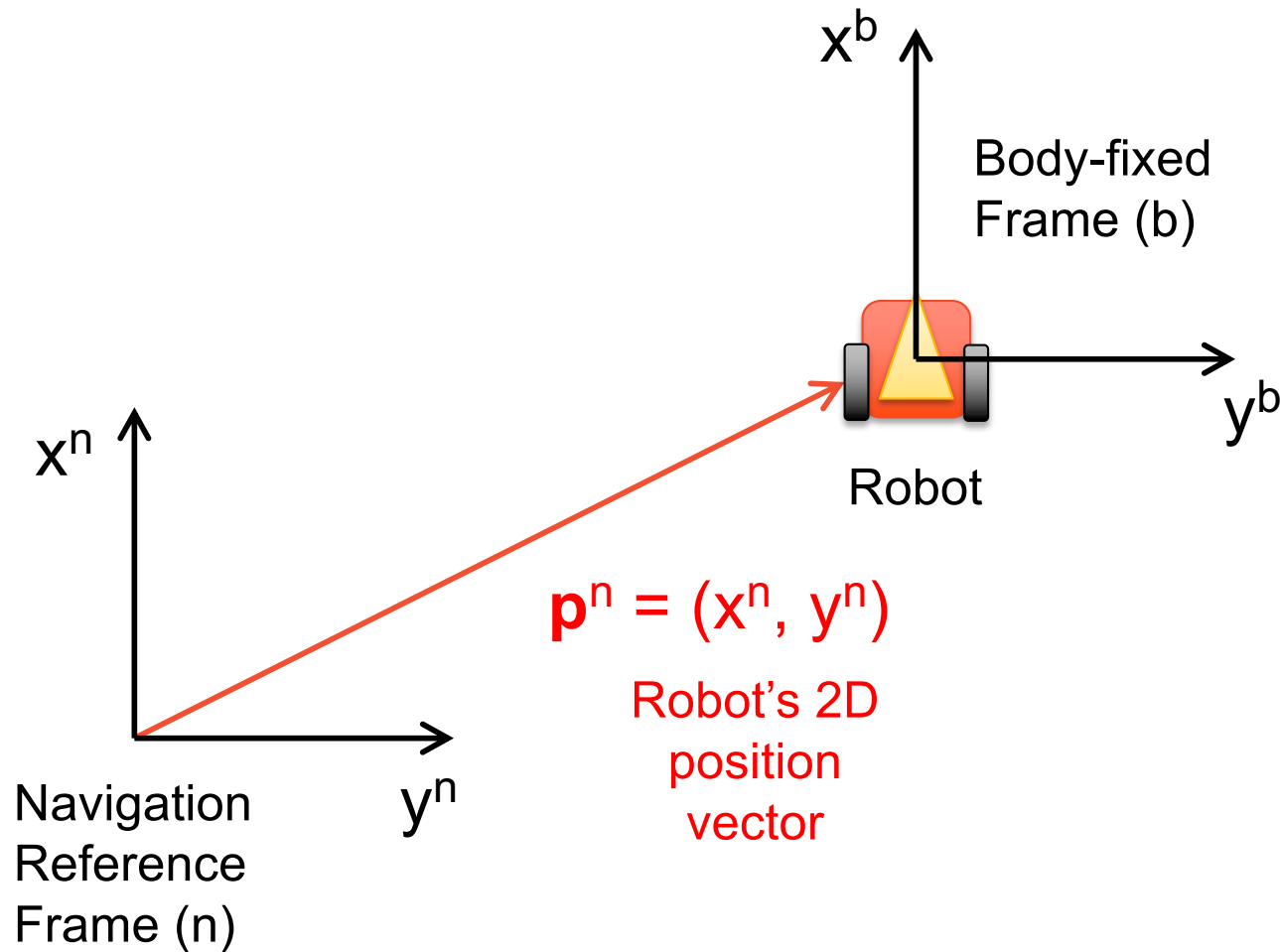
Basics: Coordinate Systems and Frames of Reference



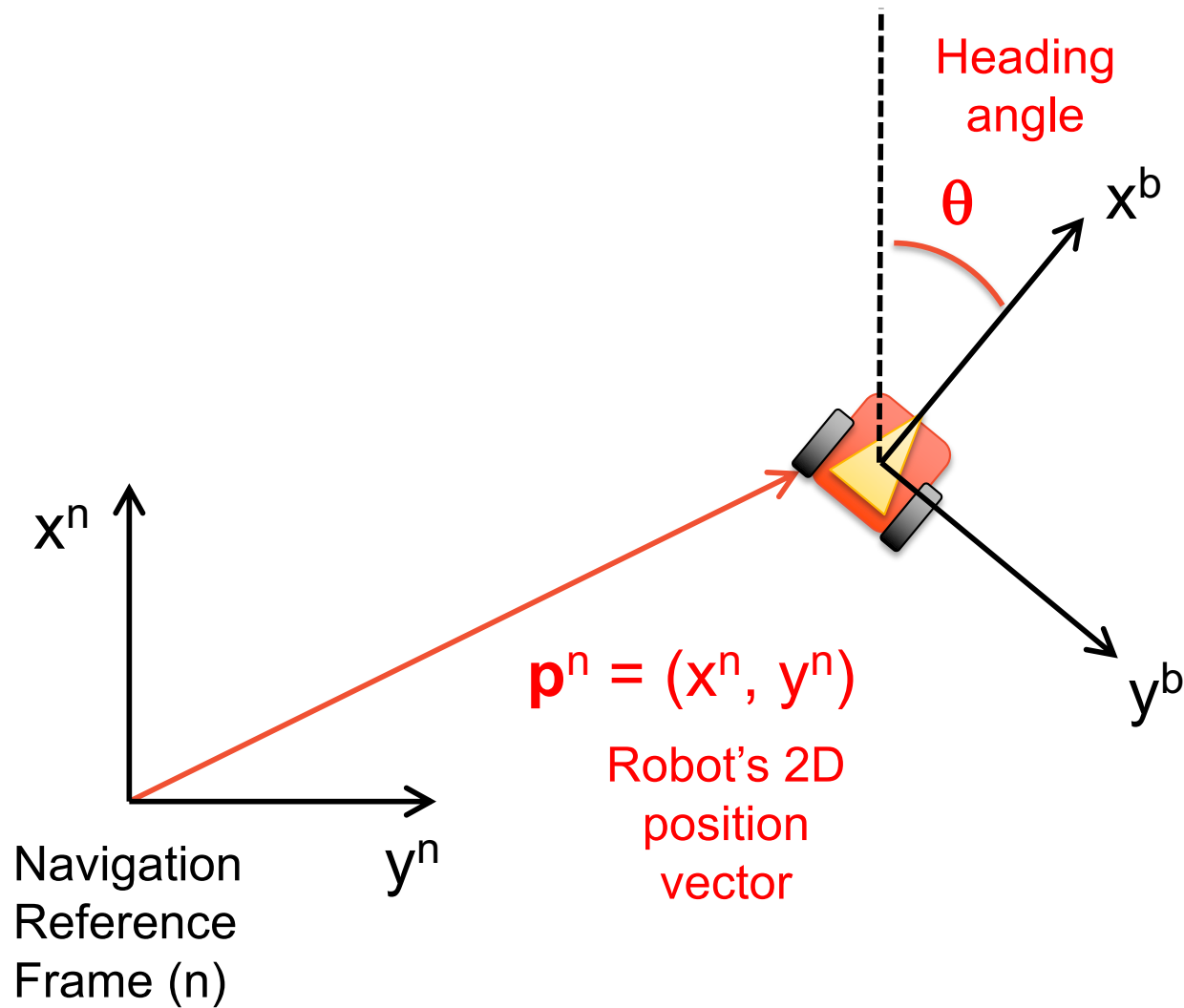
Basics: Coordinate Systems and Frames of Reference



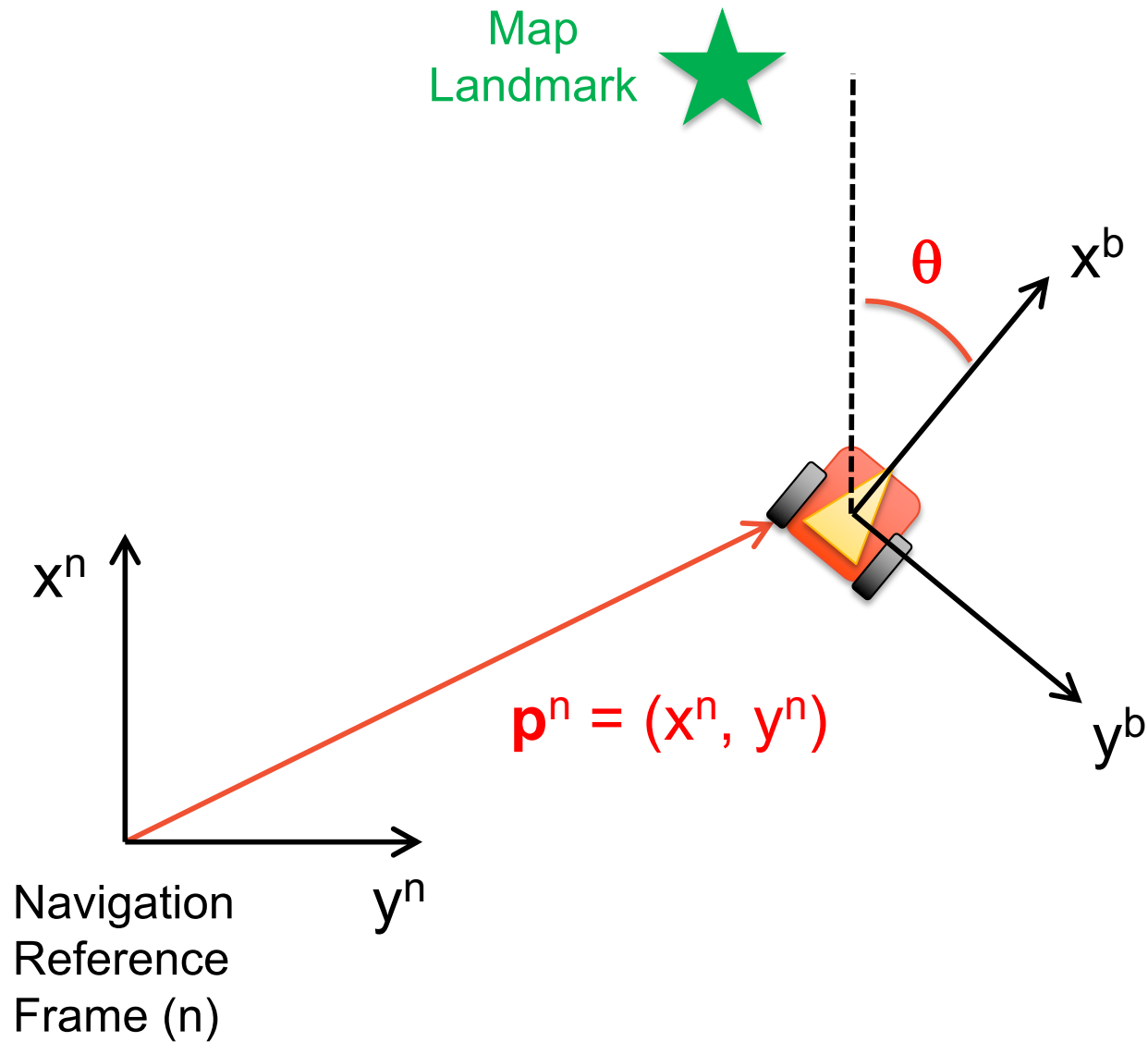
Basics: Coordinate Systems and Frames of Reference



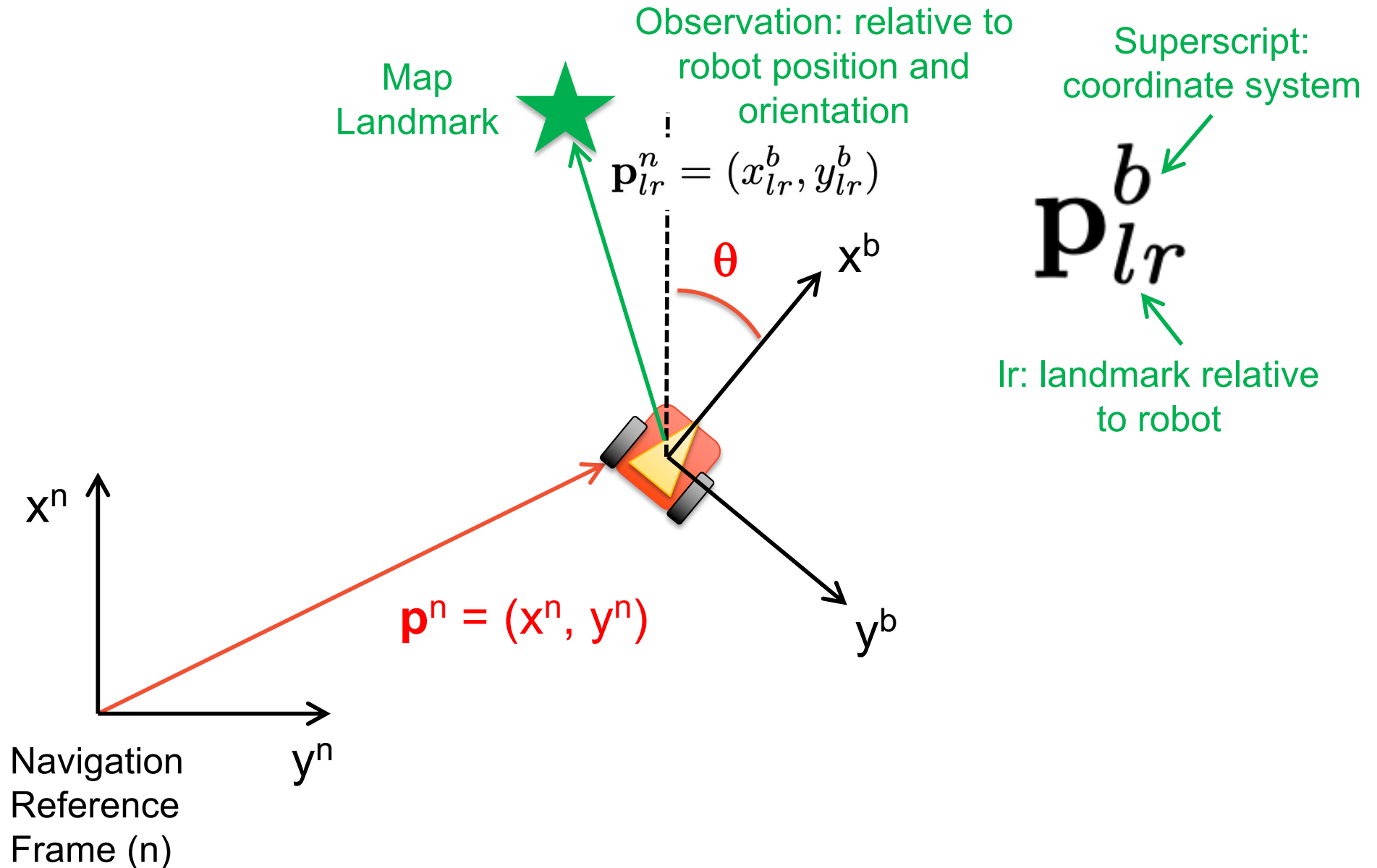
Basics: Coordinate Systems and Frames of Reference



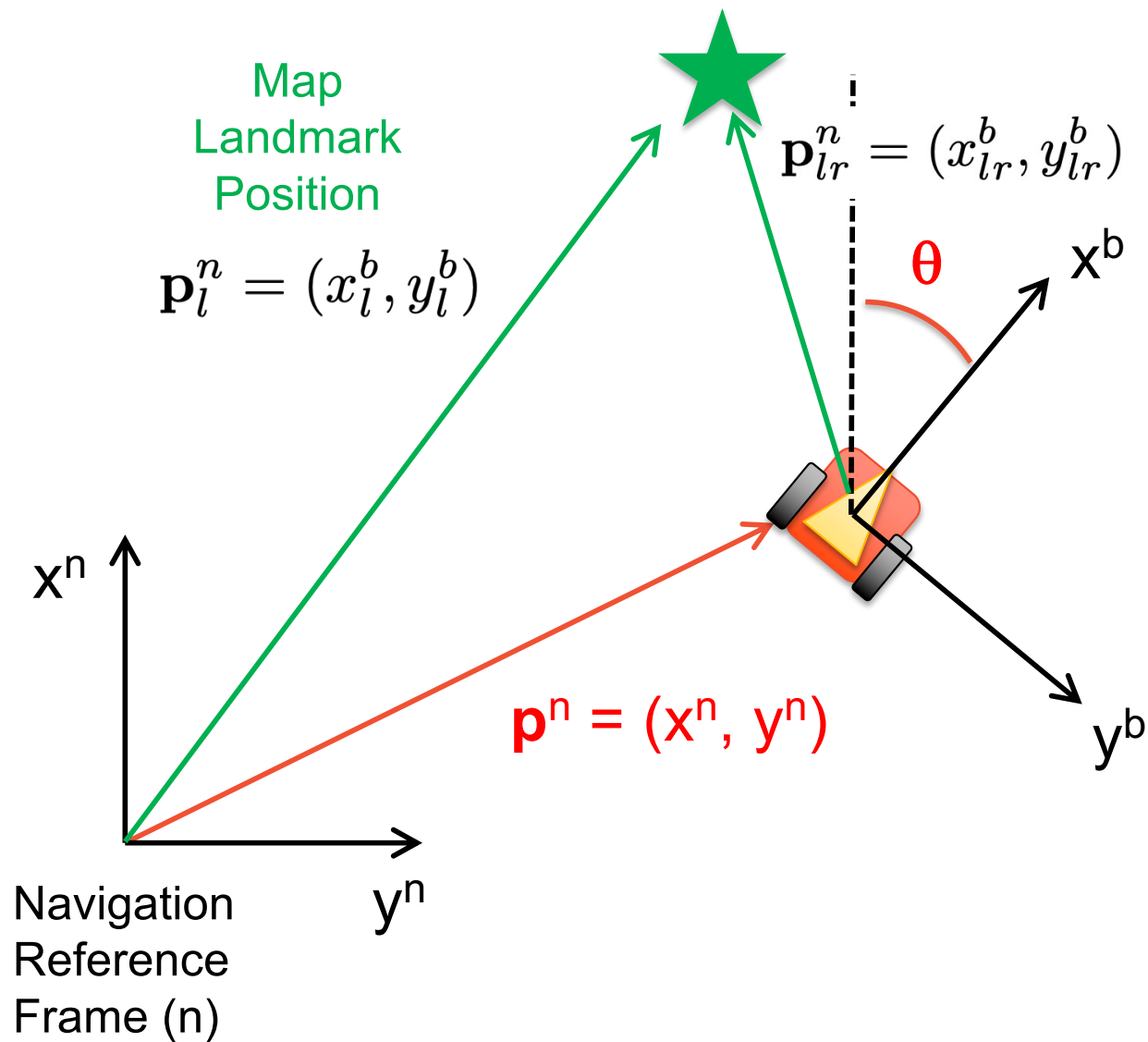
Basics: Coordinate Systems and Frames of Reference



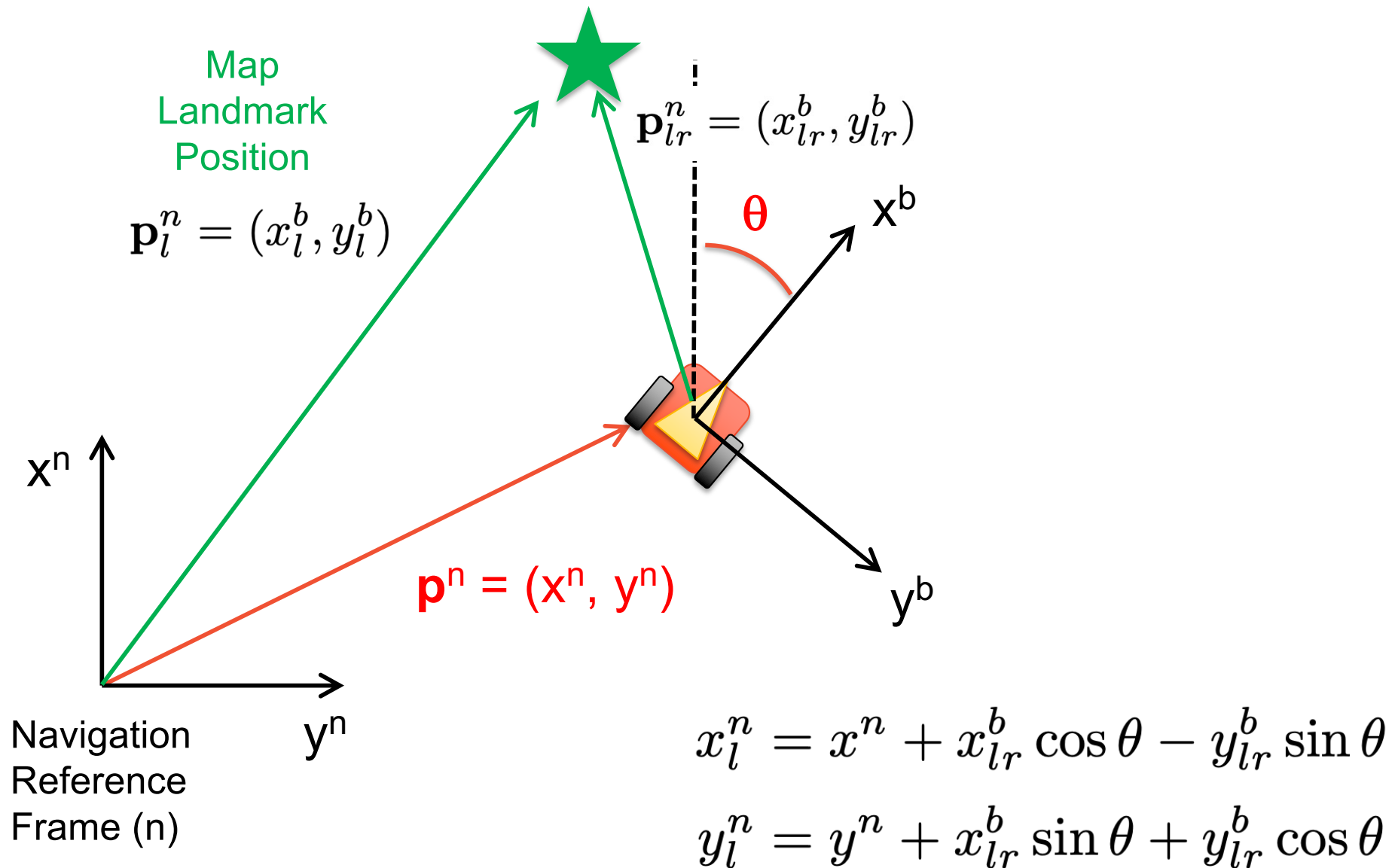
Basics: Coordinate Systems and Frames of Reference



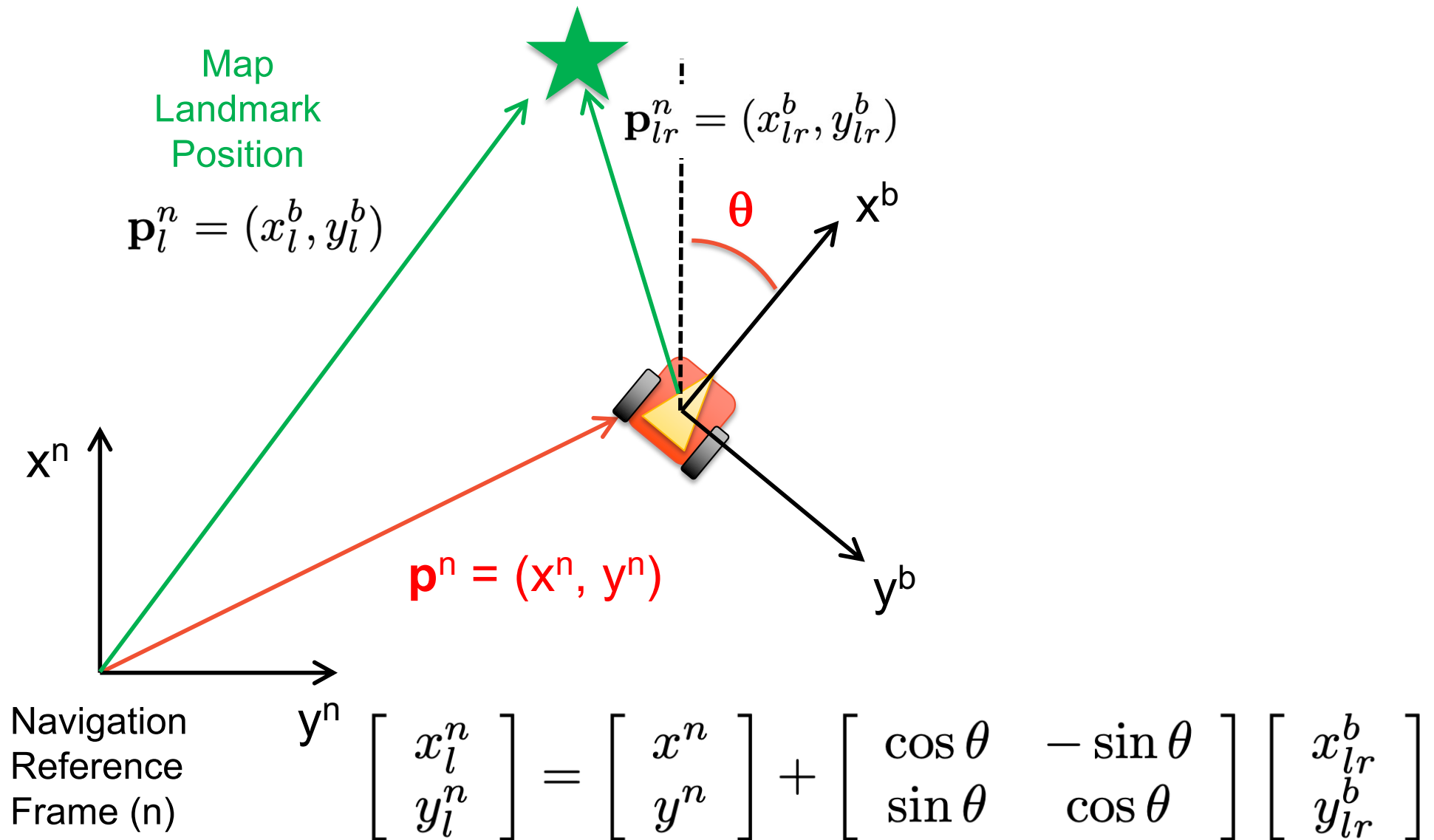
Basics: Coordinate Systems and Frames of Reference



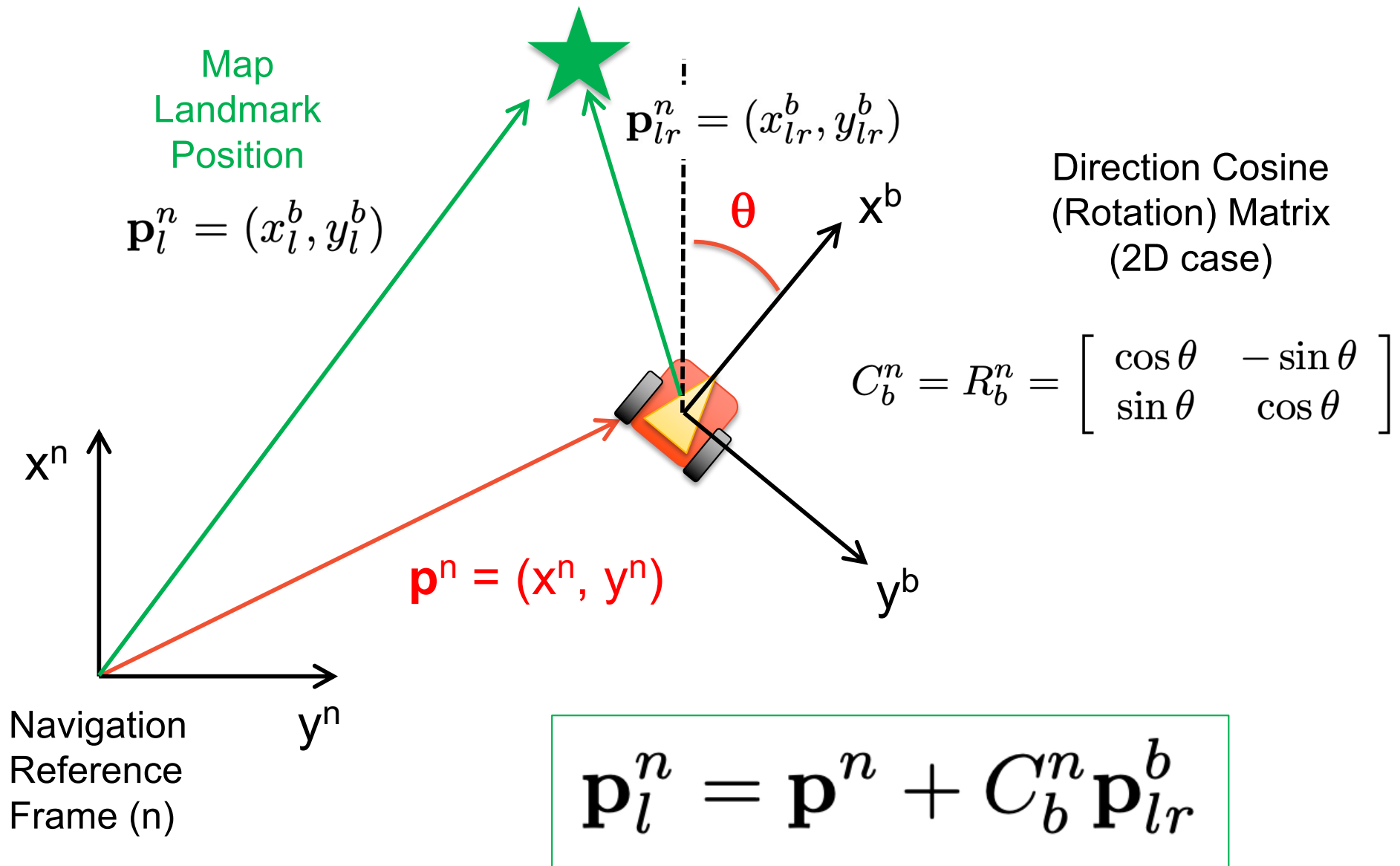
Basics: Coordinate Systems and Frames of Reference



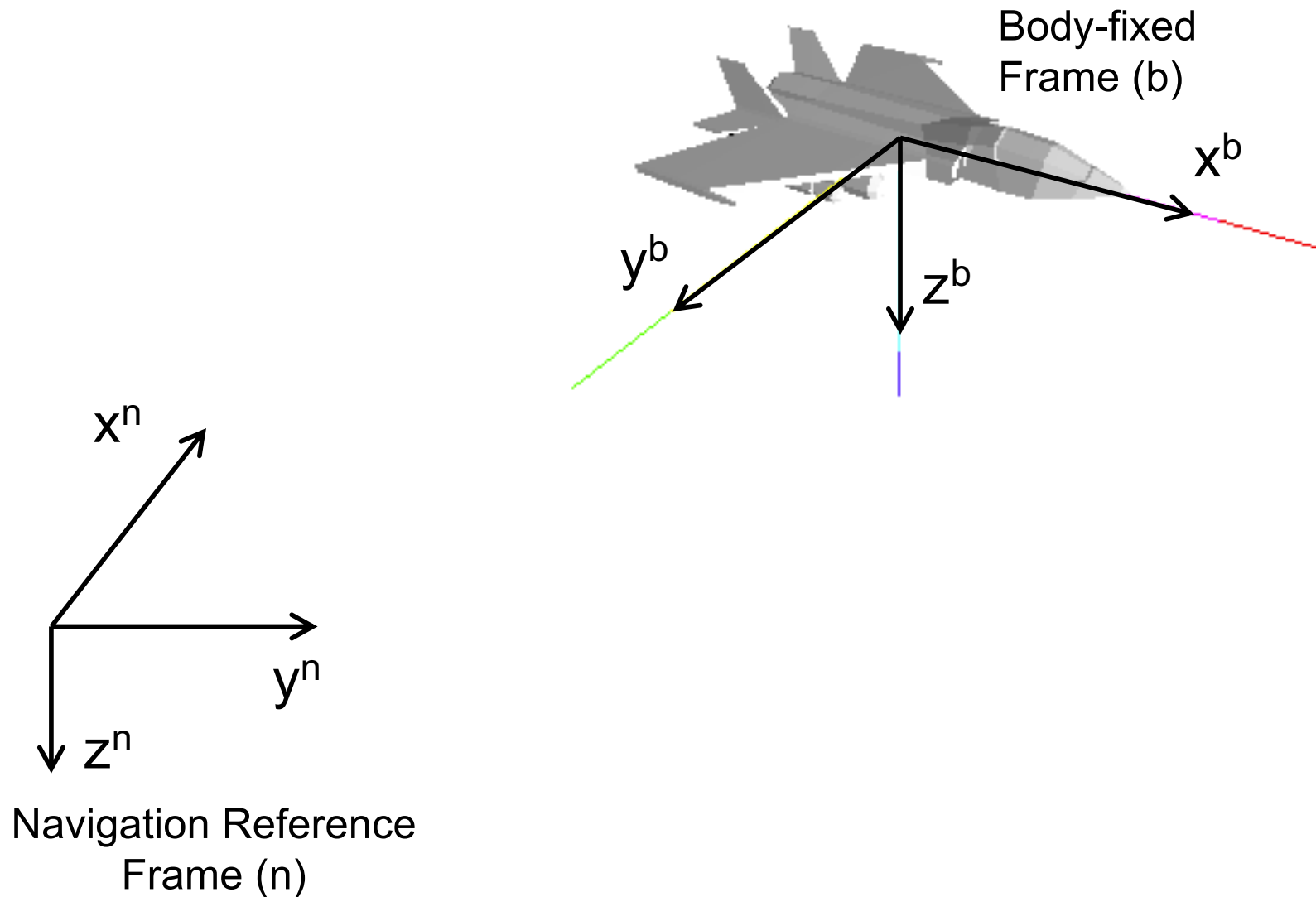
Basics: Coordinate Systems and Frames of Reference



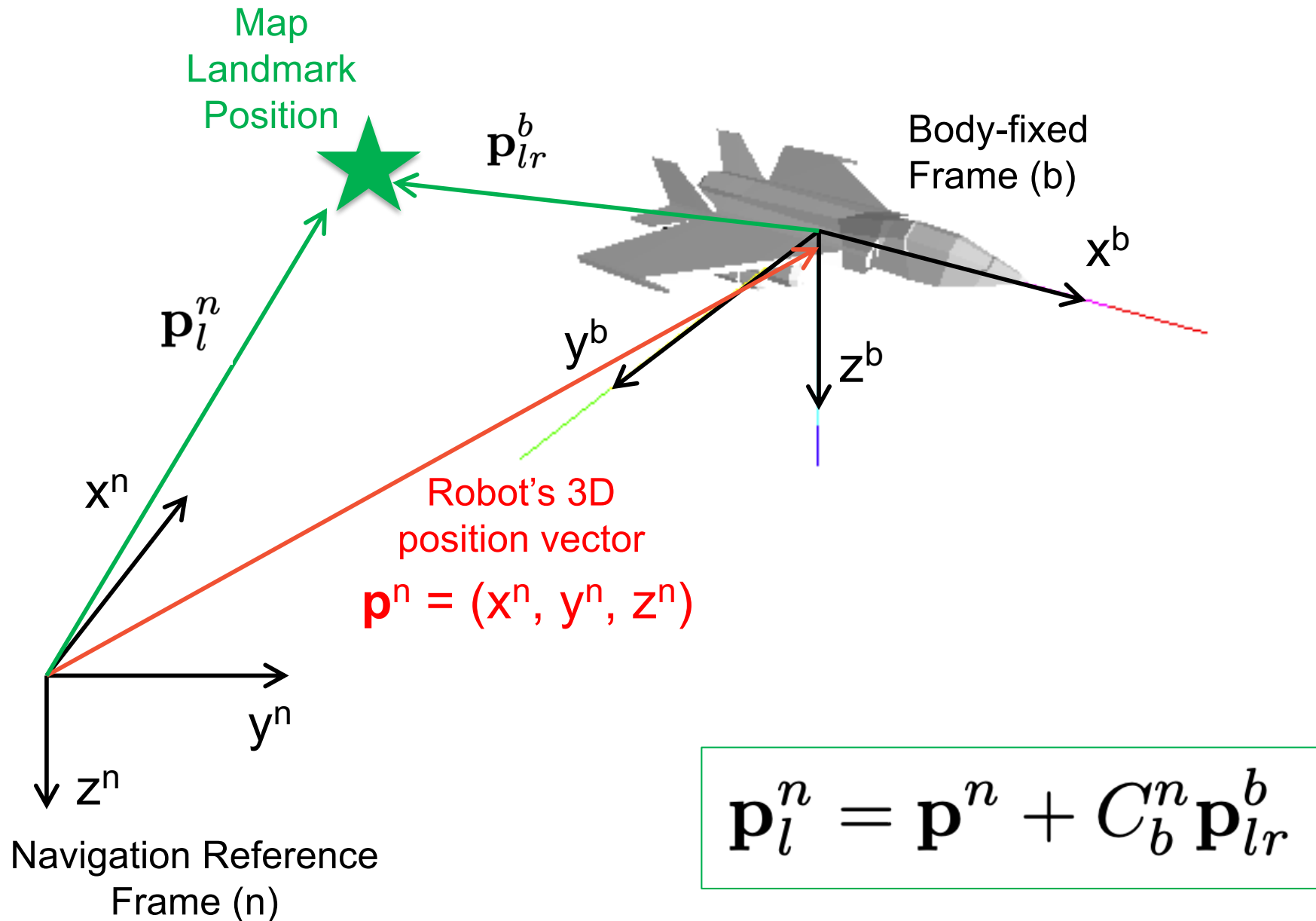
Basics: Coordinate Systems and Frames of Reference



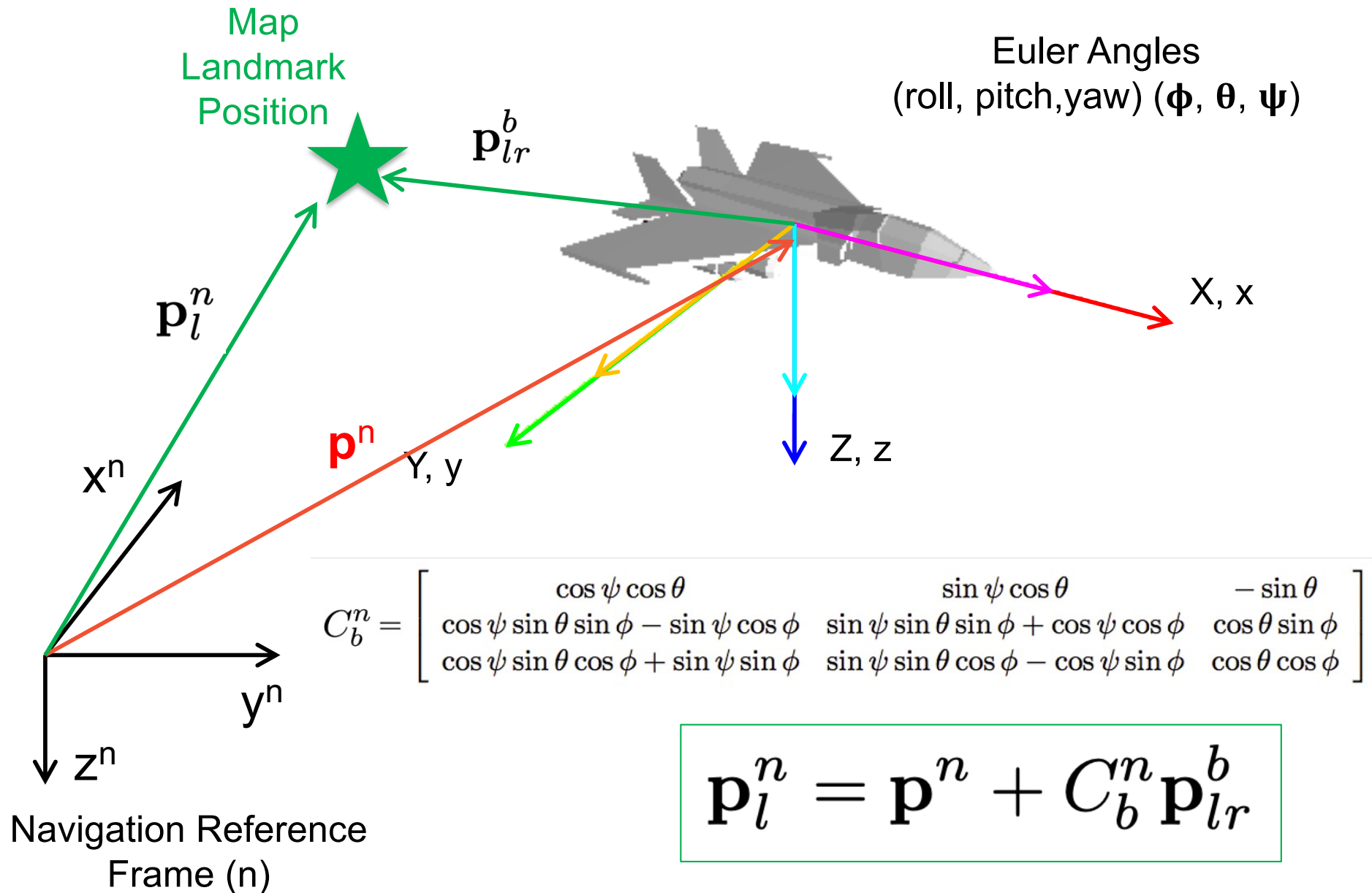
Basics: Coordinate Systems and Frames of Reference



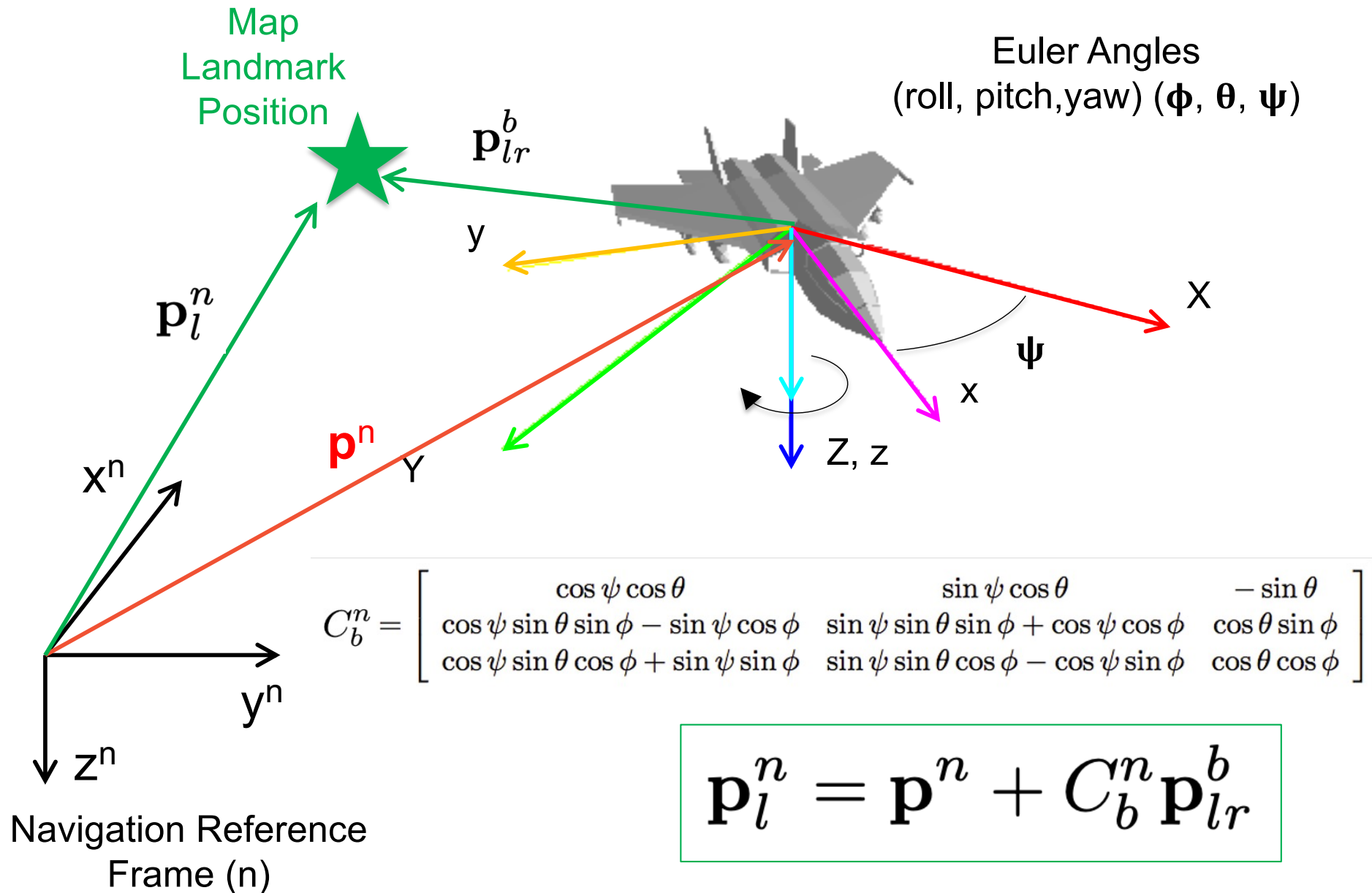
Basics: Coordinate Systems and Frames of Reference



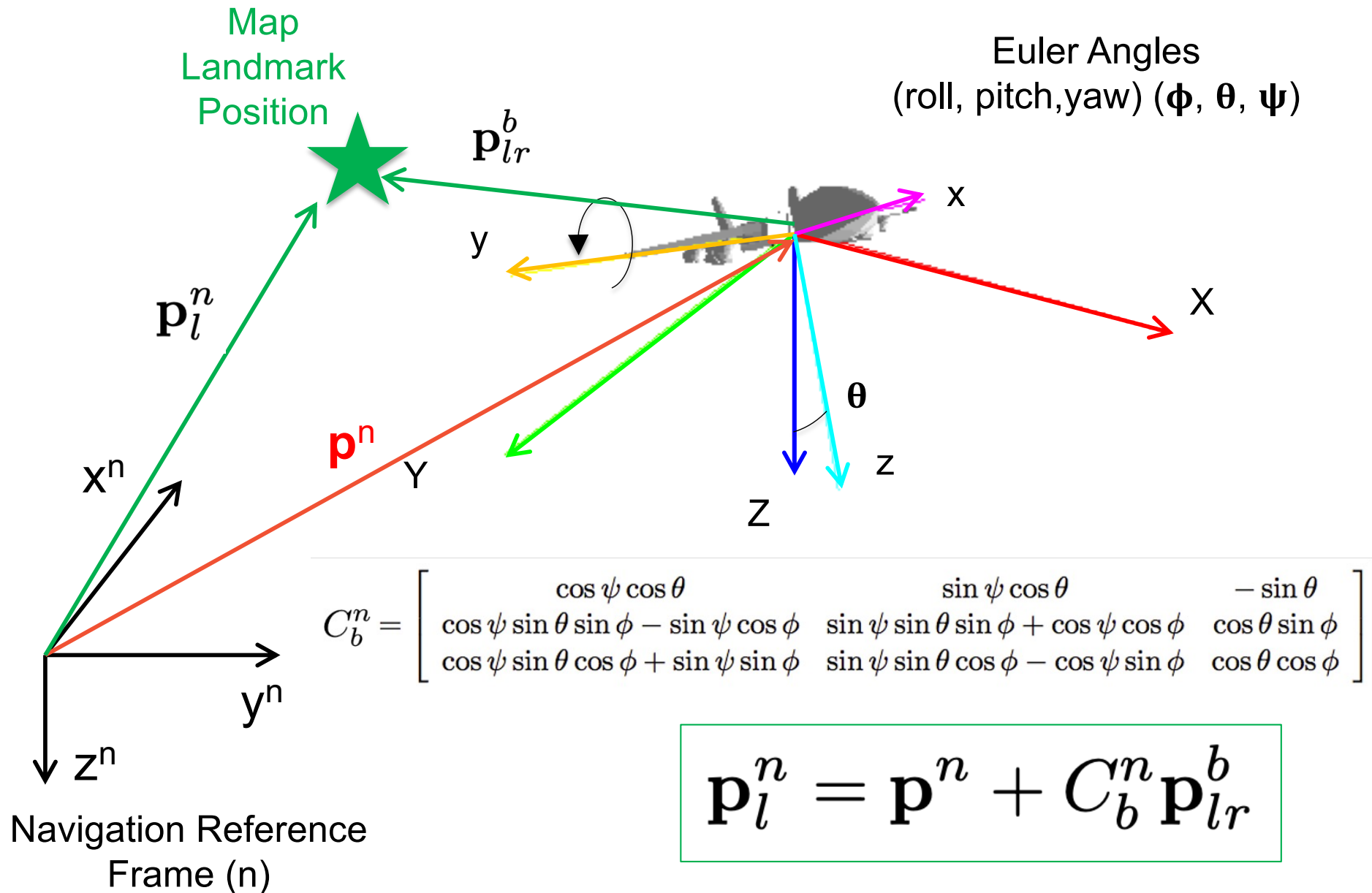
Basics: Coordinate Systems and Frames of Reference



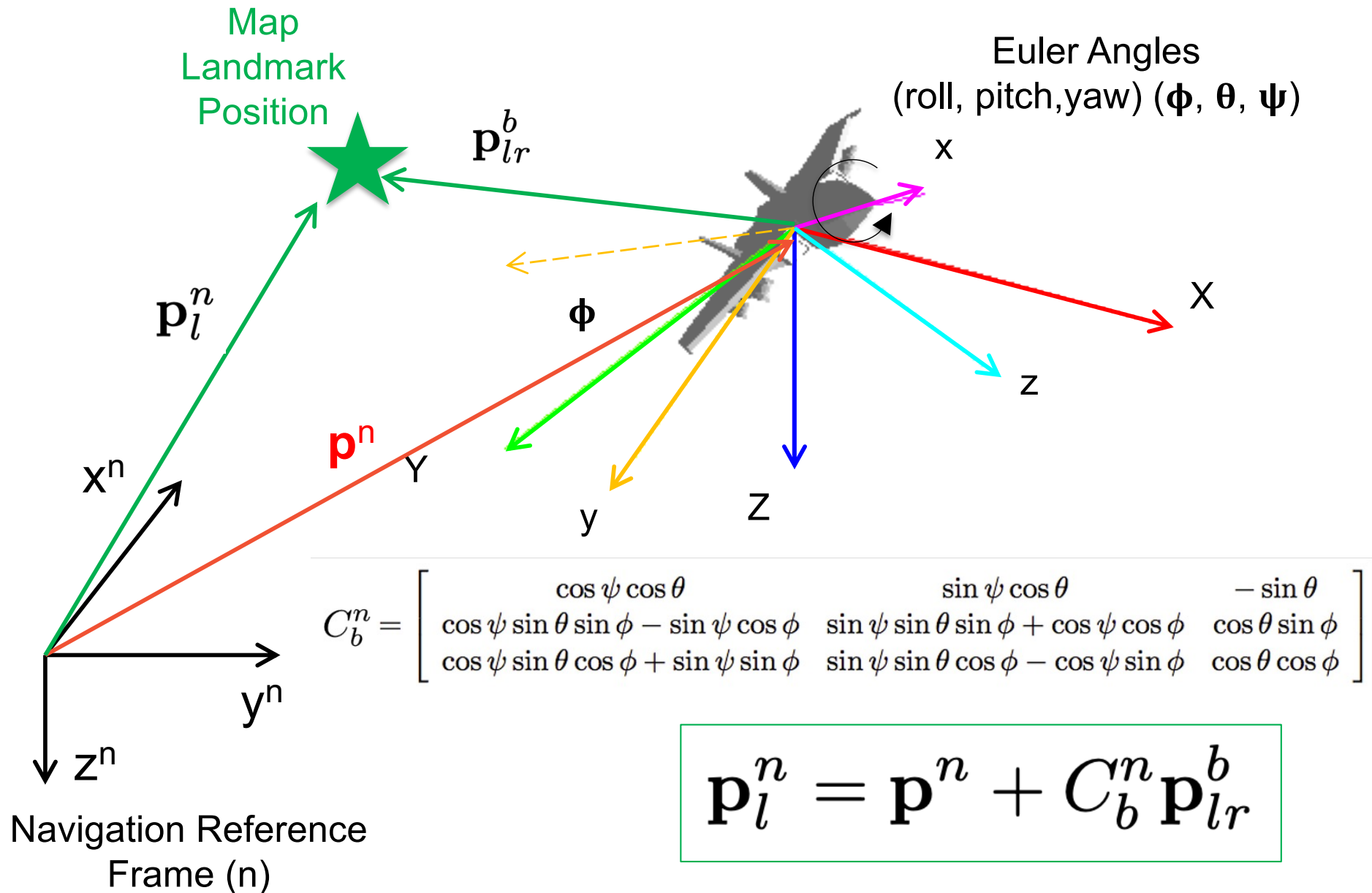
Basics: Coordinate Systems and Frames of Reference



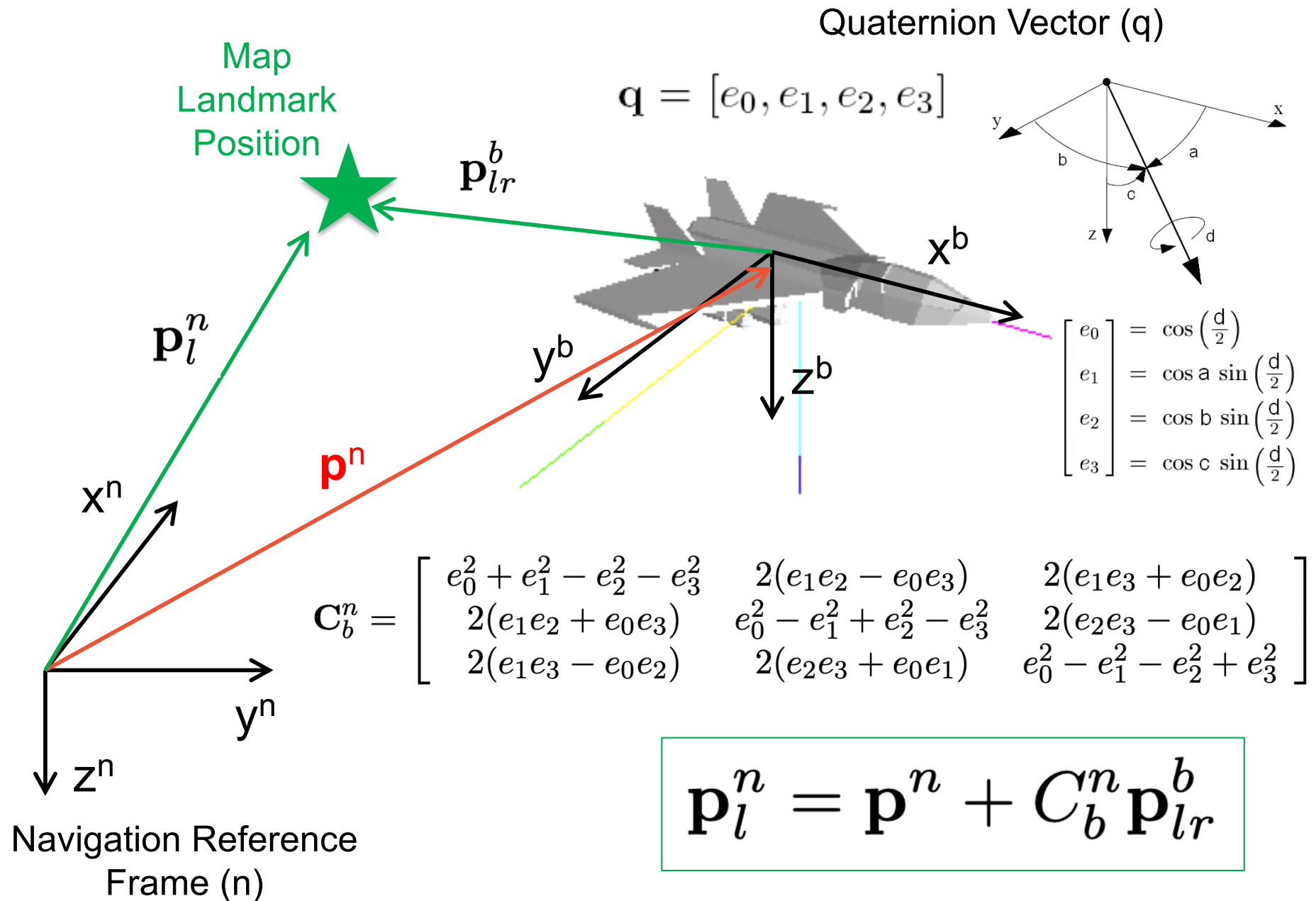
Basics: Coordinate Systems and Frames of Reference



Basics: Coordinate Systems and Frames of Reference

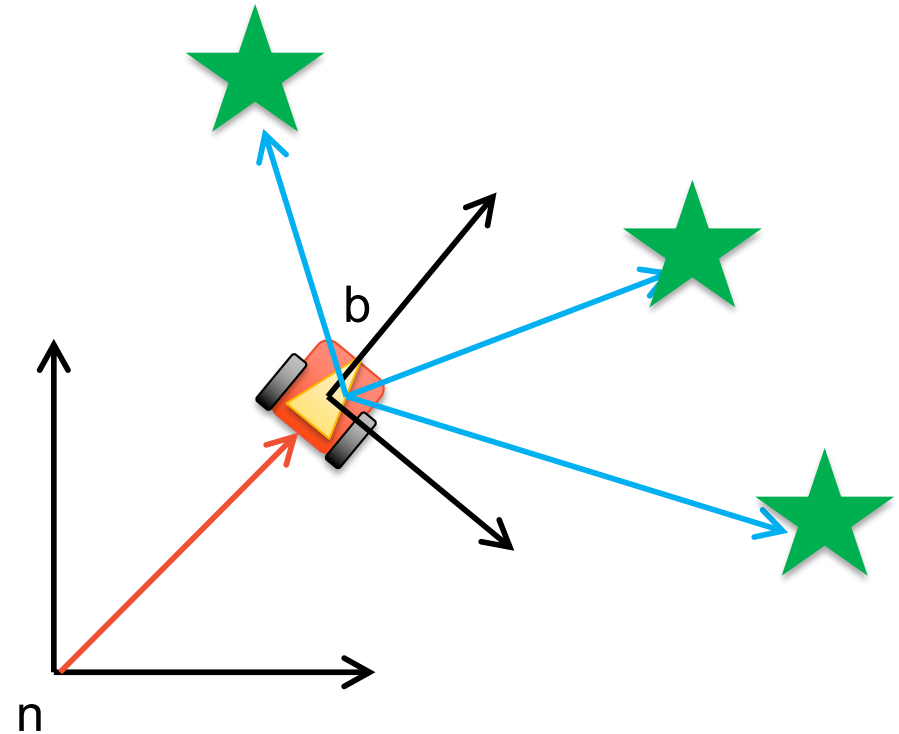


Basics: Coordinate Systems and Frames of Reference



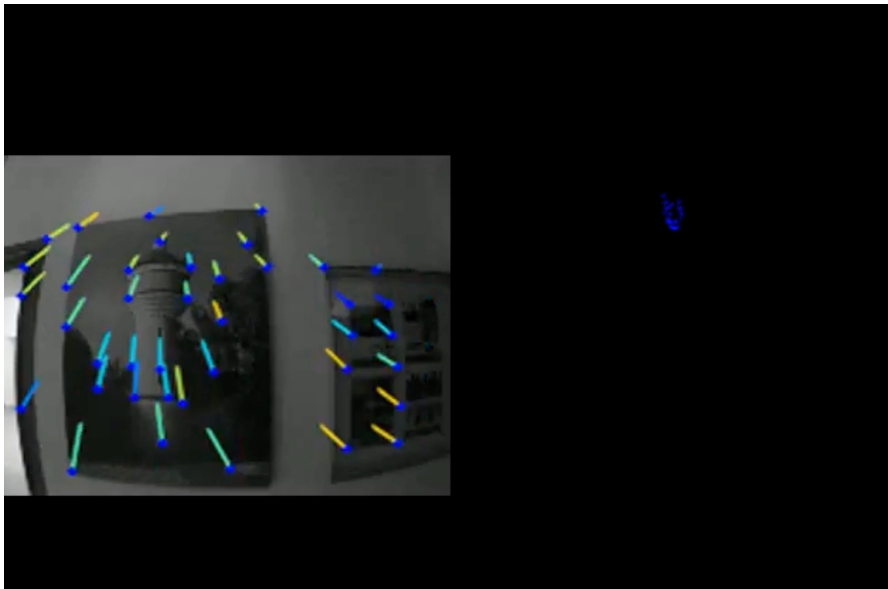
Types of Localisation and Mapping

- Fundamentally localization and mapping is an estimation problem involving the fusion of sensor data (and other information sources)
- Robot localization and mapping problems can be categorized based on available information and required accuracy in the estimation of key states

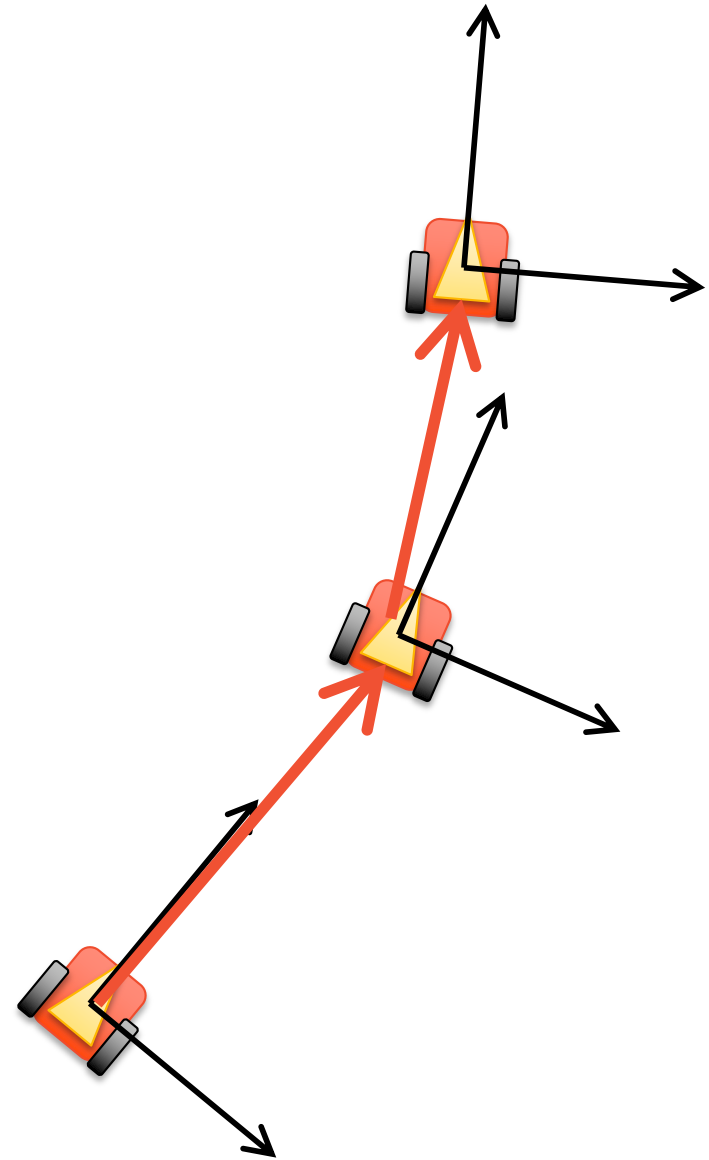


Dead-reckoning and Odometry

- Dead reckoning: determine my current pose from last pose and relative information
- Examples: measurements of velocity from wheel encoders, inertial navigation, LiDAR scan matching, visual odometry

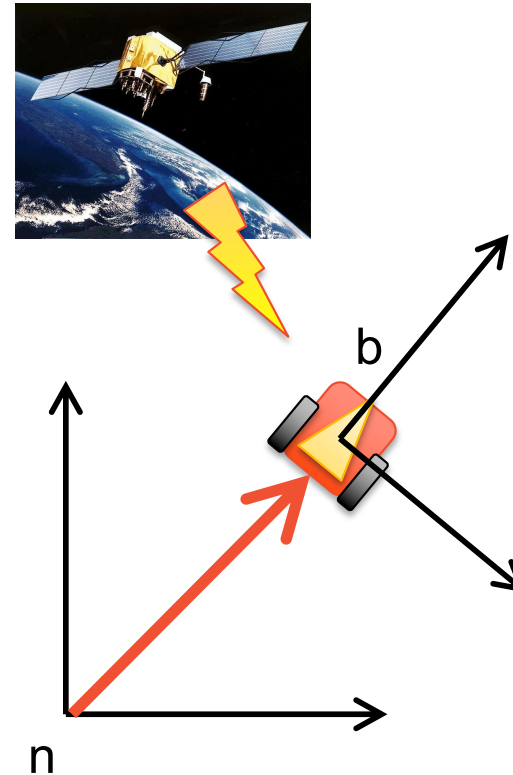


TUM Monocular Visual Odometry Dataset



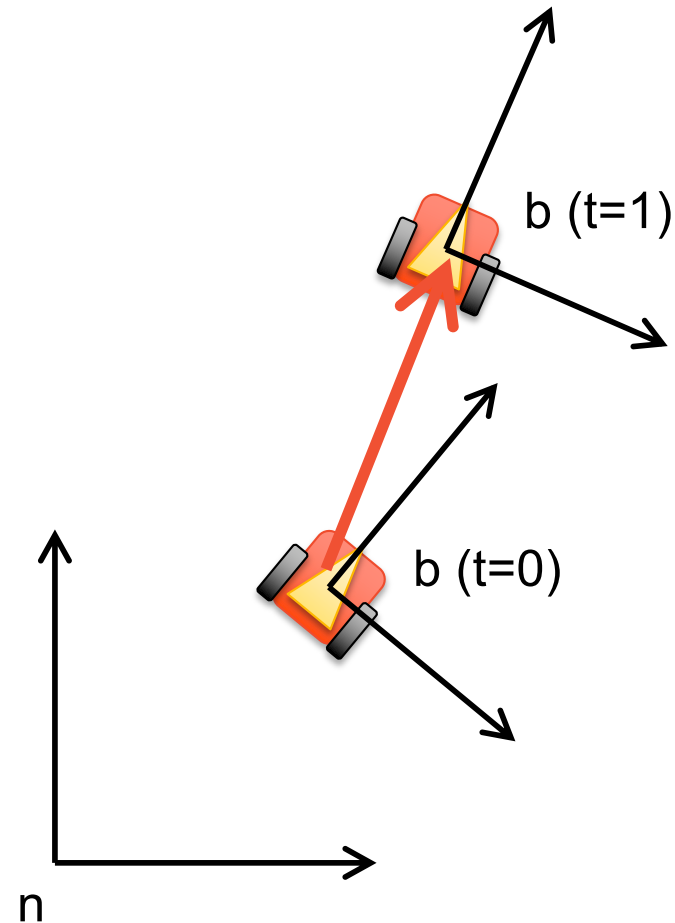
Types of Localisation and Mapping: Information Sources

- **External localization reference:**
 - Location information provided from external source
 - Examples: GNSS, Motion-capture, magnetic heading etc.



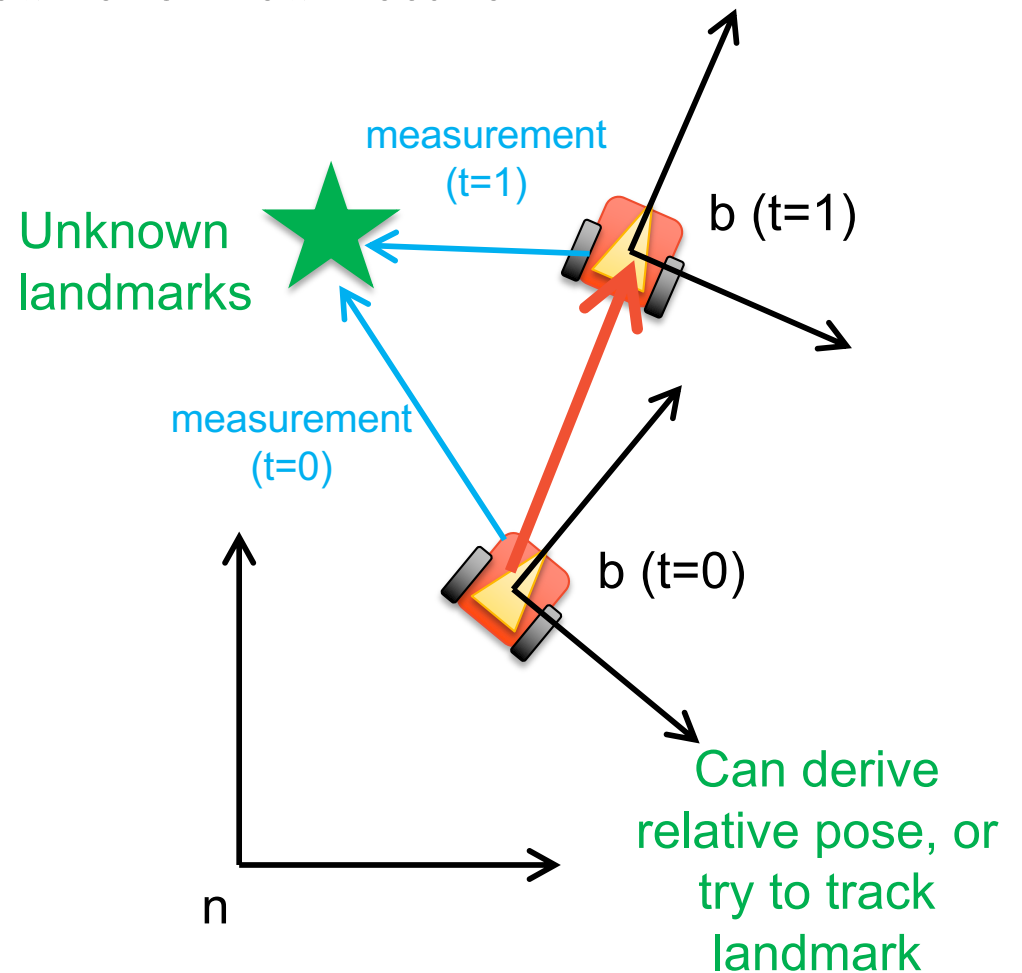
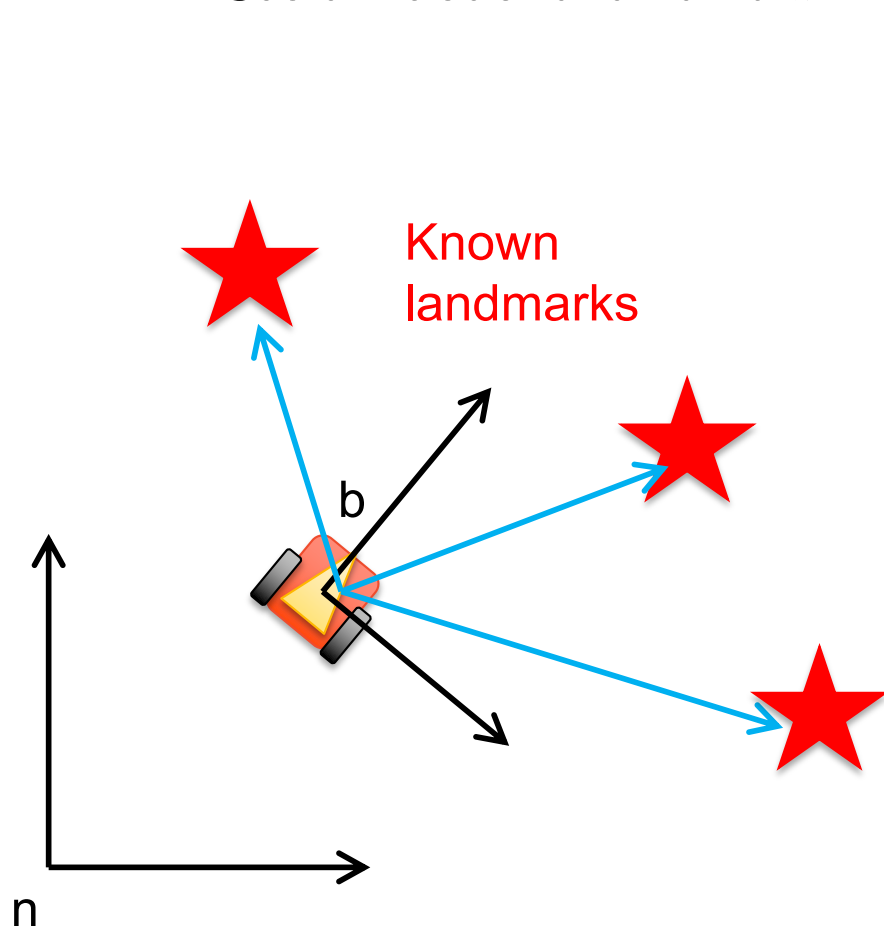
Types of Localisation and Mapping: Information Sources

- **Pose-Relative Measurements:**
 - Location information is provided relative to last pose, or as time-derivatives of pose
 - Examples: odometry, inertial navigation, scan/feature matching between subsequent sensor measurements of external, unknown landmarks



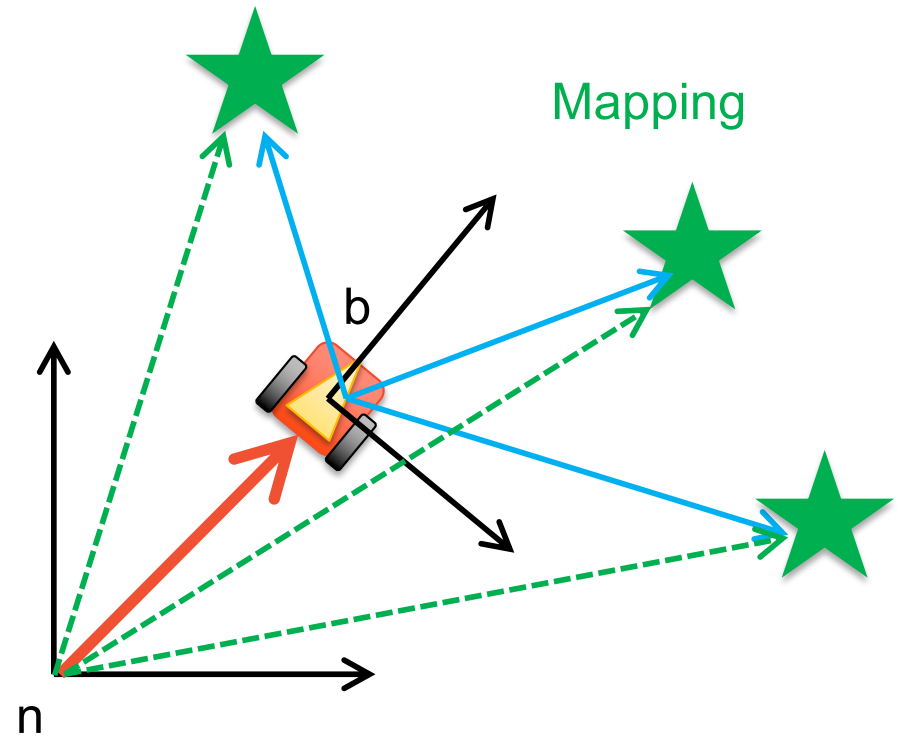
Types of Localisation and Mapping: Information Sources

- **Relative Map/Environment Measurements:**
 - Robot-relative measurements made to environment
 - Examples: scan/feature matching from LiDAR scans, vision features
 - Could include landmarks with known or unknown location



Types of Localisation and Mapping: Estimation Problem

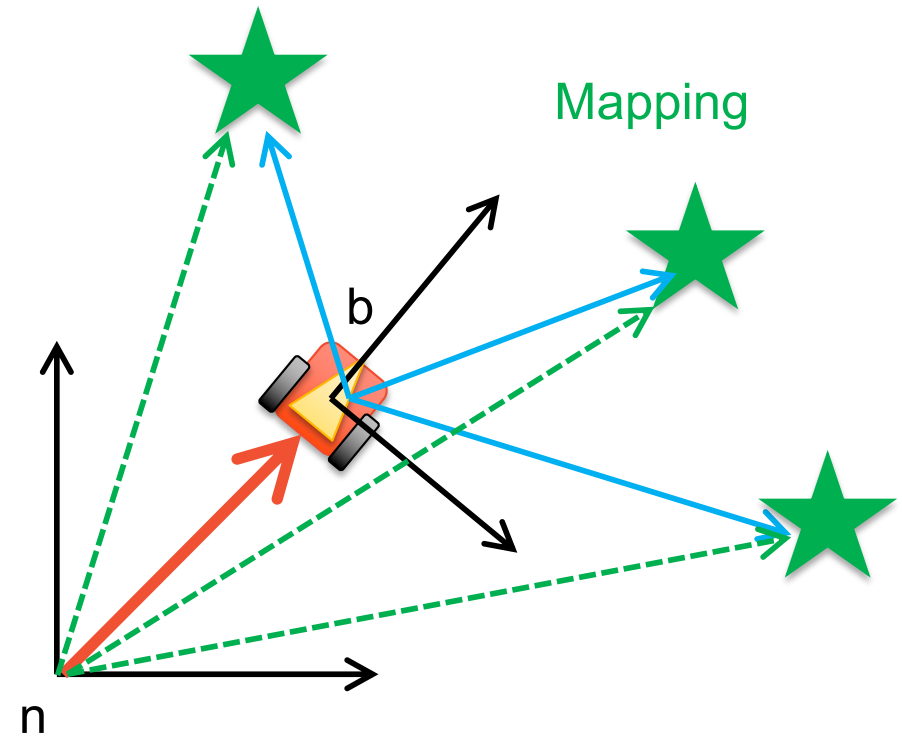
- **Mapping using external localization reference:**
 - If we assume pose of the robot is known, we can compute the locations of map features directly robot-relative sensor measurements
 - Sometimes referred to a “Direct Georeferencing”



$$\mathbf{p}_l^n = \mathbf{p}^n + C_b^n \mathbf{p}_{lr}^b$$

Types of Localisation and Mapping: Estimation Problem

- **Mapping using external localization reference:**
 - If we assume pose of the robot is known, we can compute the locations of map features directly robot-relative sensor measurements
 - Sometimes referred to a “Direct Georeferencing”

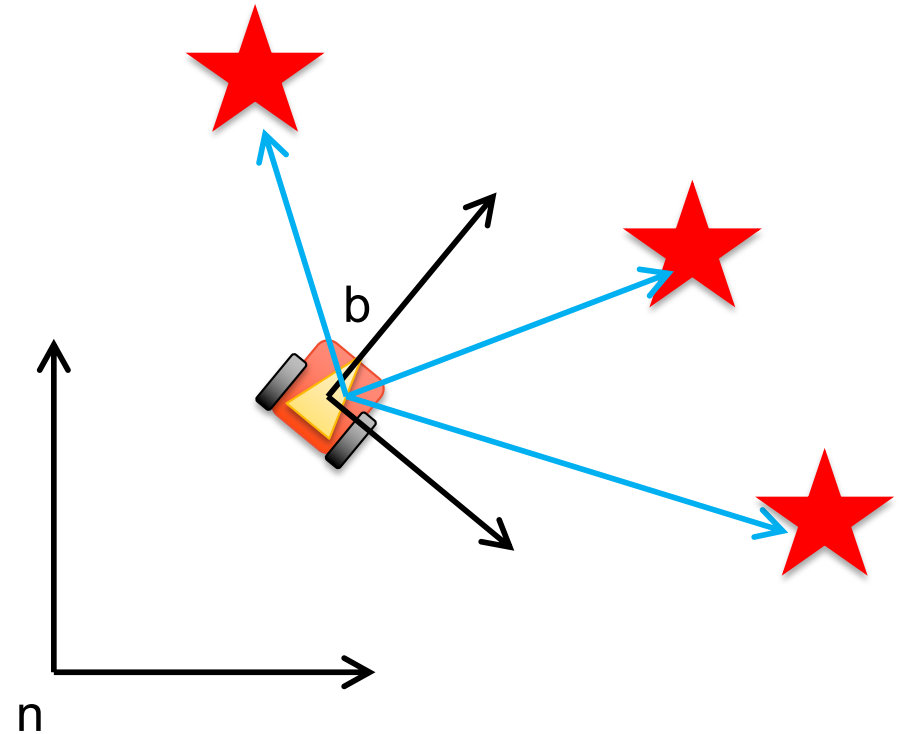


$$\mathbf{p}_l^n = \mathbf{p}^n + C_b^n \mathbf{p}_{lr}^b$$

unknown known measured

Types of Localisation and Mapping: Estimation Problem

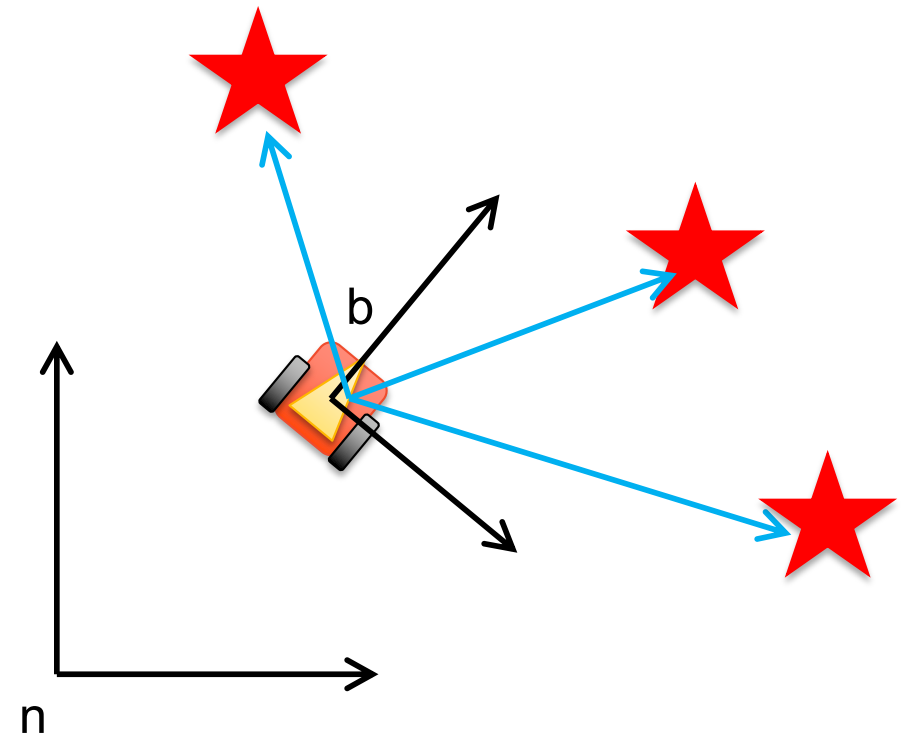
- **Beacon-based Localisation:**
 - Robot-relative measurements made to known landmarks
 - Can compute unknown pose that best matches observations and known map
 - Examples: scan/feature matching to known map, multilateration



$$\mathbf{p}_l^n = \mathbf{p}^n + C_b^n \mathbf{p}_{lr}^b$$

Types of Localisation and Mapping: Estimation Problem

- **Beacon-based Localisation:**
 - Robot-relative measurements made to known landmarks
 - Can compute unknown pose that best matches observations and known map
 - Examples: scan/feature matching to known map, multilateration

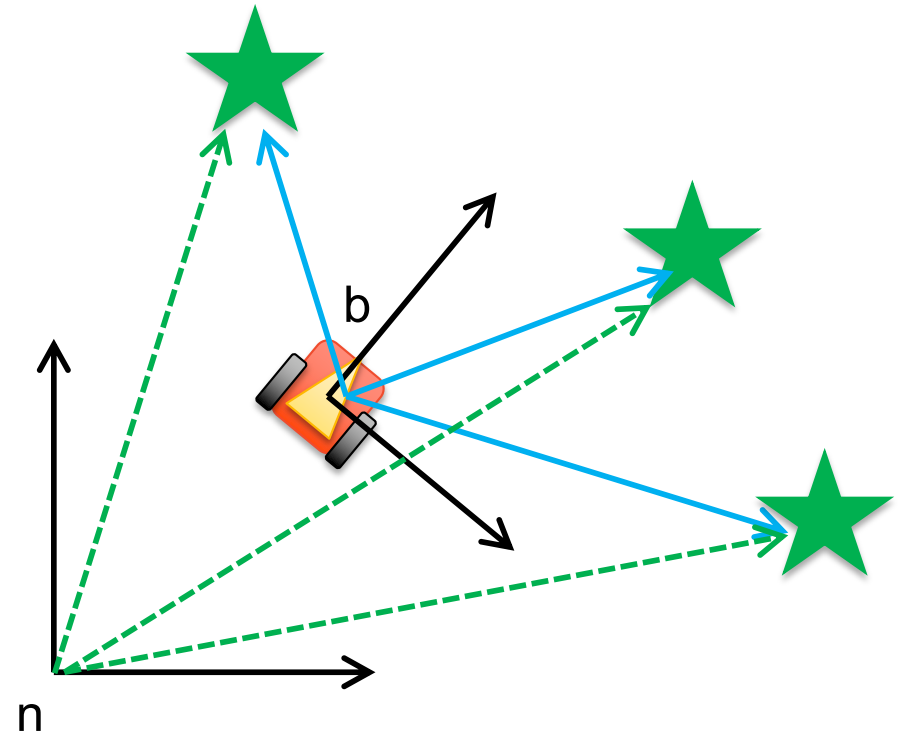


$$\mathbf{p}_l^n = \mathbf{p}^n + C_b^n \mathbf{p}_{lr}^b$$

known unknown measured

Types of Localisation and Mapping: Estimation Problem

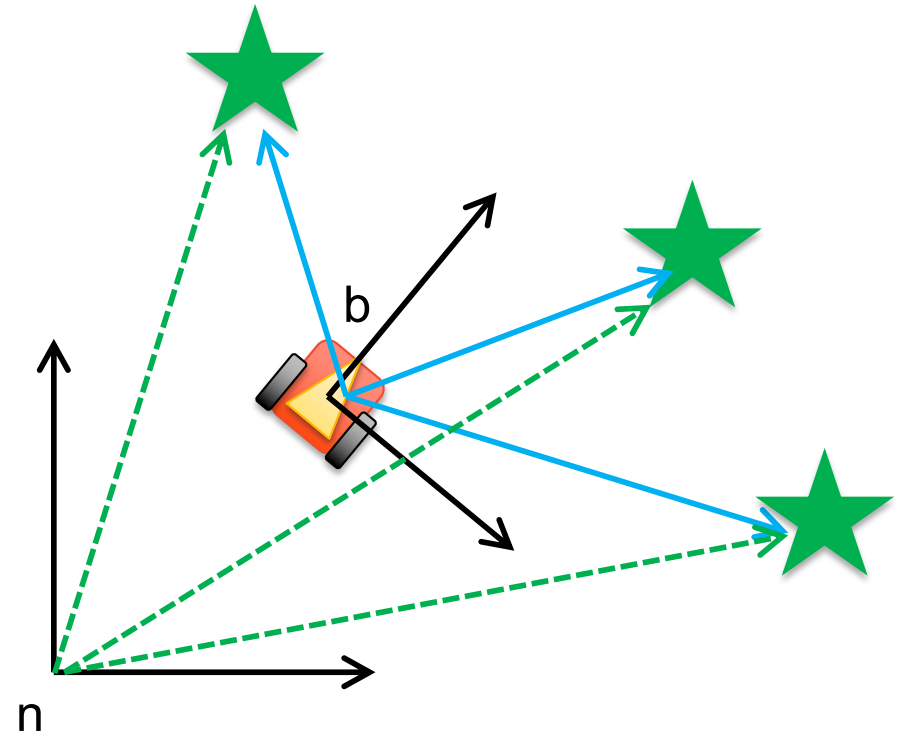
- **Simultaneous Localisation and Mapping:**
 - Robot-relative measurements made to unknown landmarks
 - Must estimate both robot and landmark location, assuming landmarks are stationary



$$\mathbf{p}_l^n = \mathbf{p}^n + C_b^n \mathbf{p}_{lr}^b$$

Types of Localisation and Mapping: Estimation Problem

- **Simultaneous Localisation and Mapping:**
 - Robot-relative measurements made to unknown landmarks
 - Must estimate both robot and landmark location, assuming landmarks are stationary

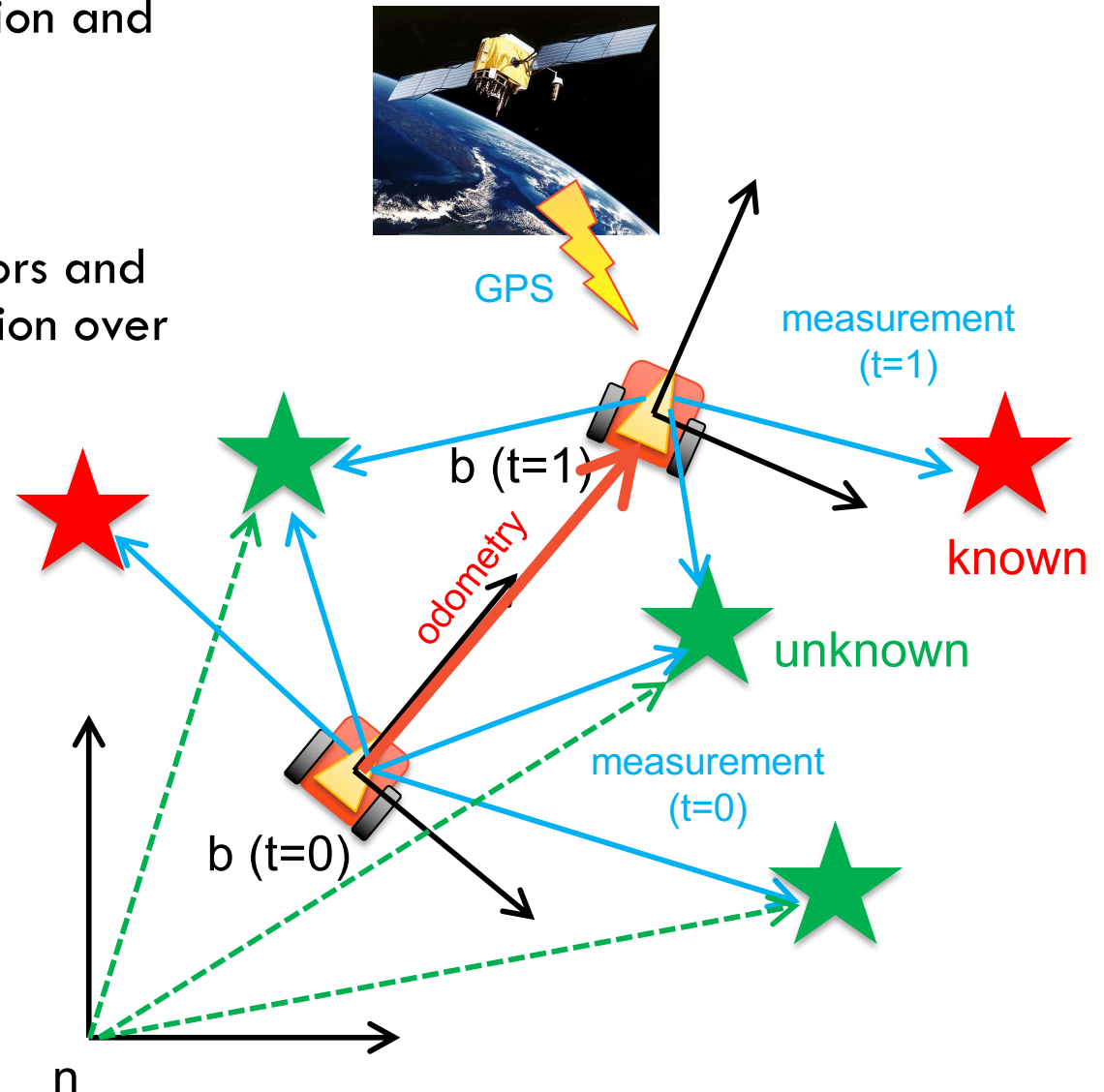


$$\mathbf{p}_l^n = \mathbf{p}^n + C_b^n \mathbf{p}_{lr}^b$$

unknown unknown measured

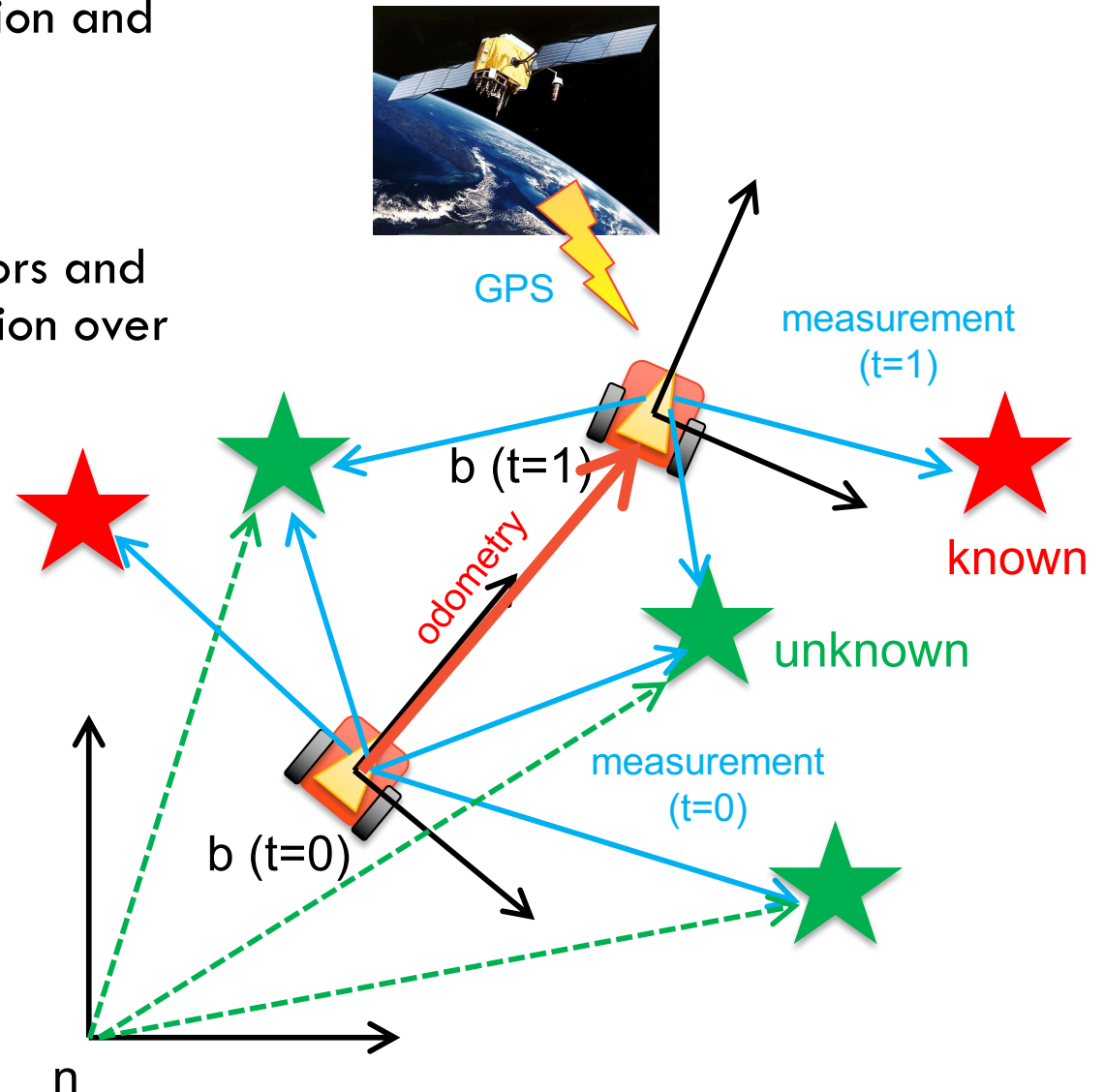
Types of Localisation and Mapping

- Many practical robot navigation and mapping problems involve combinations of information sources/estimation tasks
 - Sensors contain noise/errors and may not provide information over all poses states
- Localisation and mapping is a multi-sensor estimation problem
- Even though some estimated states are directly measured, it still may improve results to estimate/optimize these (e.g. GPS-aided SLAM)



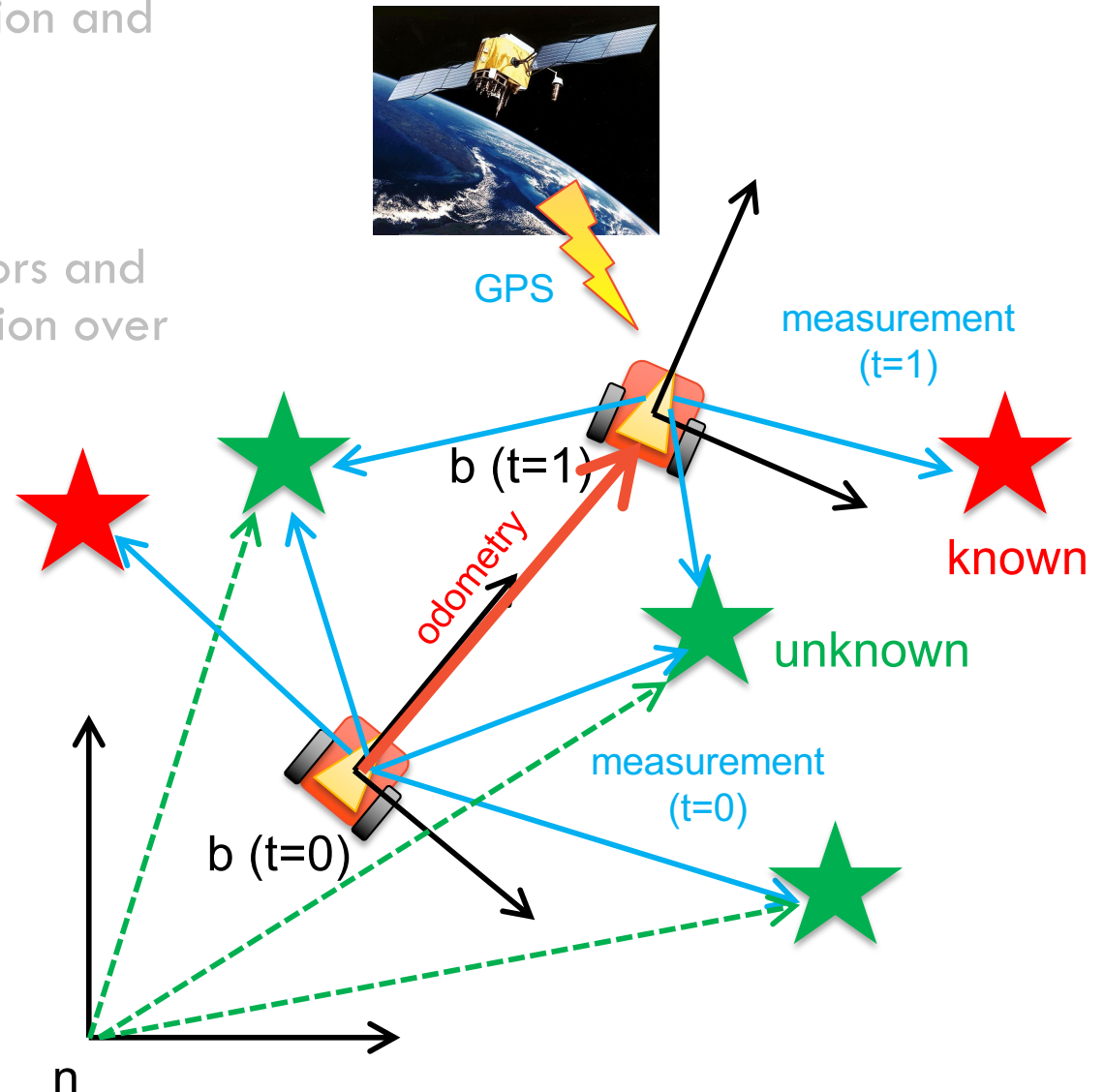
Types of Localisation and Mapping

- Many practical robot navigation and mapping problems involve combinations of information sources/estimation tasks
 - Sensors contain noise/errors and may not provide information over all poses states
- Localisation and mapping is a multi-sensor estimation problem
- Even though some estimated states are directly measured, it still may improve results to estimate/optimize these (e.g. GPS-aided SLAM)



Types of Localisation and Mapping

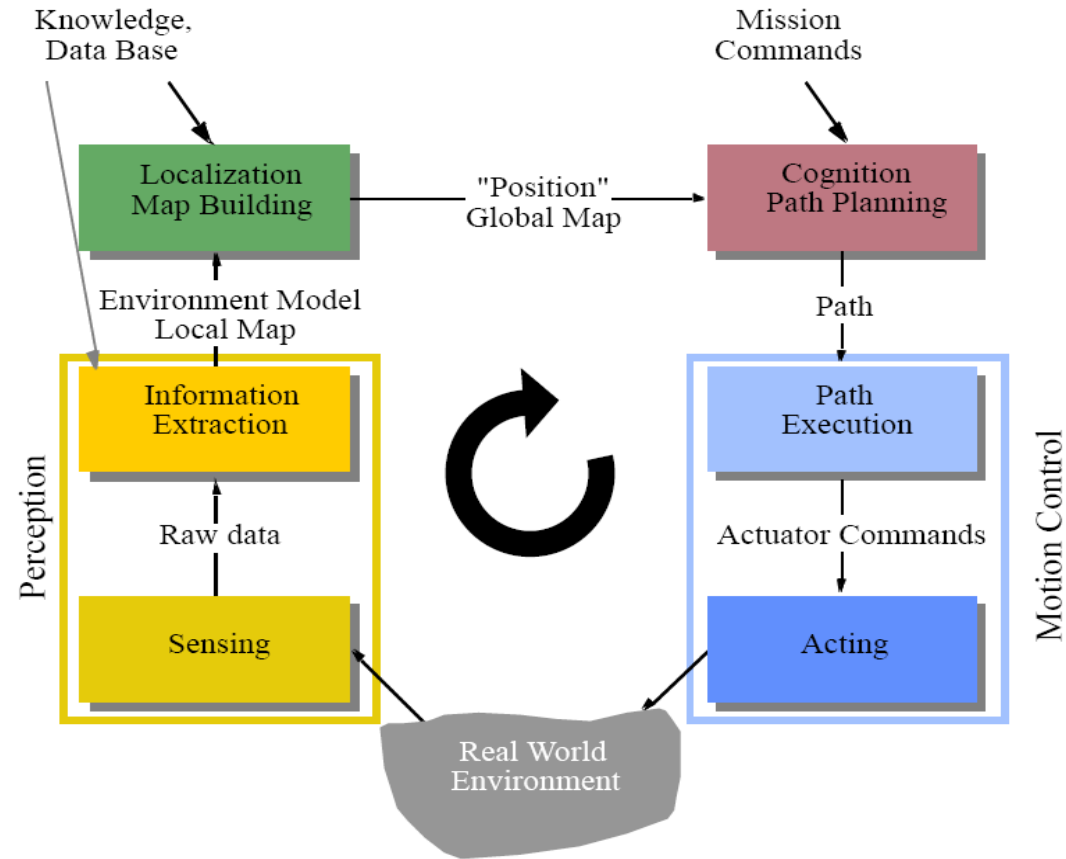
- Many practical robot navigation and mapping problems involve combinations of information sources/estimation tasks
 - Sensors contain noise/errors and may not provide information over all poses states
- Localisation and mapping is a multi-sensor estimation problem
- Even though some estimated states are directly measured, it still may improve results to estimate/optimize these (e.g. GPS-aided SLAM)



Localisation, Mapping and Estimation

- As a state-estimation problem:
 - Estimated states (x)
 - Sensor
Measurements/External
information (z)
 - Assume errors/noise on
measurements

General Control Scheme for Mobile Robot Systems



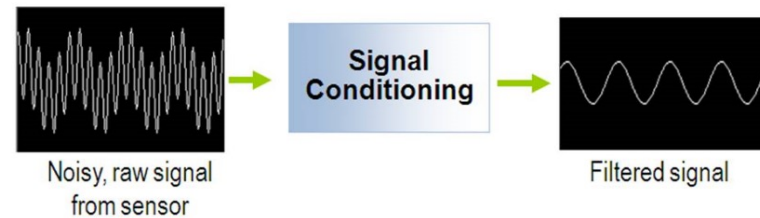
© R. Siegwart, ETH Zurich - ASL

Background assumptions

- Robots operate in worlds containing errors, e.g. Odometry, sensor, ...
- Robots operate dynamically in their world
 - Their pose and position may change at each time step
 - Need to estimate states in the future given the robot configuration and input at each step.
- Cycle through a filter at each step
 - Predict
 - Measure
 - Estimate
 - By fusing prediction and measurements

Filters

- Used in many application areas, e.g. Engineering, optics, signals, computing, philosophy...
- Manipulates the properties of a signal
 - Removes unwanted components
 - Enhances wanted components
- Action of filter often defined by transfer function
 - New value of signal based on old value
 - Can be instantaneous or
 - Time-based “recency” weighted.



Recency Weighted filters

- How much do you believe the new instantaneous reading verses historical readings?
- Simple Widrow-Hoff update
(similar to Delta rule, LMS rule, ADALINE update)

new value = old value + learning rate * (instant value – old value)

$$x(t+1) = x(t) + \beta(z(t) - x(t)) \quad 0 \leq \beta \leq 1 \text{ where}$$

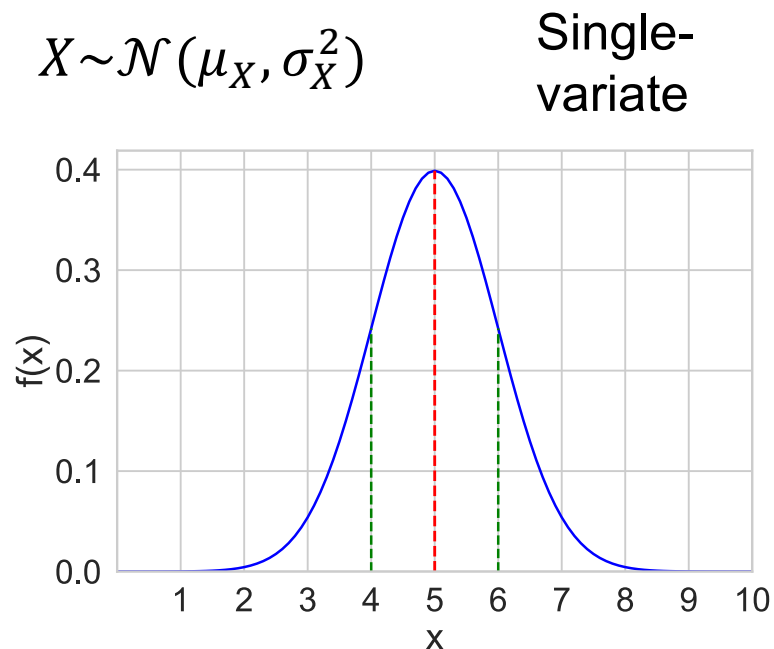
Alternatively consider ratio of instant value with old

value:

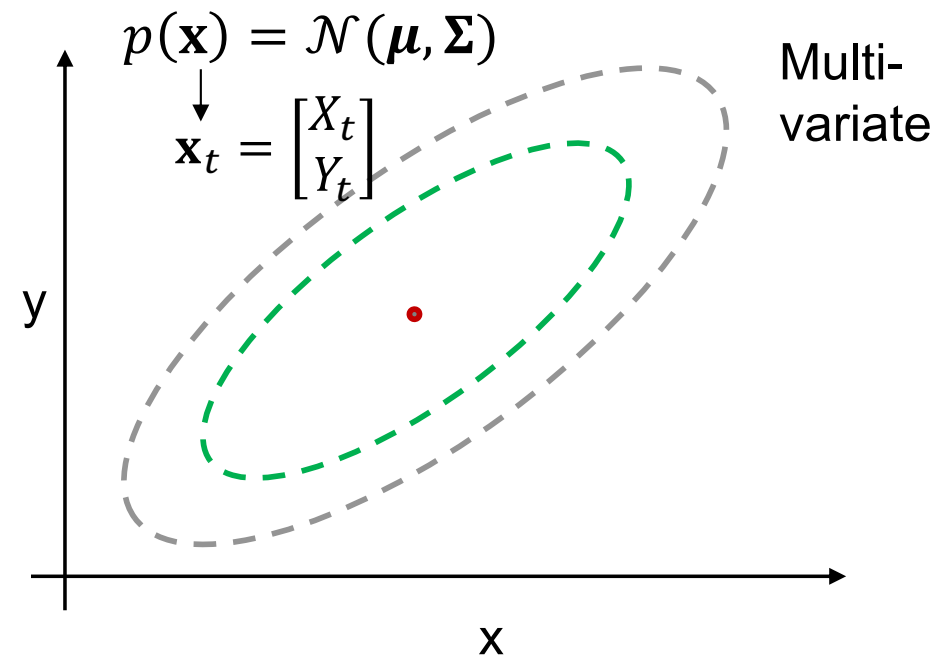
$$x(t+1) = \beta \times z(t) + (1 - \beta) \times x(t)$$

Exercise: consider $x(t+2)$, then substitute $x(t+1)$ and continue!

Learning objectives

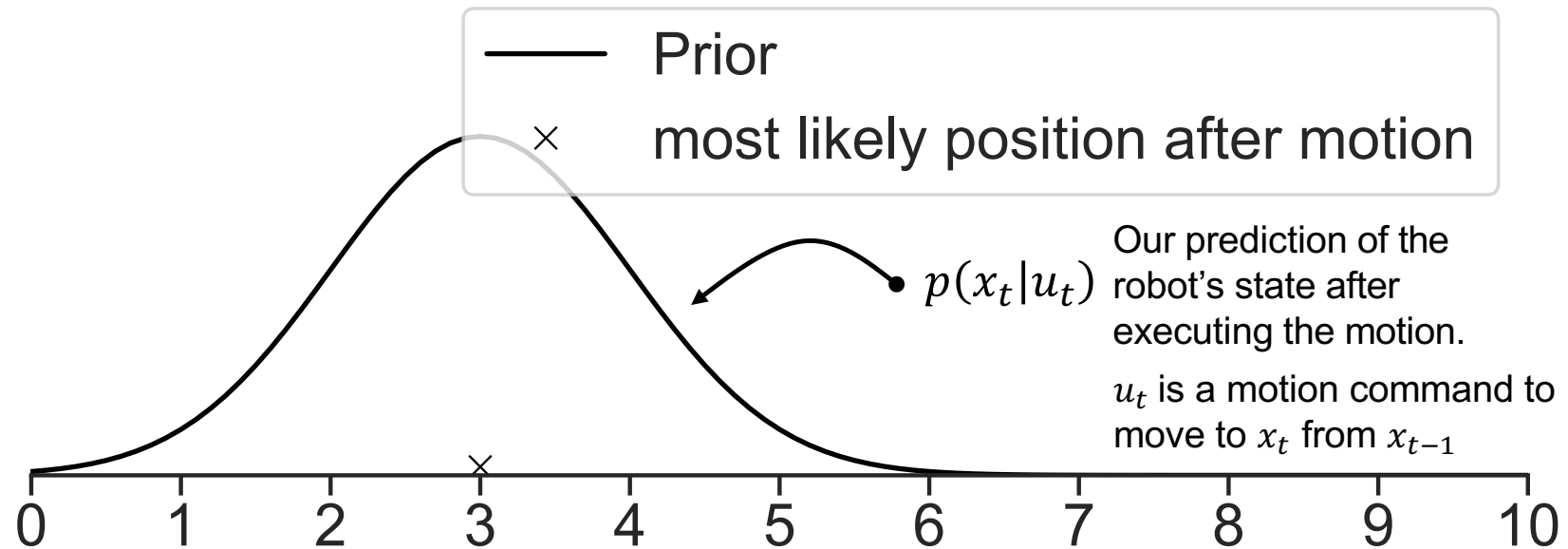


- Multivariate Normal Distribution.
- Recursive Bayesian filtering.
 - Discrete Bayesian filter
 - Linear Kalman filter



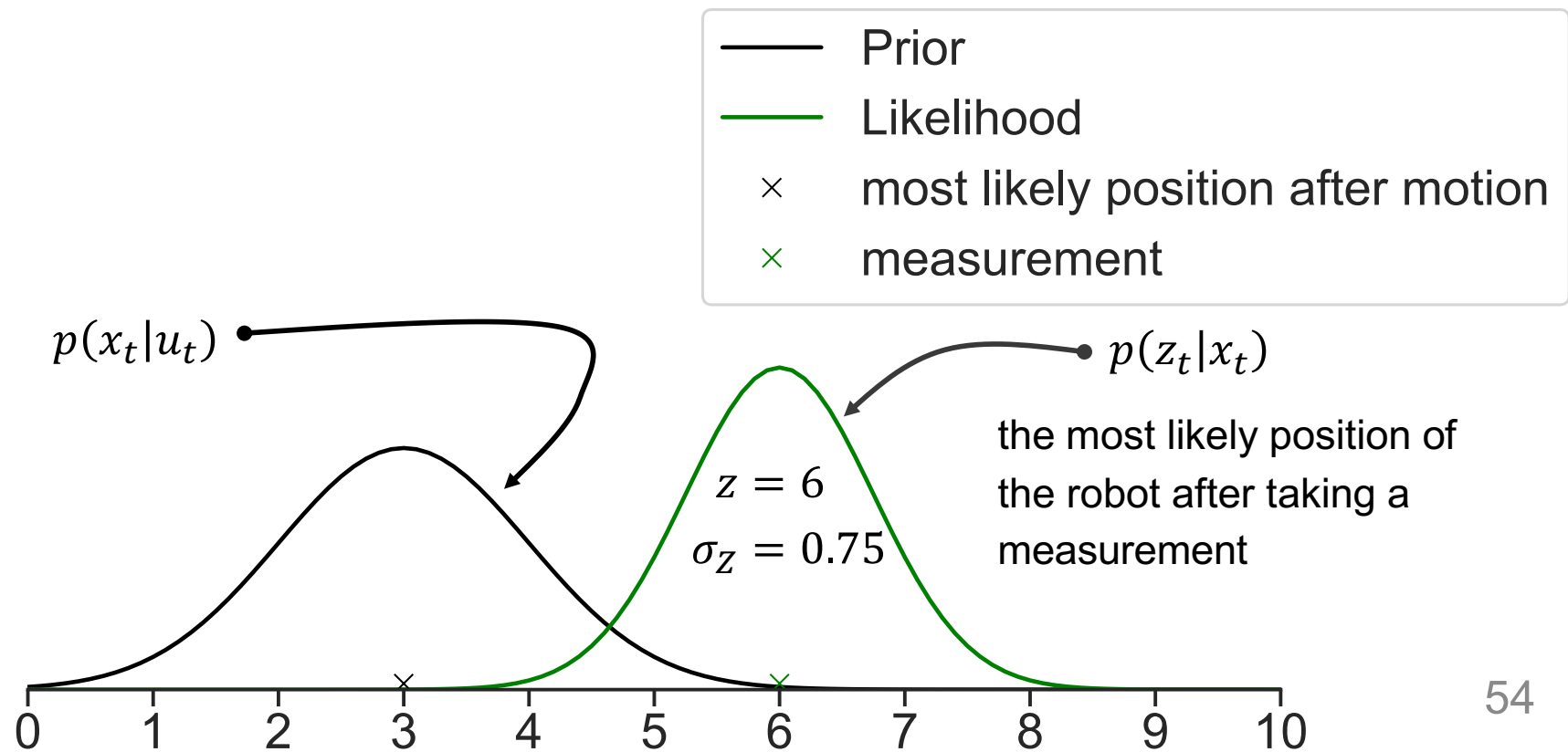
Recap

After executing a motion:



53

Recap

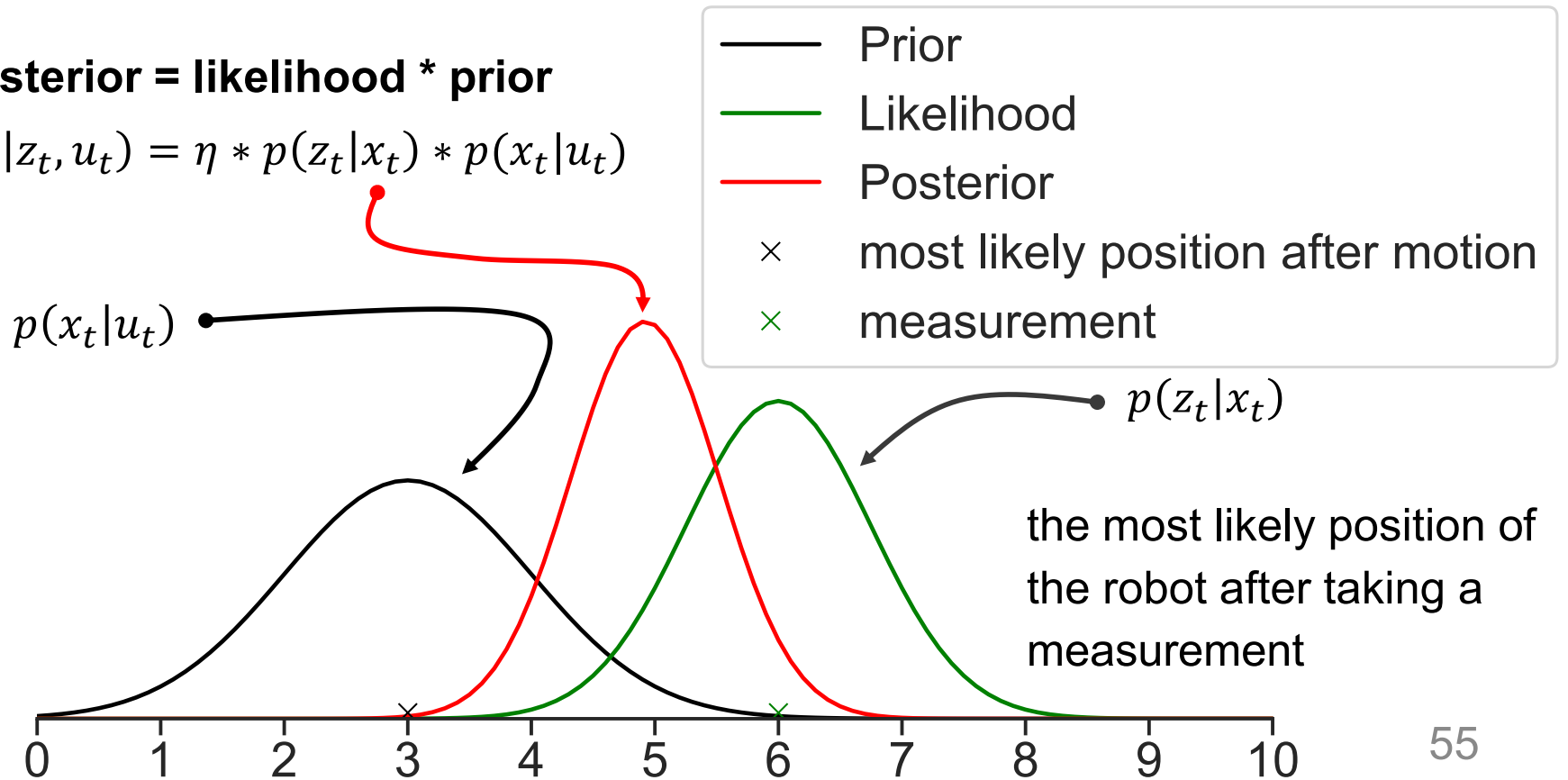


54

Recap

posterior = likelihood * prior

$$p(x_t|z_t, u_t) = \eta * p(z_t|x_t) * p(x_t|u_t)$$



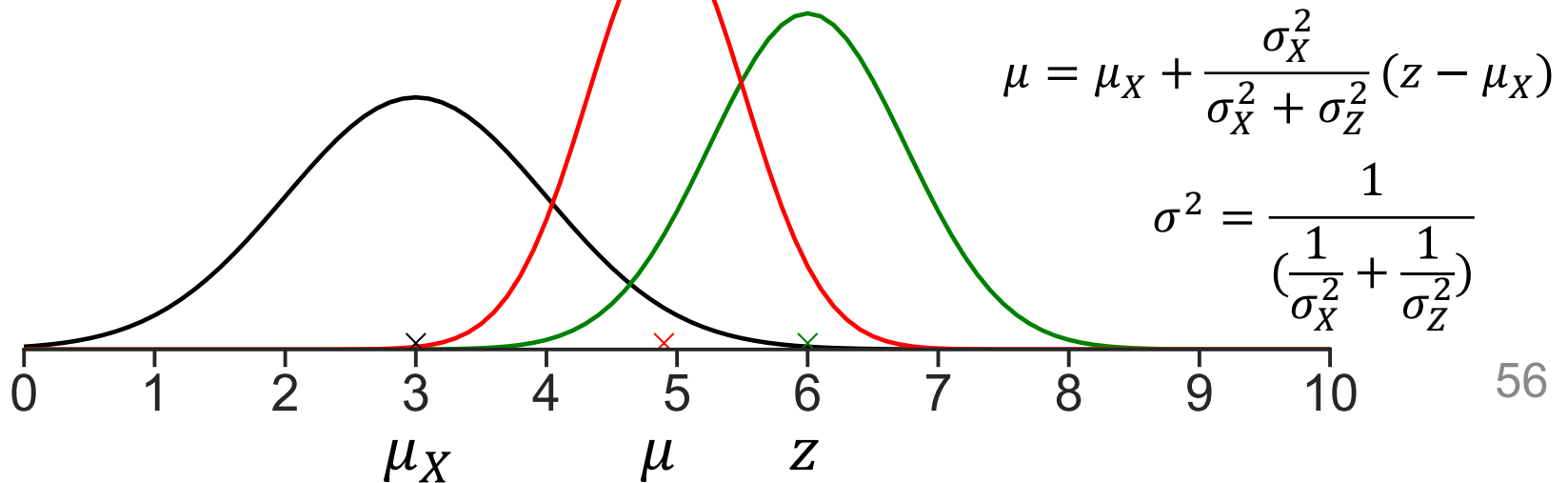
55

Recap

posterior = likelihood * prior

$$p(x_t|z_t, u_t) = \eta * p(z_t|x_t) * p(x_t|u_t)$$

- Prior
- Likelihood
- Posterior
- × most likely position after motion
- × measurement



56

Kalman Filter

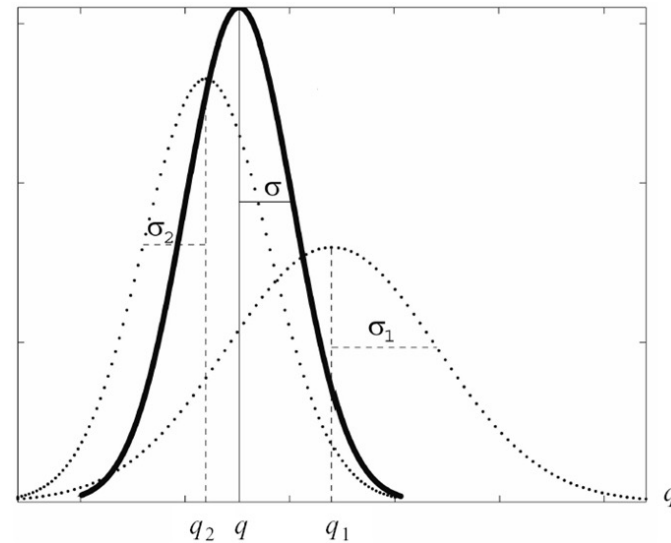
- Given values q_1 and q_2 with standard deviations σ_1 and σ_2 , obtain estimate q and its corresponding σ

$$K = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

$$q = q_1 + K(q_2 - q_1)$$

$$\sigma^2 = \sigma_1^2 - K\sigma_1^2$$

- q, σ : estimate
- q_1, σ_1 :
measurement
- q_2, σ_2 : prediction



Siegwart & Nourbakhsh, 2004

Introduction to Kalman Filter (1)

- Two measurements no dynamics

$$\hat{q}_1 = q_1 \text{ with variance } \sigma_1^2$$

$$\hat{q}_2 = q_2 \text{ with variance } \sigma_2^2$$

- Weighted least-square

$$S = \sum_{i=1}^n w_i (\hat{q} - q_i)^2$$

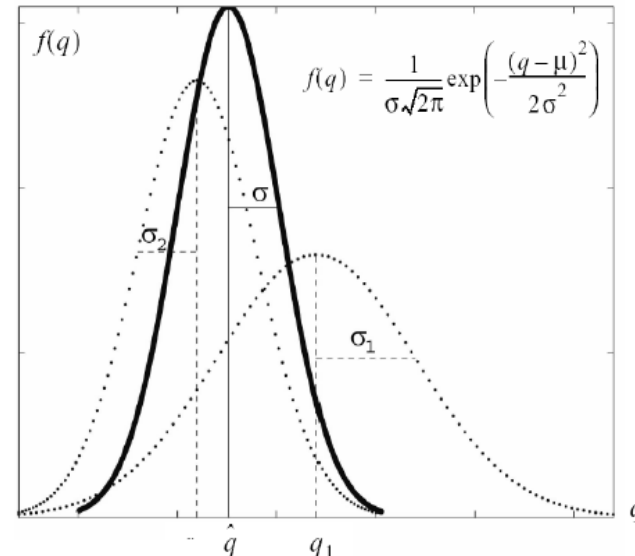
- Finding minimum error

$$\frac{\partial S}{\partial \hat{q}} = \frac{\partial}{\partial \hat{q}} \sum_{i=1}^n w_i (\hat{q} - q_i)^2 = 2 \sum_{i=1}^n w_i (\hat{q} - q_i) = 0$$

- After some calculation and rearrangements

$$\hat{q} = q_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2} (q_2 - q_1)$$

- Another way to look at it – weighed mean



cs.gmu.edu/~kosecka/cs685/handouts.htm

Discrete Kalman Filter

- Estimates the state x of a discrete-time controlled process that is governed by the linear stochastic difference equation

$$x_t = A_t x_{t-1} + B_t u_t + \varepsilon_t$$

- with a measurement

$$z_t = C_t x_t + \delta_t$$

$$A_t$$

Matrix (nxn) that describes how the state evolves from t to $t-1$ without controls or noise.

$$B_t$$

Matrix (nx1) that describes how the control u_t changes the state from t to $t-1$.

$$C_t$$

Matrix (kxn) that describes how to map the state x_t to an observation z_t .

$$\varepsilon_t$$

Random variables representing the process and measurement noise that are assumed to be independent and normally distributed with covariance R_t and Q_t respectively.

$$\delta_t$$

cs.gmu.edu/~kosecka/cs685/handouts.htm

What does a Kalman Filter do, anyway?

Given the linear dynamical system:

$$x(k+1) = F(k)x(k) + G(k)u(k) + v(k)$$

$$y(k) = H(k)x(k) + w(k)$$

$x(k)$ is the n - dimensional state vector (unknown)

$u(k)$ is the m - dimensional input vector (known)

$y(k)$ is the p - dimensional output vector (known, measured)

$F(k), G(k), H(k)$ are appropriately dimensioned system matrices (known)

$v(k), w(k)$ are zero - mean, white Gaussian noise with (known)
covariance matrices $Q(k), R(k)$

**the Kalman Filter is a recursion that provides the
“best” estimate of the state vector x .**

What does a Kalman Filter do, anyway?

Given the linear dynamical system:

new state = old state + input + noise

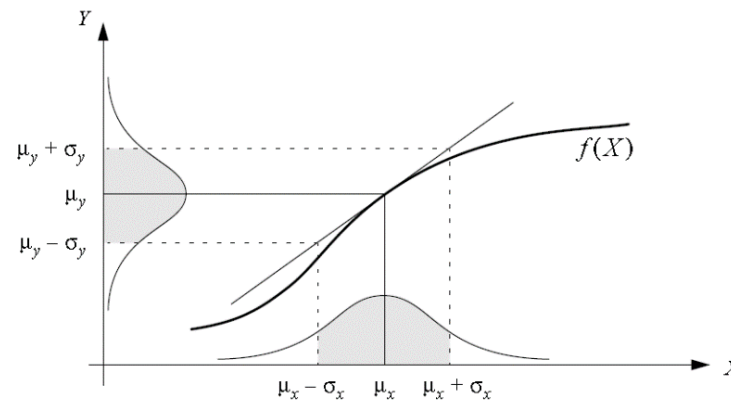
$$x(k+1) = F(k)x(k) + G(k)u(k) + v(k)$$

measurement = mapping x state + noise

$$y(k) = H(k)x(k) + w(k)$$

This is a linear transform!

$$y(k) = m \times x(k)$$



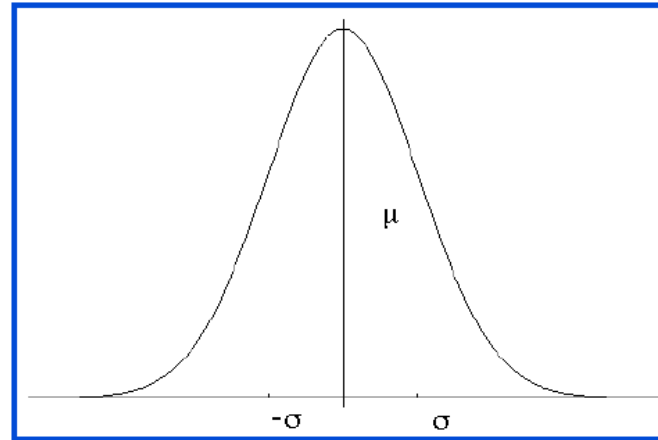
One-dimensional case of a nonlinear error propagation problem

Gaussians

$$p(x) \sim N(\mu, \sigma^2):$$

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2} \frac{(x-\mu)^2}{\sigma^2}}$$

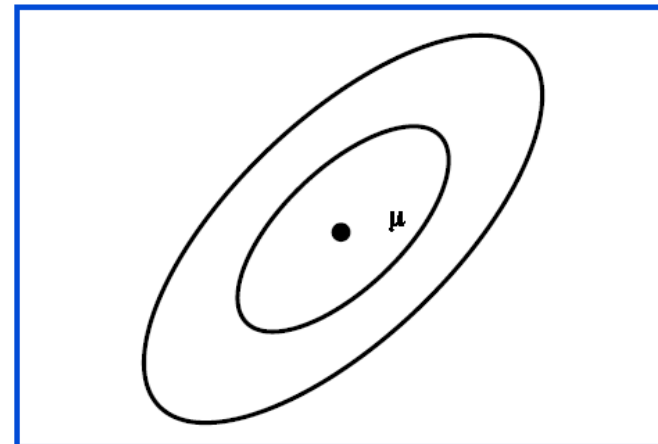
Univariate



$$p(\mathbf{x}) \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma}):$$

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}|^{1/2}} e^{-\frac{1}{2} (\mathbf{x}-\boldsymbol{\mu})' \boldsymbol{\Sigma}^{-1} (\mathbf{x}-\boldsymbol{\mu})}$$

Multivariate



cs.gmu.edu/~kosecka/cs685/handouts.htm

Properties of Gaussians

- Univariate

$$\left. \begin{array}{l} X \sim N(\mu, \sigma^2) \\ Y = aX + b \end{array} \right\} \Rightarrow Y \sim N(a\mu + b, a^2\sigma^2)$$

Linear transform of average value

$$\left. \begin{array}{l} X_1 \sim N(\mu_1, \sigma_1^2) \\ X_2 \sim N(\mu_2, \sigma_2^2) \end{array} \right\} \Rightarrow p(X_1) \cdot p(X_2) \sim N\left(\frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2} \mu_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2} \mu_2, \frac{1}{\sigma_1^{-2} + \sigma_2^{-2}}\right)$$

- Multivariate

$$\left. \begin{array}{l} X \sim N(\mu, \Sigma) \\ Y = AX + B \end{array} \right\} \Rightarrow Y \sim N(A\mu + B, A\Sigma A^T)$$

Must consider
covariates in
multivariate
analysis

$$\left. \begin{array}{l} X_1 \sim N(\mu_1, \Sigma_1) \\ X_2 \sim N(\mu_2, \Sigma_2) \end{array} \right\} \Rightarrow p(X_1) \cdot p(X_2) \sim N\left(\frac{\Sigma_2}{\Sigma_1 + \Sigma_2} \mu_1 + \frac{\Sigma_1}{\Sigma_1 + \Sigma_2} \mu_2, \frac{1}{\Sigma_1^{-1} + \Sigma_2^{-1}}\right)$$

- We stay in the "Gaussian world" as long as we start with Gaussians and perform only linear transformations

cs.gmu.edu/~kosecka/cs685/handouts.htm

Kalman Filter

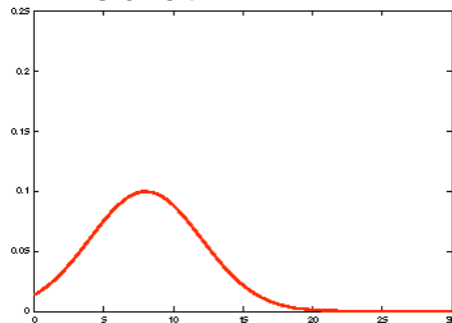
Note similarity to Widrow-Hoff update

$$\text{Estimate } bel(x_t) = \begin{cases} \mu_t = \bar{\mu}_t + K_t(z_t - \bar{\mu}_t) \\ \sigma_t^2 = (1 - K_t)\bar{\sigma}_t^2 \end{cases} \quad \text{with } K_t = \frac{\bar{\sigma}_t^2}{\bar{\sigma}_t^2 + \bar{\sigma}_{obs,t}^2} \quad \text{univariate}$$

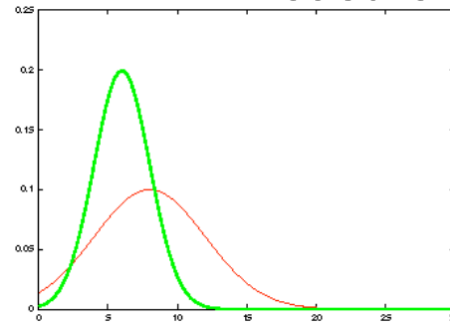
$$\text{multi } bel(x_t) = \begin{cases} \mu_t = \bar{\mu}_t + K_t(z_t - C_t \bar{\mu}_t) \\ \Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \end{cases} \quad \text{with } K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$$

measurement – mapping x
predicted mean state

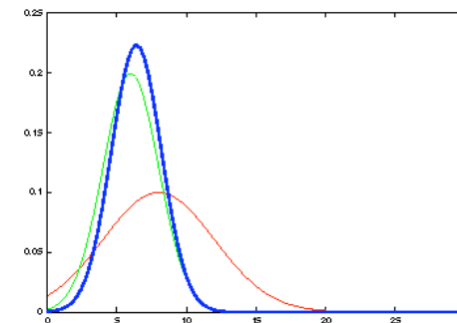
Predict



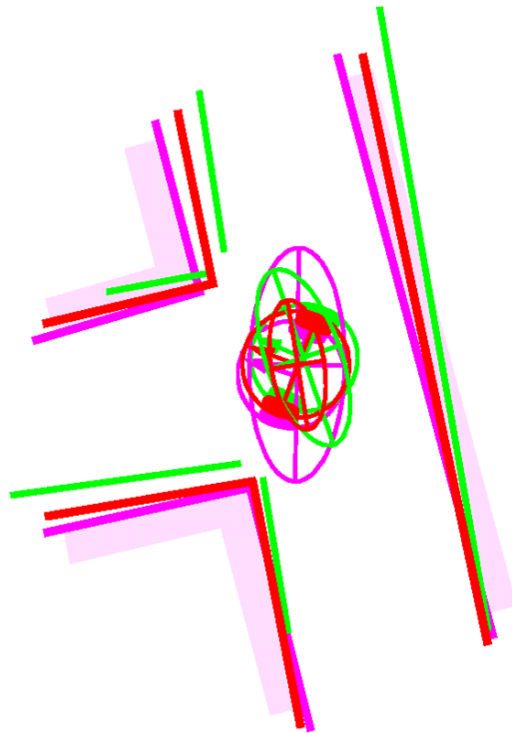
Measure



Estimate



Kalman Filter

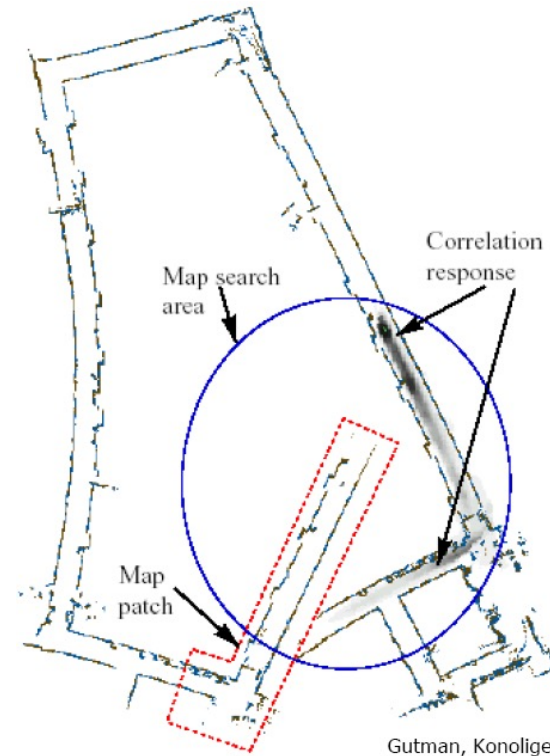


- Prediction
- Measurement
- Kalman filter estimate

Siegwart & Nourbakhsh, 2004

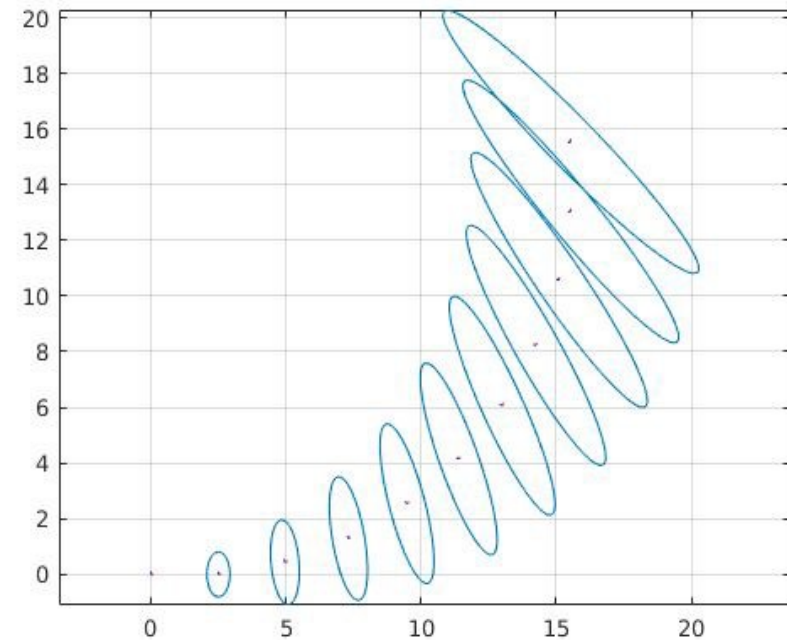
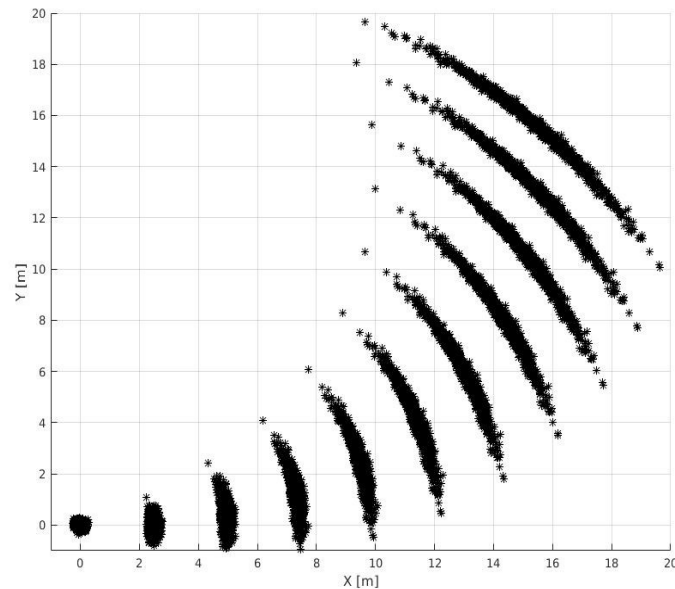
Simultaneous Localisation and Mapping (SLAM)

- Attempts to solve chicken-and-egg problem:
 - To know your location, you need a map
 - To build a map, you need to know your location
- Difficulties:
 - Different locations may look identical to the robot's sensors
 - The same location may look different when viewed from a different perspective
 - Or it may change over time (dynamic environments)
 - How to decide whether robot has returned to a previously visited location? (loop closing)
- A very active research area



Learning objectives

- Extended Kalman filter.
- Landmark-based localization.
- Range and bearing sensors.



Recap

The Kalman Filter Steps

Prediction:

$$\bar{\mu}_t = \mathbf{A}\mu_{x_{t-1}} + \mathbf{B}\mathbf{u}_t$$

$$\bar{\Sigma}_t = \mathbf{A}\Sigma_{t-1}\mathbf{A}^T + \mathbf{R}$$

Update/Correction:

$$\mu_t = \bar{\mu}_t + \mathbf{K}_t(\mathbf{z}_t - \mathbf{H}\bar{\mu}_t)$$

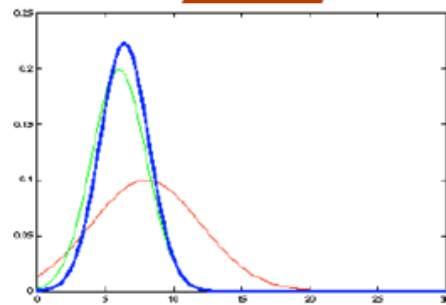
$$\Sigma_t = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\bar{\Sigma}_t$$

$$\mathbf{K}_t = \bar{\Sigma}_t\mathbf{H}^T(\mathbf{H}_t\bar{\Sigma}_t\mathbf{H}^T + \mathbf{Q})^{-1}$$

CRICOS No. 00213J

Kalman Filter: next step

We have estimated position → now we move the robot with known input (u_t)



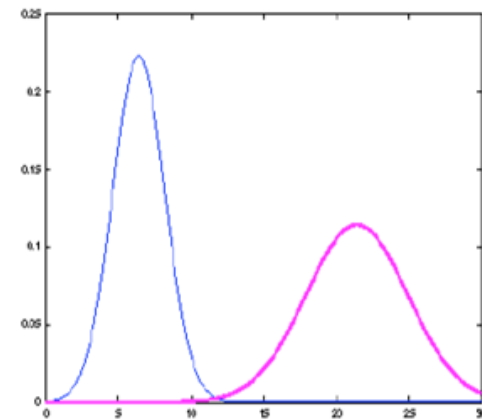
Prediction

$$\overline{bel}(x_t) = \begin{cases} \bar{\mu}_t = a_t \mu_{t-1} + b_t u_t \\ \bar{\sigma}_t^2 = a_t^2 \sigma_{t-1}^2 + \sigma_{act,t}^2 \end{cases}$$

$$\overline{bel}(x_t) = \begin{cases} \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \\ \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t \end{cases}$$

Kalman Filter Algorithm

1. Algorithm **Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):
2. Prediction:
3. $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$
4. $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$
5. Correction:
6. $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$
7. $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$
8. $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$
9. Return μ_t, Σ_t



cs.gmu.edu/~kosecka/cs685/handouts.htm

Kalman filter assumptions

- The state and the noise are normally distributed.
- The motion and the measurement models are linear.



- This lecture tackles the case when this is not true.

- If the above is met then Kalman filter is the optimal solution!

Odometry-based state transition function

$f(\mathbf{x}_{t-1}, \mathbf{u}_t)$ Nonlinear

Distance traveled between t-1 and t

$$\mathbf{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} = \begin{bmatrix} x_{t-1} + \delta d \times \cos(\theta_{t-1}) \\ y_{t-1} + \delta d \times \sin(\theta_{t-1}) \\ \theta_{t-1} + \delta \theta \end{bmatrix}$$

Change in heading between t-1 and t

CRICOS No. 00213J

Can we still use this motion model?

linear



$$\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1} + \mathbf{B}\mathbf{u}_t + \mathbf{v}_t$$

We want this to be Gaussian

nonlinear



$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t) + \mathbf{v}_t,$$

odometry

$$\mathcal{N}(\mu_{x_{t-1}}, \Sigma_{t-1})$$

CRICOS No. 00213J

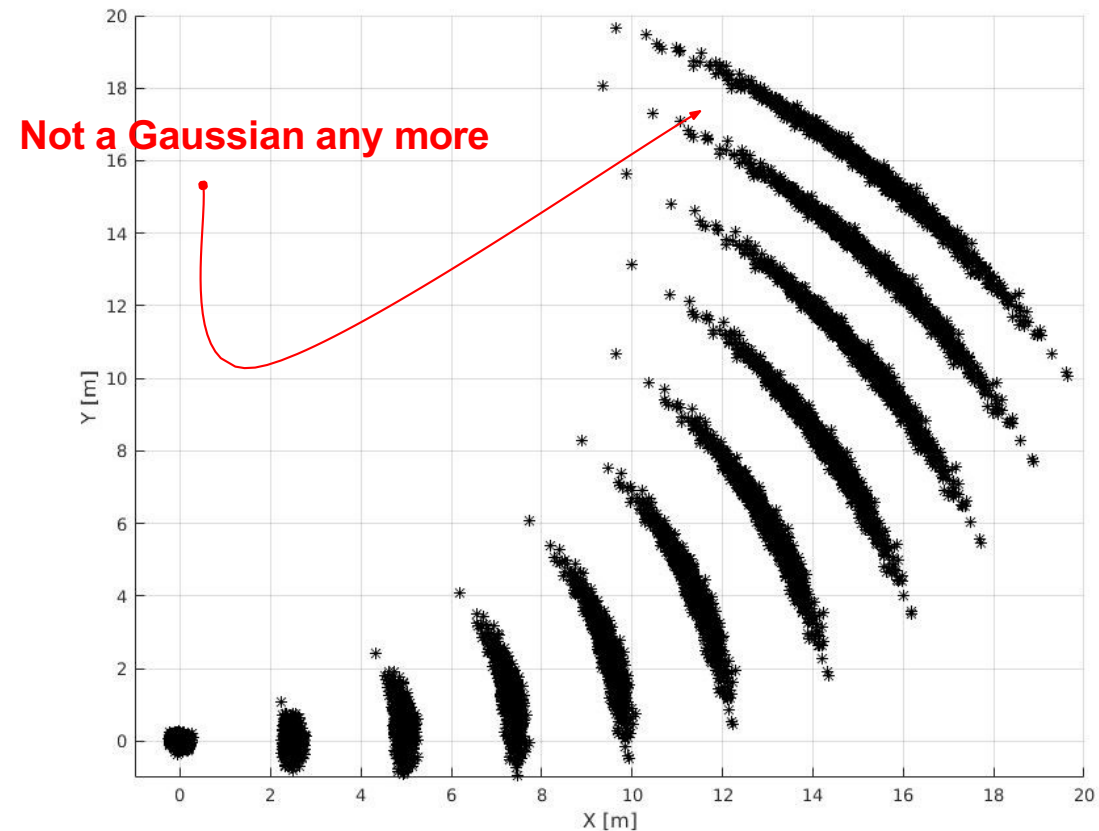
$$\mathbf{v}_t \sim \mathcal{N}(0, \mathbf{R})$$

What happens to Gaussians when they pass through nonlinear functions?

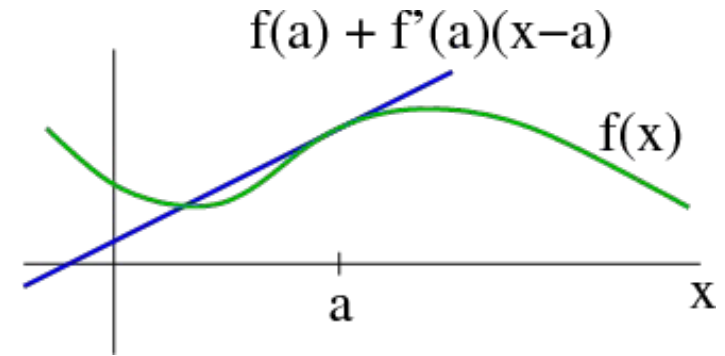
```
clear all
N = 1000; % sample N points from a Gaussian
X = mvnrnd([0,0,0],[0.01 0 0;
                  0 0.01 0;
                  0 0 0.01],N);

figure(1)
clf
hold on
axis([-1 20 -1 20])
scatter(X(:,1),X(:,2),'k*')

Y = zeros(N,3);
for s = 1:10 % do 10 steps
    delta_d = 2.5; % move
    delta_theta = 10 * pi / 180; % turn
    for i = 1:N % pass all the points through f
        Y(i,1) = X(i,1) + delta_d * cos(X(i,3));
        Y(i,2) = X(i,2) + delta_d * sin(X(i,3));
        Y(i,3) = X(i,3) + delta_theta;
    end
    scatter(Y(:,1),Y(:,2),'k*')
    X = Y;
end
```



Linearization: First Order Taylor Series Expansion



$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t) + \mathbf{v}_t,$$

$$f(\mathbf{x}_{t-1}, \mathbf{u}_t) = f(\mu_{t-1}, \mathbf{u}_t) + \frac{\partial f(\mu_{t-1}, \mathbf{u}_t)}{\partial \mathbf{x}_{t-1}} (\mathbf{x}_{t-1} - \mu_{t-1})$$

$$f(\mathbf{x}_{t-1}, \mathbf{u}_t) = f(\mu_{t-1}, \mathbf{u}_t) + \mathbf{J}_{x_t} (\mathbf{x}_{t-1} - \mu_{t-1})$$

CRICOS No. 00213J

Jacobian matrix calculated at each time step

Jacobian Matrix

Given a set $y=f(x)$ of n equations in n variables $x_1, \dots, x_n$, written explicitly as

$$\mathbf{y} \equiv \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_n(\mathbf{x}) \end{bmatrix} \quad \text{or} \quad \begin{cases} y_1 = f_1(x_1, \dots, x_n) \\ \vdots \\ y_n = f_n(x_1, \dots, x_n), \end{cases}$$

The Jacobian matrix, sometimes simply called "the Jacobian" (Simon and Blume 1994) is the matrix of all first-order partial derivatives of a vector- (or scalar)-valued function with respect to another vector:

$$\mathbf{J}(x_1, \dots, x_n) = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial x_1} & \dots & \frac{\partial y_n}{\partial x_n} \end{bmatrix}.$$

The determinant of $\mathbf{J}$ is the Jacobian determinant (confusingly, often called "the Jacobian" as well) and is denoted

$$J = \left| \frac{\partial(y_1, \dots, y_n)}{\partial(x_1, \dots, x_n)} \right|.$$

<http://mathworld.wolfram.com/Jacobian.html>

The Extended Kalman Filter: Prediction step

$$\bar{\mu}_t = f(\mu_{t-1}, \mathbf{u}_t)$$

$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{R}$$

What if the noise in the odometry is not simply additive

Jacobian matrix w.r.t pose

$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$

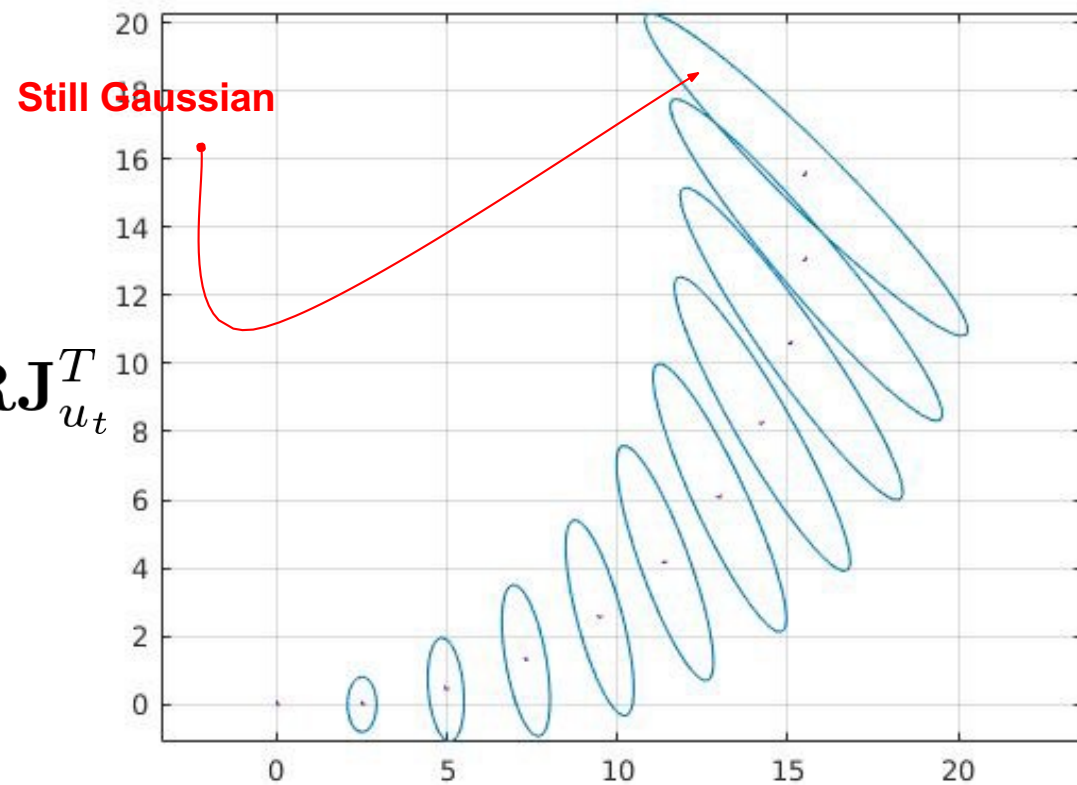
Jacobian matrix w.r.t odometry

CRICOS No. 00213J

Linearization keeps it normal!

$$\bar{\mu}_t = f(\mu_{t-1}, \mathbf{u}_t)$$

$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$



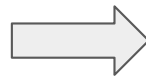
Nonlinear measurement model

linear



$$\mathbf{z}_t = \mathbf{H}\mathbf{x}_t + \mathbf{w}_t$$

nonlinear



$$\mathbf{z}_t = h(\mathbf{x}_t) + \mathbf{w}_t$$

Most of the time the measurement model is not linear as well

CRICOS No. 00213J
 $\mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q})$

Linearization: First Order Taylor Series Expansion

$$\mathbf{z}_t = h(\mathbf{x}_t) + \mathbf{w}_t$$

$$h(\mathbf{x}_t) = h(\mu_t) + \frac{\partial h(\mu_t)}{\partial \mathbf{x}_t} (\mathbf{x}_t - \mu_t)$$

$$h(\mathbf{x}_t) = h(\mu_t) + \mathbf{G}_t (\mathbf{x}_t - \mu_t)$$

Jacobian matrix calculated at each time step

The extended Kalman filter: Update step

$$\mu_t = \bar{\mu}_t + \mathbf{K}_t(\mathbf{z}_t - h(\bar{\mu}_t))$$

$$\Sigma_t = (\mathbf{I} - \mathbf{K}_t \mathbf{G}_t) \bar{\Sigma}_t$$

$$\mathbf{K}_t = \bar{\Sigma}_t \mathbf{G}_t^T (\mathbf{G}_t \bar{\Sigma}_t \mathbf{G}_t^T + \mathbf{Q})^{-1}$$

CRICOS No. 00213J

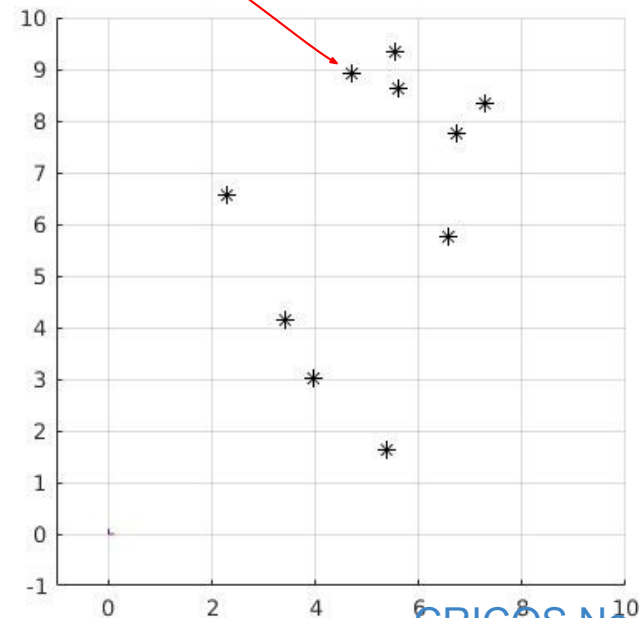
Landmark-based localization

- Given a set of landmarks with known positions (we will call this set our map).
- Given that we have a sensor onboard the robot that can detect these landmarks (we will assume the sensor can give us the range and bearing to each landmark in the map relative to the robot).
- Given that we have an idea about the motion of the robot through the odometry information.

We want to track the pose of the robot in the map while it is moving around.

Landmark-based map

$$\mathbf{M} = \begin{bmatrix} x_{l_1} \\ y_{l_1} \\ \vdots \\ x_{l_n} \\ y_{l_n} \end{bmatrix}$$



CRICOS No. 00213J

Range and bearing measurement model

$$h(\bar{\mu}_t, i) = \begin{bmatrix} r^i \\ \beta^i \end{bmatrix} = \begin{bmatrix} \sqrt{(x_r - x_{l_i})^2 + (y_r - y_{l_i})^2} \\ \text{atan2}(y_{l_i} - y_r, x_{l_i} - x_r) - \theta_r \end{bmatrix}$$

Range and bearing with respect to robot's own frame of reference at time step t .

Coordinates of landmark i

Robot's pose

Jacobians Matrices

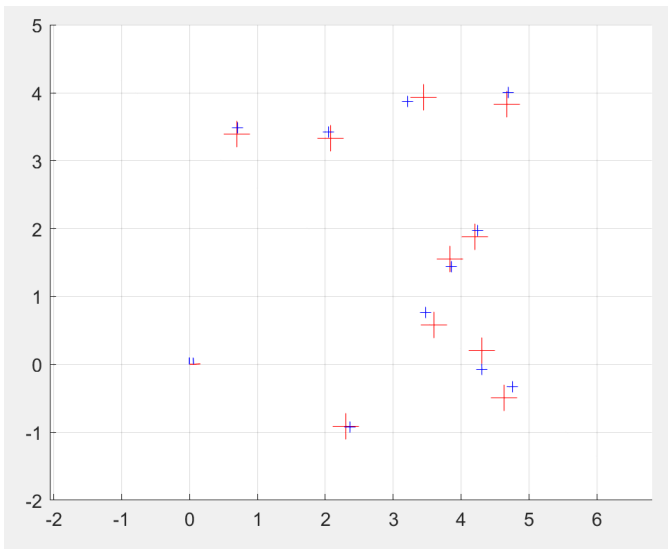
$$\mathbf{J}_x = \begin{bmatrix} 1 & 0 & -\delta d \times \sin(\theta) \\ 0 & 1 & \delta d \times \cos(\theta) \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{J}_u = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix}$$

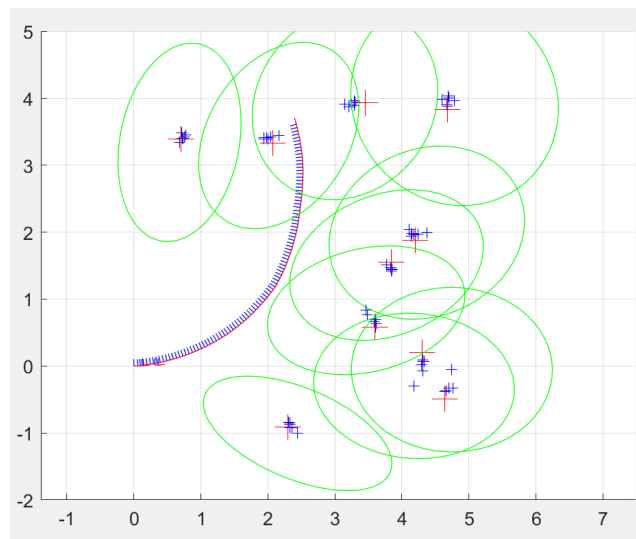
$$\mathbf{G} = \begin{bmatrix} -\frac{x_l - x_r}{r} & -\frac{y_l - y_r}{r} & 0 \\ \frac{y_l - y_r}{r^2} & -\frac{x_l - x_r}{r^2} & -1 \end{bmatrix}$$

CRICOS No. 00213J

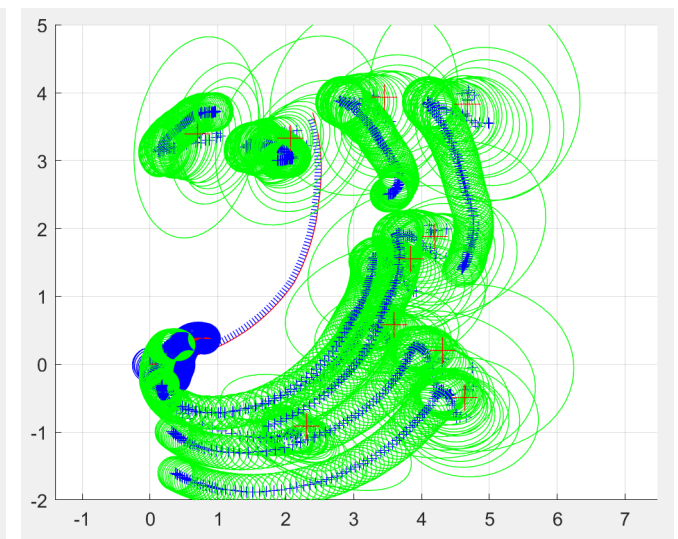
Not SLAM



Ground truth (red -unknown)



Just Odometry



Just Localization

Putting it all together

Prediction step:

$$\bar{\mu}_t = f(\mu_{t-1}, \mathbf{u}_t)$$

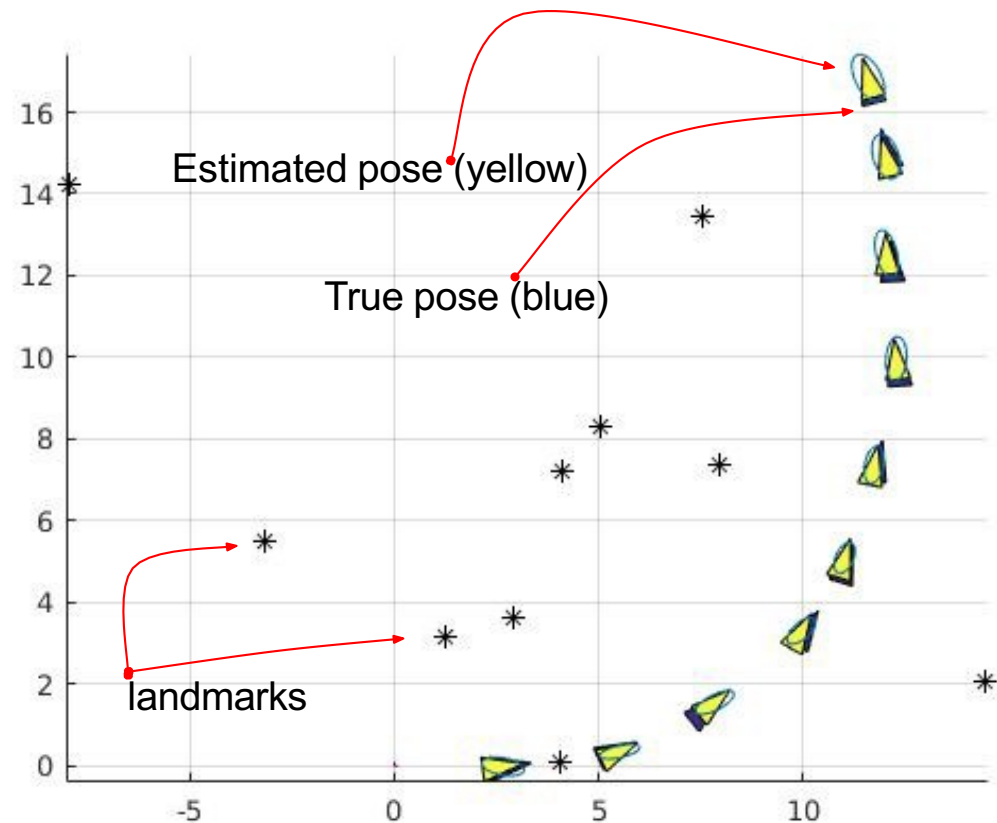
$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$

Update step:

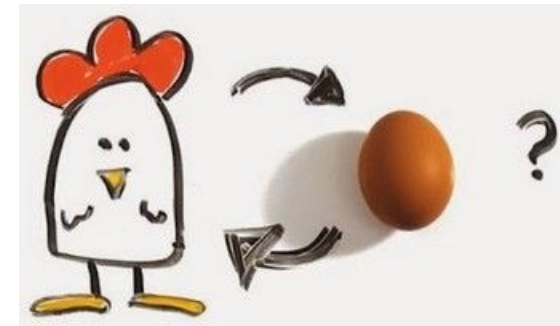
For each landmark do:

$$\mu_t = \bar{\mu}_t + \mathbf{K}_t(\mathbf{z}_t - h(\bar{\mu}_t))$$

$$\Sigma_t = (\mathbf{I} - \mathbf{K}_t \mathbf{G}_t) \bar{\Sigma}_t$$



Simultaneous **L**ocalization **A**nd **M**apping



Feras Dayoub

CRICOS No. 00213J

Learning objectives

- SLAM using an extended Kalman filter.

Recap

Prediction step:

$$\bar{\mu}_t = f(\mu_{t-1}, \mathbf{u}_t)$$

$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$

Update step:

For each observed landmark do:

$$\bar{\mu}_t = \bar{\mu}_t + \mathbf{K}_t^i (\mathbf{z}_t^i - h(\bar{\mu}_t, i))$$

$$\bar{\Sigma}_t = (\mathbf{I} - \mathbf{K}_t^i \mathbf{G}_t^i) \bar{\Sigma}_t$$

localization

Prediction step:

$$\bar{\mu}_t = \mu_{t-1}$$

$$\bar{\Sigma}_t = \Sigma_{t-1}$$

Update step:

For each observed landmark do:

$$\bar{\mu}_t = \bar{\mu}_t + \mathbf{K}_t^i (\mathbf{z}_t^i - h(\bar{\mu}_t, i, \mathbf{x}_t^r))$$

$$\bar{\Sigma}_t = (\mathbf{I} - \mathbf{K}_t^i \mathbf{G}_t^i) \bar{\Sigma}_t$$

mapping

CRICOS No. 00213J

Assumptions

- The robot does not know its pose in the map.
- The wheel encoders are noisy.
- The robot does not know the position of the landmarks in the map.
- The sensor onboard the robot is noisy.
- The robot can associate the measurements with the landmarks.

– The task

- The robot should **localize itself** inside a map using a set of landmarks and at the same time use its pose and sensor **to map** the positions of the landmarks.

CRICOS No. 00213J

SLAM: the chicken or egg problem

- We need the position of the landmarks (i.e the map) to estimate the pose of the robot.
- And in order to estimate the position of the landmarks in the map we need the true pose of the robot...
- In this lecture we are going to do the two above processes at the same time. This is called simultaneous localisation and mapping (SLAM).

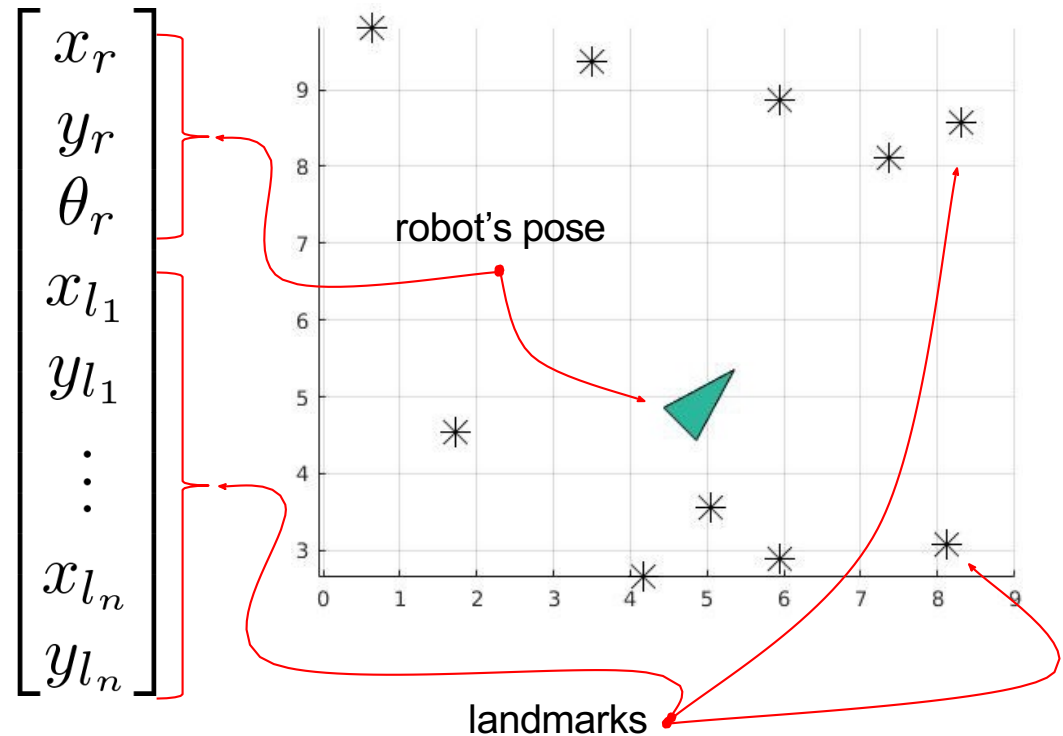
**Localize yourself in a map that you are building
using the estimation of your pose in it!**

CRICOS No. 00213J

The state vector

- The state vector contains both the pose of the robot and the positions of the landmarks in the map.

$$\mathbf{x}_t = \begin{bmatrix} \mathbf{x}^r \\ M \end{bmatrix} =$$



CRICOS No. 00213J

We still live in a Gaussian world!

$$\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$$

$$\mu = \begin{bmatrix} \mu^r \\ \mu_{l1} \\ \vdots \\ \mu_{ln} \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} \Sigma_{\mathbf{x}^r} \\ \Sigma_{l_1 \mathbf{x}^r} \\ \vdots \\ \Sigma_{l_n \mathbf{x}^r} \end{bmatrix}$$

You used this in the localization case

You used this in the mapping case

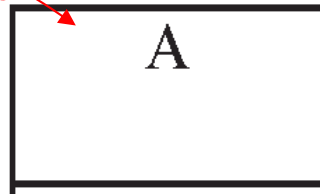
$$\begin{bmatrix} \Sigma_{\mathbf{x}^r l_1} & \dots & \Sigma_{\mathbf{x}^r l_n} \\ \Sigma_{l_{11}} & \dots & \Sigma_{l_{1n}} \\ \vdots & \ddots & \vdots \\ \Sigma_{l_{n1}} & \dots & \Sigma_{l_{nn}} \end{bmatrix}$$

The mean vector of the robot pose

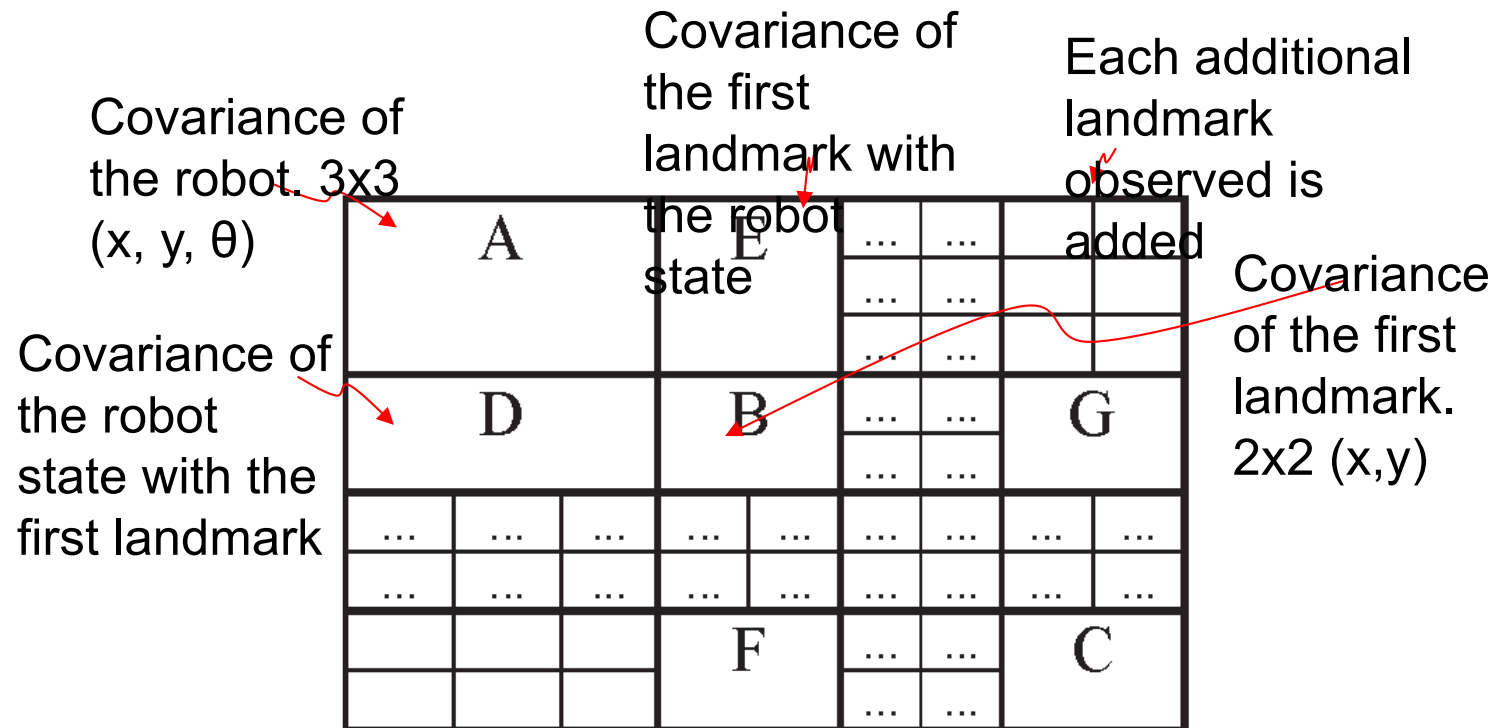
CRICOS No. 00213J

Covariance matrix

Covariance of
the robot. 3x3
(x, y, θ)



Covariance matrix

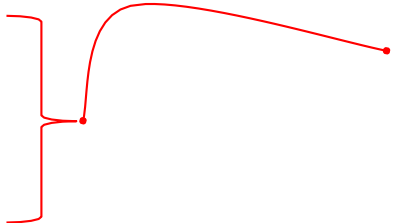


The same set of equations

Prediction step:

$$\bar{\mu}_t = f(\mu_{t-1}, \mathbf{u}_t)$$

$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$

- 
- We predict both the pose of the robot and the positions of the landmarks.
 - In the prediction step the robot moves and the landmarks stay static.

Update step:

For each landmark $\mathbf{z}_t^i$ do:

$$\bar{\mu}_t = \bar{\mu}_t + \mathbf{K}_t^i (\mathbf{z}_t^i - h(\bar{\mu}_t, i))$$

$$\bar{\Sigma}_t = (\mathbf{I} - \mathbf{K}_t^i \mathbf{G}_t^i) \bar{\Sigma}_t$$

end

$$\begin{aligned} \mu_t &= \bar{\mu}_t \\ \Sigma_t &= \bar{\Sigma}_t \end{aligned}$$

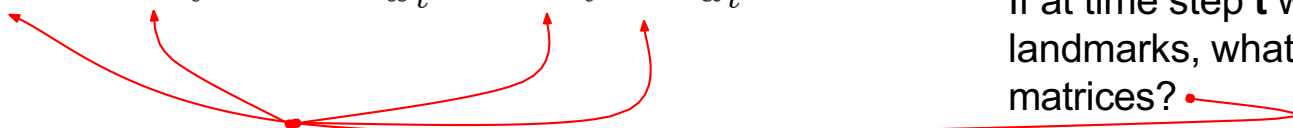
$$\mathbf{K}_t = \bar{\Sigma}_t \mathbf{G}_t^T (\mathbf{G}_t \bar{\Sigma}_t \mathbf{G}_t^T + \mathbf{Q})^{-1}$$

Prediction step:

$$\bar{\boldsymbol{\mu}}_t = \begin{bmatrix} f_r(\boldsymbol{\mu}_{t-1}^r, \mathbf{u}_t) \\ \mu l_{1,t-1} \\ \vdots \\ \mu l_{n_{t-1}} \end{bmatrix}$$

$\mathbf{J}_{x_t}$ The Jacobian matrix of $\mathbf{f}$ w.r.t the state vector.

$\mathbf{J}_{u_t}$ The Jacobian matrix of $\mathbf{f}$ w.r.t the odometry.

$$\bar{\boldsymbol{\Sigma}}_t = \mathbf{J}_{x_t} \boldsymbol{\Sigma}_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$
A diagram consisting of red arrows. One arrow points from the $\bar{\boldsymbol{\Sigma}}_t$ term in the equation to a red dot. From this dot, three arrows point to the $\mathbf{J}_{x_t}$, $\mathbf{J}_{u_t}$, and $\mathbf{R}$ terms in the equation. A fourth arrow points from the red dot to the question text on the right.

If at time step $\mathbf{t}$ we have mapped $\mathbf{n}$ landmarks, what is the dimension of these matrices?

CRICOS No. 00213J

The prediction step

The same matrices from the
localization case

$$\mathbf{J}_{x_t} = \begin{bmatrix} \mathbf{J}_{x_t^r} & 0_{3 \times 2n} \\ 0_{2n \times 3} & I_{2n \times 2n} \end{bmatrix}$$

$$\mathbf{J}_{u_t} = \begin{bmatrix} \mathbf{J}_{u_t^r} \\ 0_{2n \times 2} \end{bmatrix}$$

$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$

$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$

$$\begin{bmatrix} \mathbf{J}_{x_t^r} & 0_{3 \times 2n} \\ 0_{2n \times 3} & I_{2n \times 2n} \end{bmatrix} \begin{bmatrix} \Sigma_{\mathbf{x}^r} & \Sigma_{\mathbf{x}^r l_1} & \dots & \Sigma_{\mathbf{x}^r l_n} \\ \Sigma_{l_1 \mathbf{x}^r} & \Sigma_{l_{11}} & \dots & \Sigma_{l_{1n}} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{l_n \mathbf{x}^r} & \Sigma_{l_{n1}} & \dots & \Sigma_{l_{nn}} \end{bmatrix} \begin{bmatrix} \mathbf{J}_{x_t^r} & 0_{3 \times 2n} \\ 0_{2n \times 3} & I_{2n \times 2n} \end{bmatrix}^T$$

We use the same treatment with this term as well.

CRICOS No. 00213J

The same set of equations

Prediction step:

$$\bar{\mu}_t = f(\mu_{t-1}, \mathbf{u}_t)$$

$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$



Update step:

For each landmark $\mathbf{z}_t^i$ do:

$$\bar{\mu}_t = \bar{\mu}_t + \mathbf{K}_t^i (\mathbf{z}_t^i - h(\bar{\mu}_t, i))$$

$$\bar{\Sigma}_t = (\mathbf{I} - \mathbf{K}_t^i \mathbf{G}_t^i) \bar{\Sigma}_t$$

end

$$\mu_t = \bar{\mu}_t$$

$$\Sigma_t = \bar{\Sigma}_t$$



Similar to the mapping case but with the fact that the pose of the robot is now part of the state vector

$$\mathbf{K}_t = \bar{\Sigma}_t \mathbf{G}_t^T (\mathbf{G}_t \bar{\Sigma}_t \mathbf{G}_t^T + \mathbf{Q})^{-1}$$

CRICOS No. 00213J

The same measurement function we used for localization and for mapping.

$$h(\bar{\mu}_t, i) = \begin{bmatrix} r^i \\ \beta^i \end{bmatrix} = \begin{bmatrix} \sqrt{(x_r - x_{l_i})^2 + (y_r - y_{l_i})^2} \\ \text{atan2}(y_{l_i} - y_r, x_{l_i} - x_r) - \theta_r \end{bmatrix}$$

Range and bearing with respect to robot's own frame of reference at time step t.

Coordinates of landmark i from the vector $\bar{\mu}_t$

The predicted pose of the robot from the vector $\bar{\mu}_t$

The Jacobian matrix of the measurement function

$$\mathbf{G}_t^i = \frac{\partial h(\bar{\mu}_t, i)}{\partial \bar{\mu}_t}$$

$$= \begin{bmatrix} -\frac{x_{l_i} - x_r}{r} & -\frac{y_{l_i} - y_r}{r} & 0 & \dots & \frac{x_{l_i} - x_r}{r} & \frac{y_{l_i} - y_r}{r} & \dots \\ \frac{y_{l_i} - y_r}{r^2} & -\frac{x_{l_i} - x_r}{r^2} & -1 & \dots & -\frac{y_{l_i} - y_r}{r^2} & \frac{x_{l_i} - x_r}{r^2} & \dots \end{bmatrix}$$

zeros



CRICOS No. 00213J

The same set of equations

Prediction step:

$$\bar{\mu}_t = f(\mu_{t-1}, \mathbf{u}_t)$$

$$\bar{\Sigma}_t = \mathbf{J}_{x_t} \Sigma_{t-1} \mathbf{J}_{x_t}^T + \mathbf{J}_{u_t} \mathbf{R} \mathbf{J}_{u_t}^T$$



Update step:

For each landmark $\mathbf{z}_t^i$ do:

$$\bar{\mu}_t = \bar{\mu}_t + \mathbf{K}_t (\mathbf{z}_t^i - h(\bar{\mu}_t, i))$$

$$\bar{\Sigma}_t = (\mathbf{I} - \mathbf{K}_t^i \mathbf{G}_t^i) \bar{\Sigma}_t$$

end

$$\mu_t = \bar{\mu}_t$$

$$\Sigma_t = \bar{\Sigma}_t$$

$$\mathbf{K}_t = \bar{\Sigma}_t \mathbf{G}_t^T (\mathbf{G}_t \bar{\Sigma}_t \mathbf{G}_t^T + \mathbf{Q})^{-1}$$



But what if we see a landmark for the first time?



CRICOS No. 00213J

Landmark initialization

$$\bar{\mu}_t^* = \begin{bmatrix} \bar{\mu}_t \\ l_{new} \end{bmatrix} = \begin{bmatrix} \bar{\mu}_t \\ x l_{new} \\ y l_{new} \end{bmatrix}$$

You already know how to find these as we already encountered them in the mapping case during last lecture.

Simply expand the state vector with the coordinates of the new landmark in the map!

CRICOS No. 00213J

The landmark initialisation function

$$l_{new} = q(\mathbf{z}_t^{new}, \bar{\mu}_t)$$

$$\mathbf{z} = \begin{bmatrix} r \\ \beta \end{bmatrix}$$

Sensor measurement to a never seen before landmark.

$$l_{new} = \begin{bmatrix} x_r + r \times \cos(\theta_r + \beta) \\ y_r + r \times \sin(\theta_r + \beta) \end{bmatrix}$$

The first three components of $\bar{\mu}_t$

CRICOS No. 00213J

The Jacobian of the landmark initialisation function
w.r.t the $\mathbf{z}$

$$\mathbf{L}_z = \frac{\partial q(\mathbf{z}, \bar{\mu}_t)}{\partial \mathbf{z}}$$
$$= \begin{bmatrix} \cos(\theta_r + \beta) & -r \times \sin(\theta_r + \beta) \\ \sin(\theta_r + \beta) & r \times \cos(\theta_r + \beta) \end{bmatrix}$$

$$\mathbf{z} = \begin{bmatrix} r \\ \beta \end{bmatrix}$$

What about the covariance matrix?

$$\bar{\Sigma}_t^* = \begin{bmatrix} \bar{\Sigma}_t & 0 \\ 0 & \mathbf{L}_z \mathbf{Q} \mathbf{L}_z^T \end{bmatrix}$$

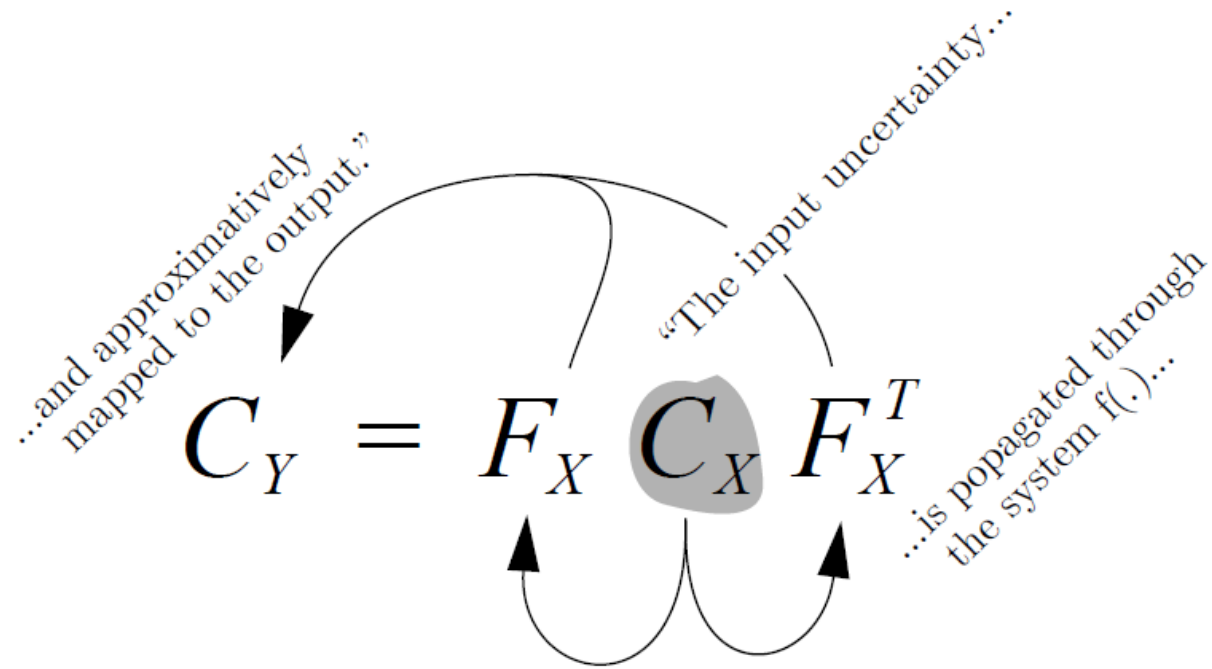
The covariance matrix
expands as well!

Zero matrices!

The covariance of
the sensor noise.

CRICOS No. 00213J

Error Propagation



An Introduction To Error Propagation:
Derivation, Meaning and Examples of Equation
Kai Oliver Arras, EPFL

Putting it all together

1. Move.
2. Perform the **prediction step** which updates the mean and covariance.
3. Make a new Measurement (range and bearing to a landmark).
4. if we have not seen the landmark before:
 - Do landmark **initialization** based on the robot estimated pose and expand the mean and the covariance.

else

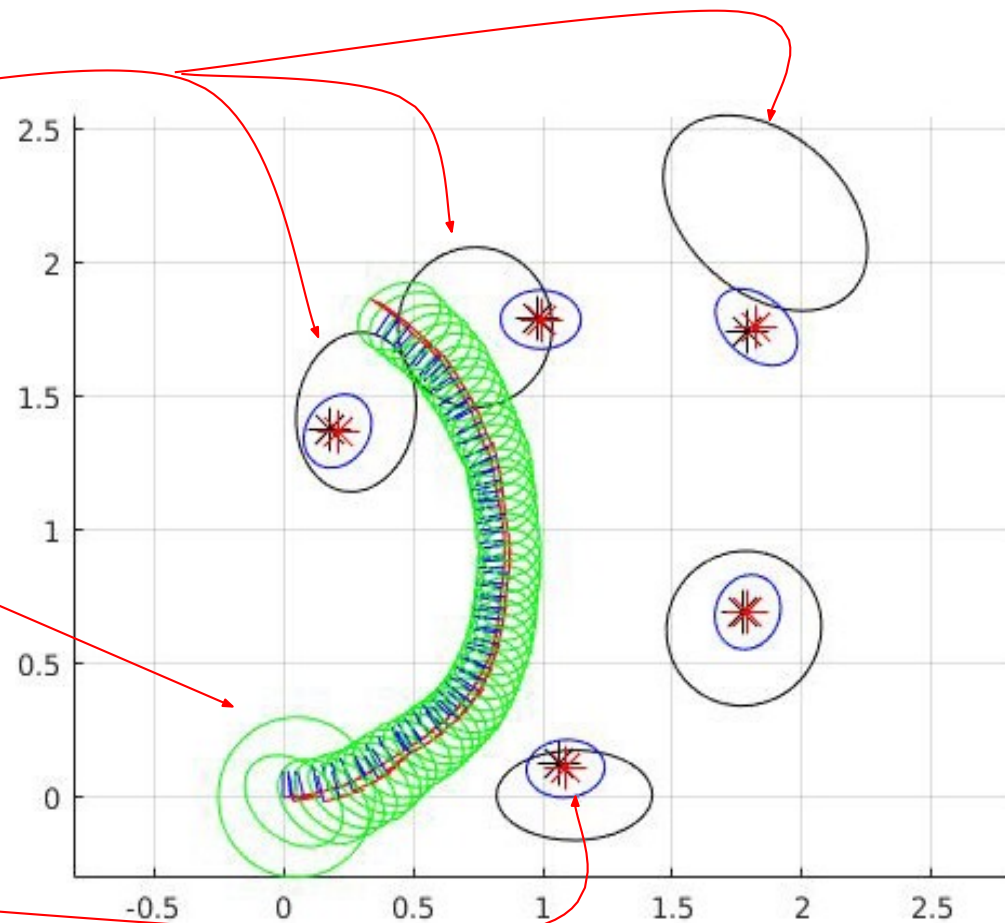
- Perform the **update step** and update the mean and the covariance.
1. Go to 1.

CRICOS No. 00213J

The uncertainty on the positions of the landmarks on initialization.

The uncertainty on the pose of the robot (green ellipses)

The uncertainty on the position of the landmarks after 50 steps (**blue ellipses**)





The strengths and weaknesses of EKF-SLAM

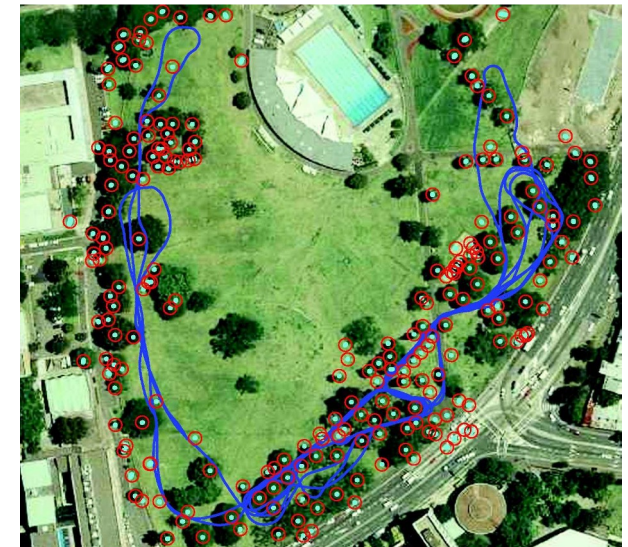
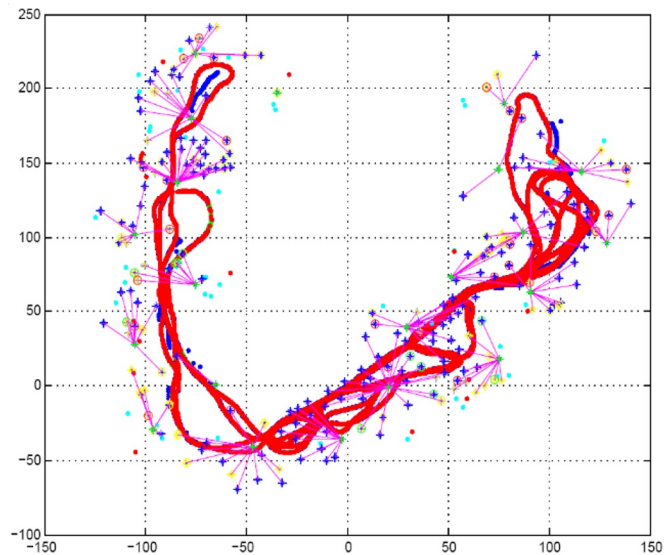
Feras Dayoub

Learning objectives

- The strengths and weaknesses of EKF-SLAM.
- The other flavours of the SLAM algorithm.

EKF-SLAM: Strengths

- Easy to implement.
- Simple parameterization using only the mean and the covariance matrix.



http://www-personal.acfr.usyd.edu.au/nebot/publications/slam/IJRR_slam.htm

Guivant, J., Nebot, E., & Durrant-Whyte, H. (2000, July). Simultaneous localization and map building using natural features in outdoor environments. In *Intelligent Autonomous Systems* (Vol. 6, No. 1, pp. 581-586).

Simultaneous Localization and Map Building Using Natural features in Outdoor Environments

Jose Guivant, Eduardo Nebot, Hugh Durrant Whyte
Australian Centre for Field Robotics
Department of Mechanical and Mechatronic Engineering
The University of Sydney, NSW 2006, Australia
{jguivant/nebot}@mech.eng.usyd.edu.au

Abstract. This work presents efficient algorithms for real time Simultaneous Localization and Map Building (SLAM). The accuracy of the algorithm is investigated with respect to standard localization algorithms using a 1cm kinematic GPS as ground truth. The issue of algorithm convergence when running in large areas of operation and for long period of time is investigated. Experimental results in unstructured environment are presented using more than 200 natural features. The complexity of feature extraction is also investigated with an approach to identify common types of outdoor natural features.

Guivant, J., Nebot, E., & Durrant-Whyte, H. (2000, July).

Simultaneous localization and map building using natural features in outdoor environments. In *Intelligent Autonomous Systems* (Vol. 6, No. 1, pp. 581-586).

2 Navigation System

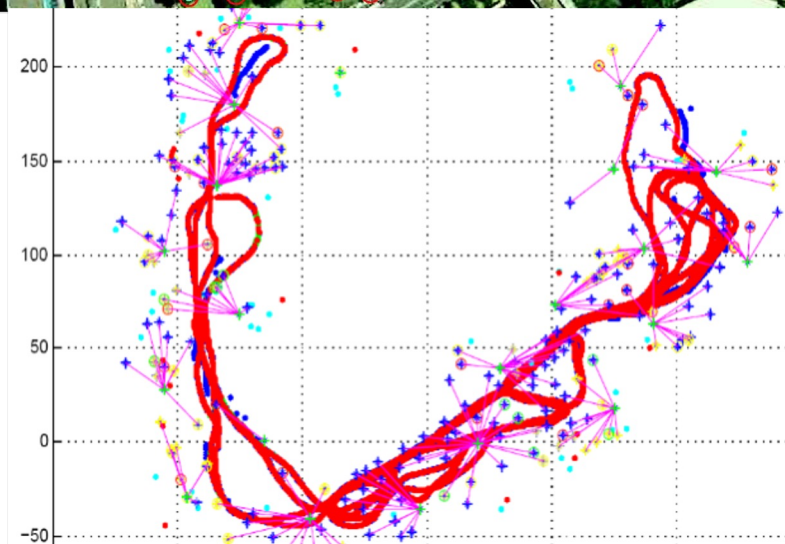
The navigation loop is based on encoders and range / bearing information provided by a laser sensor. A simple kinematic model is used for this experimentation [6]. The trajectory of the centre of the back axle is given by:

$$\begin{bmatrix} \dot{x}_c \\ \dot{y}_c \\ \dot{\phi}_c \end{bmatrix} = \begin{bmatrix} v_c \cdot \cos(\phi) \\ v_c \cdot \sin(\phi) \\ \frac{v_c}{L} \cdot \tan(\alpha) \end{bmatrix} \quad (1)$$

where v_c is the velocity of the centre of the back axle and α is the orientation of the front wheels with respect to the forward direction. The equation that relates the observation with the states is

$$\begin{bmatrix} z_r^i \\ z_\beta^i \end{bmatrix} = h(X, x_i, y_i) = \begin{bmatrix} \sqrt{(x_L - x_i)^2 + (y_L - y_i)^2} \\ \text{atan}\left(\frac{(y_L - y_i)}{(x_L - x_i)}\right) - \phi + \pi/2 \end{bmatrix} \quad (2)$$

where z and $[x, y, \phi]$ are the observation and state values respectively, and (x_i, y_i) are the positions of the beacons or natural landmarks.

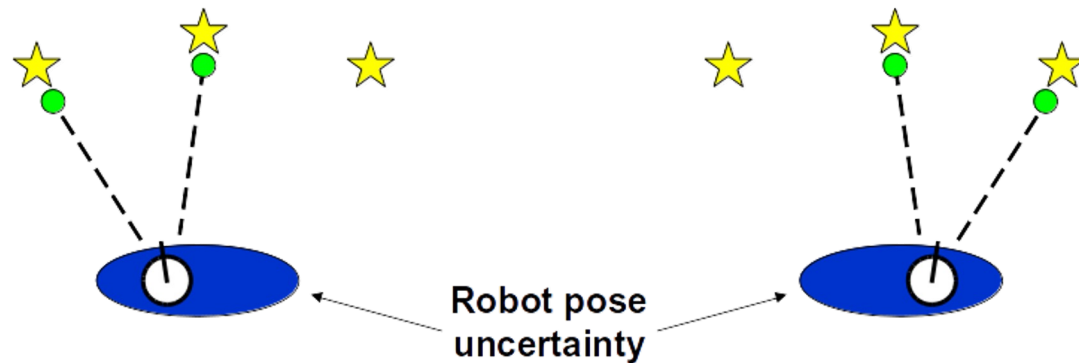


EKF-SLAM: Weaknesses

- The data association problem.
- The global localization problem.
- The linearization problem.
- The non-Gaussian world problem.
- The loop closing problem.
- The increase in complexity with the number of the landmarks problem.

The data association problem

- The problem of determining whether or not two landmarks observed at different points in time correspond to one and the same object in the physical world.
- In this unit, we assumed that the robot can determine the correspondences between its observations and the landmarks with absolute certainty. In reality, this is rarely the case.



The data association problem

-
- Solving this problem in a robust way is crucial for the success of SLAM.
 - **The simplest strategy is finding the maximum likelihood correspondence.**
 - Most deployed SLAM algorithms construct maps with tens of thousands of landmarks, or more.
 - EKF-SLAM can handle small number of landmarks (less than 1000). Above that the landmarks need to be easy to detect and the association with the measurements is not ambiguous.

Maximum Likelihood Data Association

- Compute the probability that the current measurement corresponds to each landmark in the map.
- Then choose the landmark with the highest probability of correspondence and assumes it is the correct association.

for all measurements $z_t^i = [r_t^i \ \phi_t^i]^T$ do

for all landmarks k in the map m do

$$q = (m_{k,x} - \bar{\mu}_{t,x})^2 + (m_{k,y} - \bar{\mu}_{t,y})^2$$

$$\hat{z}_t^k = \begin{bmatrix} \sqrt{q} \\ \text{atan2}(m_{k,y} - \bar{\mu}_{t,y}, m_{k,x} - \bar{\mu}_{t,x}) - \bar{\mu}_{t,\theta} \end{bmatrix}$$

$$H_t^k = \begin{bmatrix} -\frac{m_{k,x} - \bar{\mu}_{t,x}}{\sqrt{q}} & -\frac{m_{k,y} - \bar{\mu}_{t,y}}{\sqrt{q}} & 0 \\ \frac{m_{k,y} - \bar{\mu}_{t,y}}{q} & -\frac{m_{k,x} - \bar{\mu}_{t,x}}{q} & -1 \end{bmatrix}$$

$$S_t^k = H_t^k \bar{\Sigma}_t [H_t^k]^T + Q_t$$

endfor

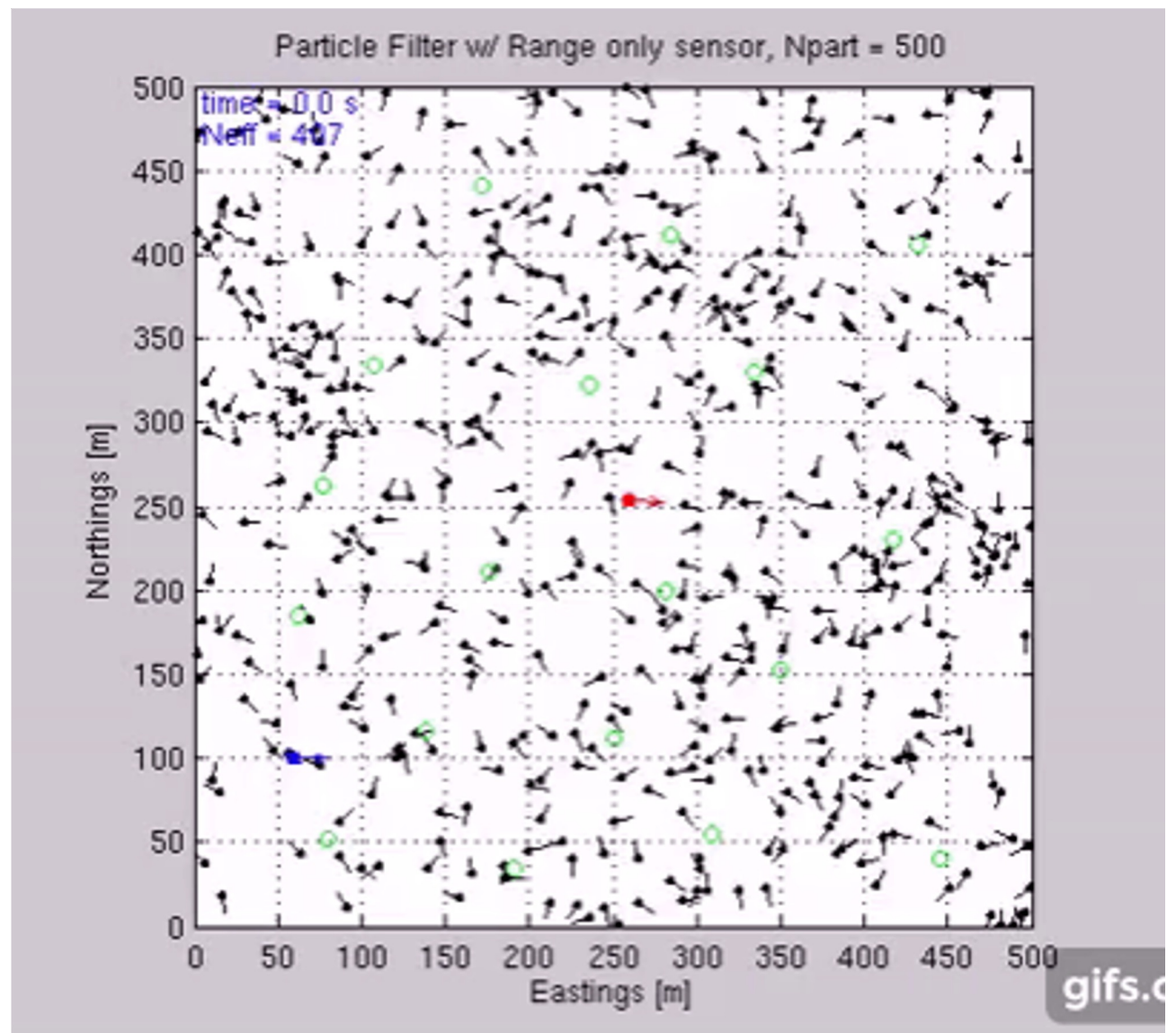
$$j(i) = \operatorname{argmax}_k \det(2\pi S_t^k)^{-\frac{1}{2}} \exp\left\{-\frac{1}{2}(z_t^i - \hat{z}_t^k)[S_t^k]^{-1}(z_t^i - \hat{z}_t^k)^T\right\}$$

endfor

The global localization problem

- The type of localization we did in this unit is called **position tracking**. It assumes that the initial robot pose is known or in the case of SLAM, we define the starting point. We also approximated the **pose uncertainty by an unimodal Gaussian distribution** (i.e it has one peak). This type of localization is called local, since the uncertainty is confined to region near the robot's true pose.
- **Global localization** is the problem where the initial pose of the robot is unknown inside of a known frame of reference. Unimodal probability distributions are usually inappropriate.

The global localization problem



The global localization problem (kidnapped robot)

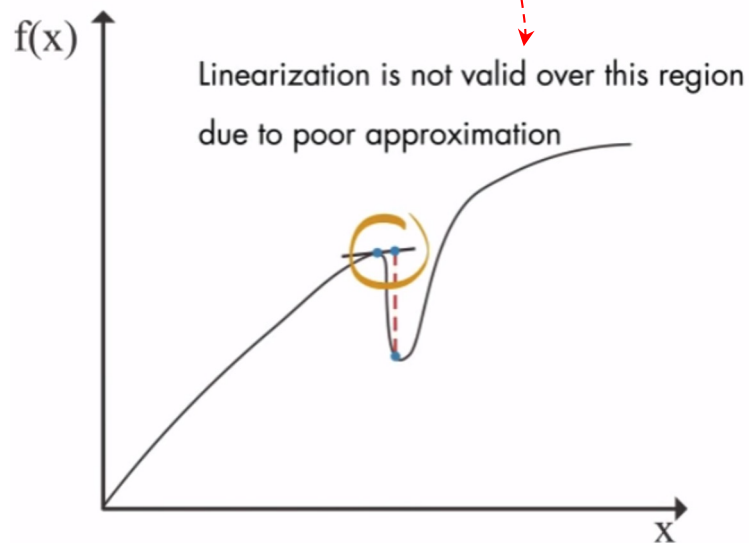
This problem is harder than both local and global localization. In this scenario, the robot can get kidnapped and teleported to some other location.

This problem is difficult because the robot do not know that it is been teleported.

EKF-SLAM can not handle this case which means it can not recover from catastrophic failures. The ability to recover from failures is essential for truly autonomous robots. The kidnapped robot case is a test for the ability of any SLAM algorithm to recover from global localization failures.

The nonlinearity problem

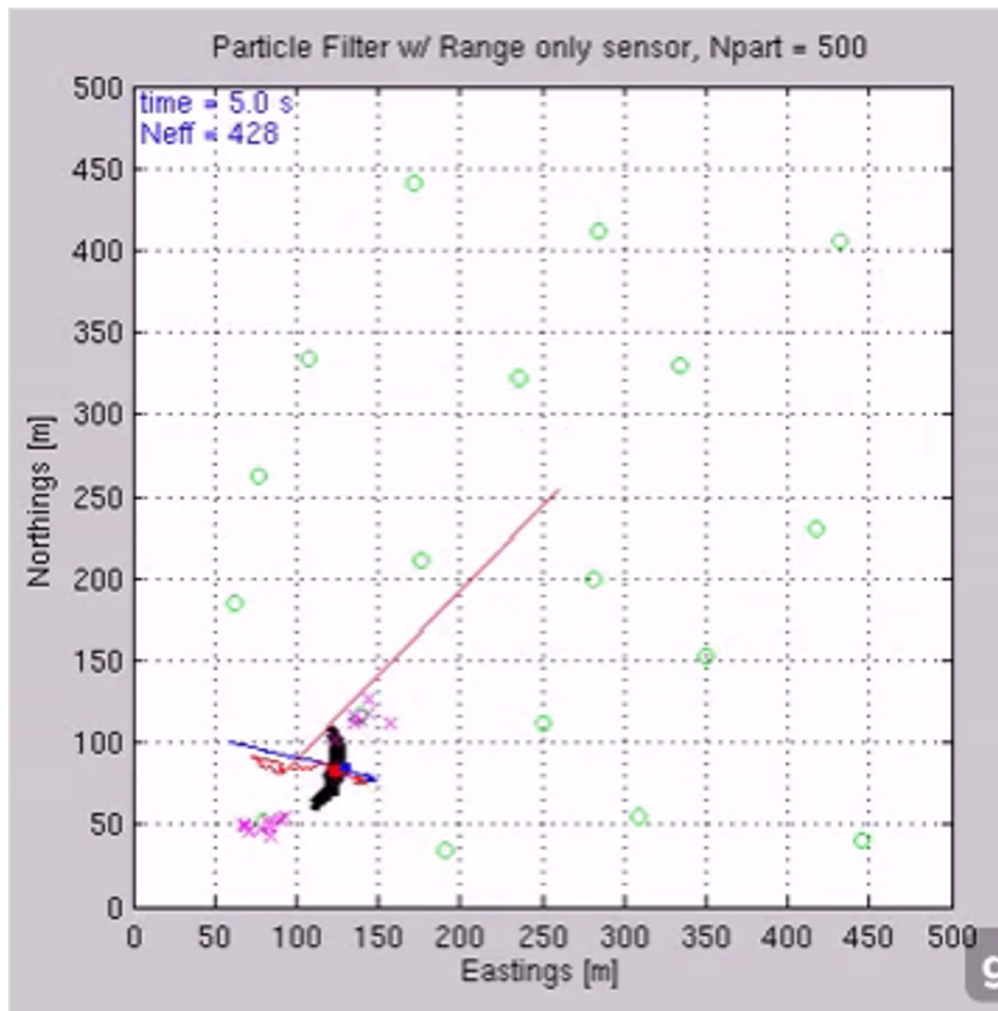
- EKF works well for **moderate nonlinearity** with small uncertainty.



- Linearizations is good in a close range to the linearization point (e.g if the standard deviation on the heading of the robot is larger than 20 degrees, linearization will make the EKF-SLAM fail)

The non-Gaussian world problem

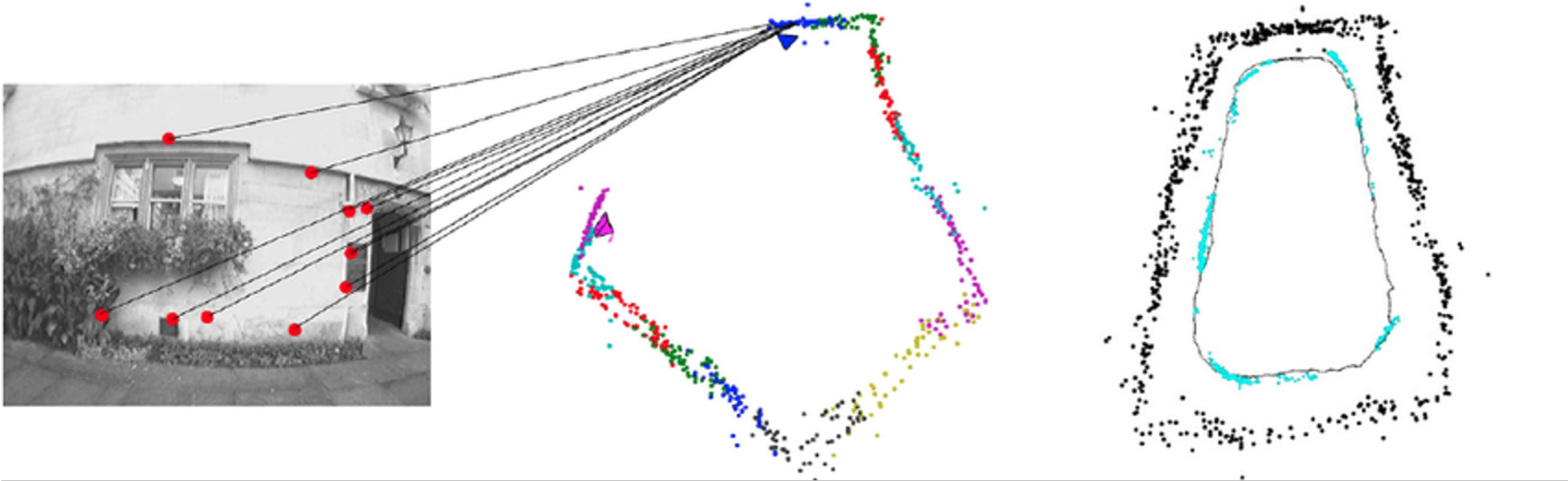
- In this unit we modeled the robot pose, the odometry noise and measurement noise as Gaussians and we used a trick to handle the nonlinearity in the system. This was good enough to perform SLAM in our green arena however EKF cannot be applied when there is high non-linearities involved in the models or when the noise is non-Gaussian.
- A popular alternative to EKF-SLAM are nonparametric filters, one of which is called the Particle filter.
- Particle filters are able to capture the non-Gaussian and multi-modal posterior distributions and also deal with nonlinear models.

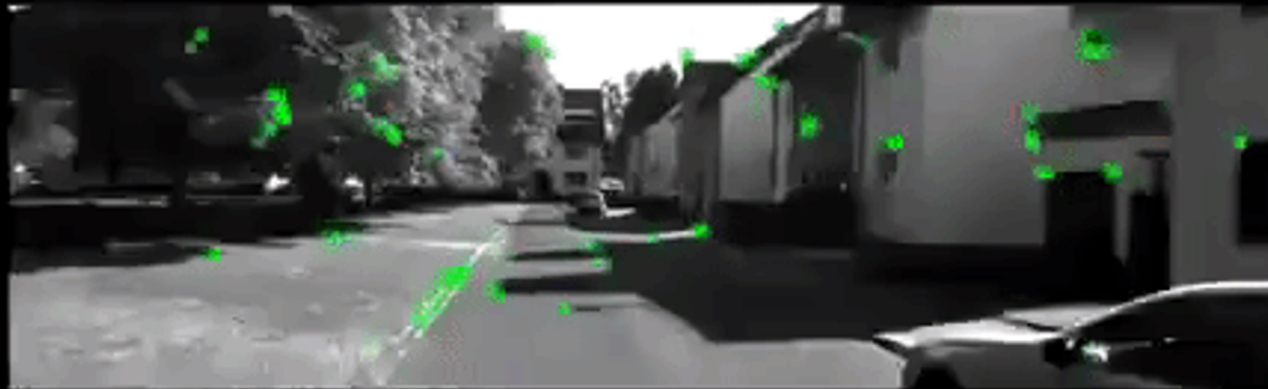


Particle filters are able to capture the non-Gaussian and multi-modal posterior distributions and also deal with nonlinear models

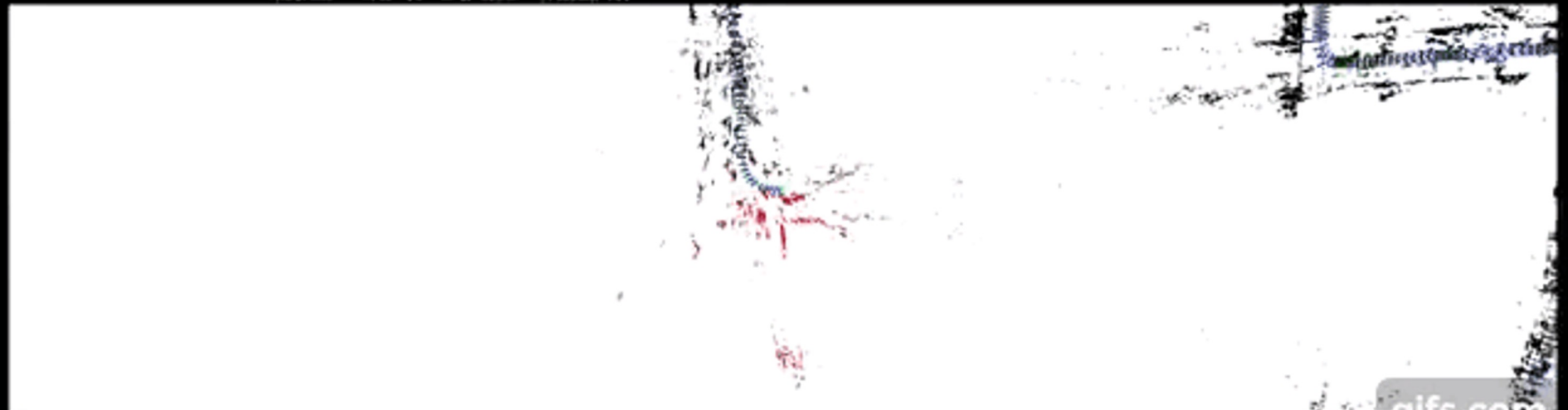
The loop closing problem

- Loop closure happens when the robot traverses a large area and then revisits an already mapped section of it.





THALIMUS - 00:01:12.00 - 00:01:12.00 - 00:01:12.00



gifs.com

The loop closing problem

- Depending on the quality of the odometry and the sensors the system will accumulate error resulting in drifts.
- When returning to a known area, the robot will see the old landmarks but assumes they are new and initializes them in the map.
- The other scenario is when the robot close the loop incorrectly because of an alias environment.
- Wrong loop closures leads to divergence in the EKF-SLAM and the filter will fail.

The increase in complexity with the number of the landmarks problem

- Maintaining a multivariate Gaussian requires time quadratic in the number of features in the map.
- EKF-SLAM is computationally intractable for large maps (> 1000 landmarks).
- One solution is to decompose the problem of building one large map into a collection of smaller maps, which can be updated more efficiently.

Open SLAM project

www.openslam.org

[Home](#)

[News](#)

Projects

[2D-I-SLSJF](#)
[CAS-Toolbox](#)
[CEKF-SLAM](#)
[COP-SLAM](#)
[DP-SLAM](#)
[EKFMonoSLAM](#)
[FalkoLib](#)
[FLIRTLib](#)
[G2O](#)
[GMapping](#)
[GridSLAM](#)
[HOG-Man](#)
[ISAM](#)
[Linear SLAM](#)
[Max-Mixture](#)
[MTK](#)
[OpenRatSLAM](#)
[ORB-SLAM](#)
[OpenSeqSLAM](#)
[ParallaxBA](#)
[Pkg. of T.Bailey](#)
[RGBDSLAM](#)
[Robomap Studio](#)
[RobotVision](#)
[ro-slam](#)
[SLAM6D](#)
[SLOM](#)
[SSA2D](#)
[tinySLAM](#)
[TJTF for SLAM](#)
[TORO](#)
[TreeMap](#)
[UFastSLAM](#)
[vertigo](#)

SLAM Tools

[EVG-Thin](#)
[KLD-Sampling](#)
[People2D](#)

[Submit](#)

[What's SLAM?](#)

[What's SVN?](#)

[People](#)

[Acknowledgment](#)

[Datasets & Links](#)

OpenSLAM
Give your algorithm to the community

What is OpenSLAM.org?

The [simultaneous localization and mapping \(SLAM\)](#) problem has been intensively studied in the robotics community in the past. Different techniques have been proposed but only a few of them are available as implementations to the community. The goal of OpenSLAM.org is to provide a platform for SLAM researchers which gives them the possibility to publish their algorithms. OpenSLAM.org provides to every interested SLAM researcher a [subversion \(svn\)](#) repository and a small webpage in order to publish and promote their work. In the repository, only the authors have full access to the files; other users are restricted to read-only access. OpenSLAM.org does not really aim to provide a repository for the daily development process of early SLAM implementations. Published algorithm should have a certain degree of robustness.

OpenSLAM.org does not force the authors to give away the copyright for their code. We only require that the algorithms are provided as source code and that the authors allow the users to use and modify the source code for their own research. Any commercial application, redistribution, etc has to be arranged between users and authors individually.

[Click to submit](#) your SLAM algorithm.

The OpenSLAM Team
[Cyrill Stachniss](#), [Udo Frese](#), [Giorgio Grisetti](#)

[disclaimer](#)

Translations of this page

- [French](#)

Conclusions and Summary

- Purpose of localisation and mapping for mobile robots: capabilities that underlie navigation, control, manipulation, higher-level objectives of robotic systems
- Different representations for location and maps
- Connections/relationship between good localisation and good mapping
- Role of sensor fusion in localisation and mapping:
 - Sensors on their own don't typically cover all the relevant information, but when fused they do e.g. IMU-GPS